

Technische Hochschule Ingolstadt

Faculty of Electrical Engineering and Information Technology

Study course Automated Driving and Vehicle Safety

Master thesis

Multi-Agent Reinforcement Learning for Behavioral Control in Mixed-Autonomy Unsignalized Intersections

First name and surname: Nguyen, Thanh Tung
Supervisor: Adam, Philip-Roman

issued on: 27.11.2025
submitted on: 27.05.2026

First examiner: Schmidtner, Prof. Dr. Stefanie
Second examiner: Schön, Prof. Dr. Torsten

Specimen for declaration in Accordance with § 30 Abs. 4 Nr. 7 APO THI

Declaration

I hereby declare that this thesis is my own work, that I have not presented it elsewhere for examination purposes and that I have not used any sources or aids other than those stated. I have marked verbatim and indirect quotations as such.

Ingolstadt, _____
(Date)



(Signature)

First name, Surname

Acknowledgments

First and foremost, I would like to express my deepest gratitude to my advising professor, Prof. Dr. Stefanie Schmidtner, for giving me the opportunity to work on such a fascinating and impactful research topic. Her academic vision and trust provided the perfect foundation for this thesis.

I am deeply indebted to my supervisor, Philip-Roman Adam. His incredible mentorship, patience, and expert guidance were instrumental in helping me navigate the complexities of this research. I am profoundly appreciative of the time, effort, and continuous feedback he invested in steering me through this entire journey.

My sincere thanks also go to Dr. Duong-Van Nguyen for his invaluable technical insights. Specifically, I am grateful for his foundational idea of utilizing the Multi-Layer Perceptron (MLP) network, which significantly shaped the architectural direction and experimental phases of my work.

A special and heartfelt thank you goes to Vu Minh Thu Nguyen for her relentless motivation. Her unwavering encouragement and the crucial "push" she provided during the demanding final days of writing were exactly what I needed to cross the finish line.

Finally, none of this would have been possible without the unconditional love, understanding, and support of my family and friends. Thank you for always believing in me, bearing with my long hours of research, and standing by my side throughout my academic journey.

Thanh Tung Nguyen
Ingolstadt, Germany
Mai 27 2026

Abstract

In my work, I propose an advanced model-free multi-agent reinforcement learning method for controlling mixed autonomy traffic in complex, unsignalized four-way (1x1) intersections. While foundational methodologies from Yan und Wu demonstrated near-optimal throughput in decentralized environments, they relied heavily on idealized constraints such as through-traffic-only maneuvers and deterministic vehicle generation. My extended methodology introduces unrestricted free movement, permitting vehicles to execute straight, left, and right turns to accurately reflect the conflict points found in real-world intersections. To simulate dynamic and bursty traffic demand, I utilize stochastic Poisson distribution inflows rather than constant, uniform arrival rates. The model is trained at a 50% autonomous vehicle (AV) penetration rate, evaluating Proximal Policy Optimization (PPO) under two distinct feature extraction architectures: traditional Multi-Layer Perceptrons (MLP) and advanced Transformer-based spatial feature attention. Furthermore, because real-world vehicle-to-vehicle (V2V) communication networks are imperfect, I test the model's operational robustness against communication degradation by introducing simulated packet loss scenarios. I demonstrate that my decentralized control policies can coordinate complex vehicular maneuvers safely and efficiently under highly stochastic constraints, bridging the gap between theoretical simulation and robust physical deployment.

List of figures

1	Network Topology Construction	12
2	Four-way 1x1 traffic network.	14
3	MLP vs. Transformer architectural in comparison.	21
4	Actor and Critic heads.	23
5	Platoon-based observation space	26
6	K-Nearest Neighbor (KNN) Observation Space.	29
7	Baseline: Signal-based baseline method training in SUMO	38
8	Baseline: Priority method training in SUMO	40
9	Raw reward mean of PG Algorithm with RMS Optimizer	41
10	Collisions of PG Algorithm with RMSProp Optimizer	42
11	Policy Loss trajectory using the RMSProp optimizer.	43
12	Policy Loss trajectory using the Adam optimizer.	44
13	Training progression of the standard Policy Gradient (PG) model using ADAM optimizer.	44
14	Comparison of PG Models and ADAM Optimizer with baseline methods .	45
15	Progression of the global mean reward during the initial PPO training phase.	48
16	Throughput and safety metrics during the initial PPO training.	49
17	Trajectory of the PPO policy loss.	50
18	Progression of the global mean reward following the implementation of Leaky ReLU and doubled collision penalties.	51
19	Throughput and collision metrics demonstrating the behavioral shift in- duced by the escalated safety penalty.	52
20	PPO policy loss of the model from 26.11.	53
21	Progression of the global mean reward under Poisson traffic flow.	54
22	Throughput and collision metrics in the stochastic environment.	54
23	Throughput and collision metrics following the integration of turning indi- cators into the observation space and a doubled collision penalty.	56
24	Progression of the global mean reward following the implementation of a 5-action space and an extreme collision penalty coefficient of 50.	58
25	Progression of the global mean reward under an extreme collision penalty coefficient of 500. The reward trajectory collapses toward zero as the agents learn to completely halt all movement.	59
26	Comparative scatter plot of network throughput and safety for the standard Policy Gradient (PG) and Proximal Policy Optimization (PPO) algorithms.	61
27	Comparison of the last PPO MLP experiment to Baseline.	62
28	Training result of the initial Embedded Transformer architecture.	63
29	Training result of the 05.02 trainng.	64
30	Heatmap comparison of performance metrics for the Platoon observation space Transformer PPO methodology against the established baseline. . . .	65
31	Traing result of 13.02, model using KNN observation space.	66
32	Comparative heatmap analysis evaluating the boundary performance of the K-Nearest Neighbor (KNN) Transformer PPO policy against established baselines.	67

33	Evaluating the optimal performance checkpoints of the standard Policy Gradient (PG), Platoon Transformer PPO, and the KNN Transformer PPO architectures.	68
34	Evaluation of hourly network throughput (hourly outflow) for the MLP PPO architecture across 16 varying inflow configurations, model 220. . . .	69
35	Evaluation of collision rates for the PPO MLP architecture, model 220. . .	70
36	Throughput evaluation for the deepened MLP PPO architecture, model 300.	70
37	Collision evaluation for the MLP PPO architecture, model 300.	70
38	Cross-sectional evaluation of hourly network throughput (outflow) utilizing the Embedded Transformer with a Platoon-based observation space, model 65.	71
39	Collision rate evaluation for the Platoon-based Transformer architecture, model 65.	71
40	Throughput evaluation matrix for the K-Nearest Neighbor (KNN) Transformer architecture, model 1000.	72
41	Collision evaluation matrix for the KNN Transformer architecture, model 1000.	72
42	Throughput evaluation matrix for the K-Nearest Neighbor (KNN) Transformer architecture, model 350.	72
43	Collision evaluation matrix for the KNN Transformer architecture, model 350.	73
44	Comparative performance evaluation of the Multi-Layer Perceptron (MLP) and Embedded Transformer architectures under a symmetric 700×700 vehicular flow configuration.	73
45	Evaluation of Model 220 (derived from the 12.12 tuning phase).	75
46	Evaluation of Model 300 (derived from the 17.12 tuning phase).	76
47	Evaluation of Model 65 (derived from the 02.02 tuning phase).	76
48	Evaluation of Model 100 (derived from the 13.02 tuning phase).	77
49	Evaluation of Model 350 (an advanced checkpoint from the 13.02 tuning phase).	77
50	Time-space (trajectory) diagram illustrating the kinematic behavior of the Transformer architecture (Model 350) under a symmetric 700×700 flow configuration.	78
51	Performance evaluation of the Embedded Transformer architecture (Model 350) subjected to simulated V2V communication degradation.	79

List of tables

1	Edge Attributes and Kinematic Constraints	13
2	Human Vehicle Kinematic Parameters and Driver Model Settings	16
3	Autonomous Vehicle Parameters	16
4	Summary of Experimental Configurations	36
5	Performance Metrics for the Equal-Phase Baseline (700×700 Inflow) . . .	37
6	Performance Metrics for the Max-Pressure Baseline (700×700 Inflow) . .	38
7	Performance Metrics for the Priority(Vertical) Baseline (700×700 Inflow)	39
8	Performance Metrics for the Priority(Horizontal) Baseline (700×700 Inflow)	39
9	Overview of Experiments and Configurations - PPO MLP Feature extractor	47
10	Checkpoint for evaluation	68
11	Inflow configuration	69

Table of contents

Acknowledgments	II
Abstract	III
List of figures	V
List of tables	VII
1 Introduction	1
1.1 Problem Statement	1
1.2 Research Gaps	2
1.3 Research Objectives and Questions	5
2 Literature Review	6
2.1 Fundamental	6
2.2 Traffic control at intersections	7
2.3 Autonomous vehicle intersection management	8
2.4 Vehicle to Vehicle (V2V) communication	10
3 Methodology	12
3.1 Simulation Framework and Traffic Scenarios	12
3.1.1 Simulation Environment: SUMO and TraCI	12
3.1.2 Network Topology Construction	12
3.1.3 Vehicle Kinematics and Mixed Autonomy	15
3.2 Reinforcement Learning Algorithm	15
3.2.1 Policy Gradients (REINFORCE)	15
3.2.2 Proximal Policy Optimization (PPO)	17
3.2.3 Optimization Strategies	19
3.3 Neural Network Architecture	21
3.3.1 Shared Feature Extractor: From MLP to Transformer	21
3.3.2 Actor and Critic Heads	23
3.4 Markov Decision Process	24
3.4.1 State Space and Observation Space	24
3.4.2 Observation Space	25
3.4.3 Action Space	31
3.4.4 Reward Function	32
3.5 Experimental Design	34
3.5.1 PG Algorithm and Optimizer	35
3.5.2 PPO Algorithm and Hyperparameter Tuning	35
3.5.3 PPO Algorithm and Transformer Architecture Integration	35
3.5.4 Summary of Key Experimental Configurations	36
4 Results and discussion	37
4.1 Baseline Experiments	37

4.2	Experiments	39
4.2.1	PG Algorithm with ADAM/RMSProp Optimizer	39
4.2.2	PPO Algorithm, ADAM Optimizer and MLP Feature extraction . .	46
4.2.3	PPO Algorithm, ADAM Optimizer and Embedded Transformer Feature extraction	62
4.3	Evaluation	68
4.4	Performance Analysis	73
4.4.1	Architecture Analysis	73
4.4.2	Safety and Throughput Analysis	74
4.4.3	Performance Analysis Across Diverse Flow Configurations	74
4.5	Robustness	78
5	Conclusion and Future Work	81
5.1	Summary	81
5.2	Future Work	82
	References	VI

1 Introduction

The global evolution of the fifth generation of cellular networks (5G) [1] and ultra reliable low-latency communication (URLLC)[3] protocols represents an important change in capability in the architecture of intelligent transportation systems (ITS) [16]. The physical deployment of these advanced communication frameworks establishes the necessary technological foundation for connected and autonomous vehicles to function not as isolated, sensor-dependent entities navigating, but as highly integrated nodes within a dynamic, interconnected spatial temporal network. This evolution toward comprehensive connected autonomy introduces the huge potential for the algorithmic, system-level control of vehicular dynamics. The objectives of this technological transition is to reduce of urban traffic congestion, reduce of emissions, including carbon, and the maximization of vehicular and pedestrian safety.

However, the transition from transportation ecosystems that dominated by human control vehicles to fully autonomous, deterministic transportation networks is in spans of multiple decades. For the foreseeable future, physical infrastructure will be defined by “mixed autonomy” [53] traffic. This is a highly heterogeneous operational environment where varying penetration rates of autonomous vehicles must coexist and dynamically interact with human-driven vehicles. Human driving behavior is chaotic and difficult to model precisely, characterized by highly variable reaction times, bounded driver rationality, reaction to distraction, and imperfect kinematic execution. Consequently, managing urban intersections, which constitute the most critical, highly congested, and statistically dangerous bottlenecks in any municipal traffic grid, under mixed autonomy conditions requires high level control that capable of predicting human unpredictability and guild them while continuously optimizing systemic flow.

1.1 Problem Statement

Urban intersections currently rely almost on heavy, fixed physical infrastructure. More over, fixed-cycle or adaptive traffic signal control systems such as the widely deployed SCOOT [17] or SCATS [29] frameworks. While these control systems have provided a reliable baseline of safety for human drivers over the past century, they are constrained by macroscopic traffic approximations. Signalized intersections lack the granular, localized intelligence required to optimize the specific kinematic trajectories of individual vehicles. This fundamental limitation leads to unnecessary vehicle idling, not optimal acceleration and deceleration profiles, and a rigid, heavily bottle-necked intersection throughput that cannot dynamically adapt to micro fluctuations in traffic demand.

The future of intersection control can be see with infrastructure free coordination. By relying on V2V communication protocols [48] supported by 5G networks, an intersection can theoretically be managed without the presence of traditional traffic lights. In this decentralized framework, autonomous vehicles actively process the spatial temporal states of surrounding human and autonomous drivers, coordinating their localized acceleration, deceleration, and yaw rates to weave through conflict points without stopping. Operating an intersection without infrastructure saves vast amounts of federal resources by eliminating the maintenance, electrical power consumption, and capital expenditure associated

with physical signal hardware. Furthermore, by replacing discrete red and green temporal signal phases with continuous, localized trajectory adjustments, such systems promise vastly enhanced vehicular flow and a reduction in collision probabilities.

The conceptual foundation for this infrastructure free control within a mixed autonomy context is decentralized partially observable Markov Decision Processes (Dec-POMDP) [4] and multi-agent reinforcement learning [13]. A foundational study by Yan and Wu [54] successfully demonstrated near-optimal intersection throughput using decentralized multi-agent reinforcement learning without reliance on centralized infrastructure. Their work proved that by sharing a neural network policy among an arbitrary number of controlled autonomous vehicles, the network could organically learn to coordinate individual agents. The models learned to exhibit traffic-signal-like platooning behaviors, achieving near-optimal throughput with an autonomous vehicle penetration rate as low as 33% to 50%.

However, the current generation of these infrastructure-free MARL models is constrained by heavily idealized boundaries. The central problem addressed in transportation research is no longer whether infrastructure-free intersection management is theoretically possible, that threshold has been crossed, but whether the underlying algorithms can be made robust, realistic, and safe enough for actual deployment in physical infrastructure. The extension of these frameworks must deal with highly complex, unrestricted vehicle kinematics, stochastic traffic inflows, dynamically changing state dimensions, and the ever-present threat of telecommunication degradation. The core operational problem requires evolving MARL frameworks from theoretical to physical robustness, ensuring that the theoretical resource savings and safety enhancements are guaranteed under adversarial real-world conditions.

1.2 Research Gaps

Despite the mathematical, theoretical, and empirical simulation success of MARL in mixed autonomy environments, particularly the foundation methodologies establishing near-optimum throughput without explicit reward shaping, the transition from closed-loop simulated grids to open-world physical infrastructure is obstructed by multiple critical vulnerabilities. An exhaustive review of the prevailing literature reveals four fundamental research gaps that must be systematically resolved before real world deployment becomes viable.

Free Movement

The existing body of work, particularly the foundation framework established by Yan and Wu [54], assumes an artificially restrictive kinematic environment to make the algorithmic easy to convergence. In their mathematical simulation modeling, the intersection is defined exclusively by through traffic. Vehicles are strictly constrained to move in straight lines across either two-way (2x1, 3x3 grids) or four-way (1x1 grid) intersections, eliminating the possibility of executing left or right turns. Furthermore, this foundation baseline focused explicitly on near-optimum throughput without considering collision penalties in its earliest iterations, effectively eliminating the unpredictability inherent in real-world traffic dynamics.

While this straight-only simplification successfully reduces the dimensionality of the state-action space and allows for the rapid convergence of policy gradients during training, it effectively strips away the most dangerous, complex, and unpredictable elements of urban intersection traffic. Left turns across opposing traffic flows represent the most statistically dangerous maneuver in intersection management, introducing highly complex, time-dependent spatial conflict points that span multiple lanes. Real-world intersections possess intricate topologies. For instance, standard experimental models implemented in simulators like SUMO often feature up to 12 incoming lanes, where the leftmost lanes are restricted to turning left, middle lanes dictate straight movement, and rightmost lanes permit straight or right-turn movements.

When vehicles are allowed true free movement, meaning traffic can turn left, turn right, or proceed straight at will, the conflict topology of the intersection transitions from a simple perpendicular grid to a highly intricate, dynamic braid of potential collisions. Addressing this gap requires, I introduce free movement into the mixed autonomy simulation. The reinforcement learning policy must be subjected to high-dimensional state spaces where it must learn to predict the yaw rate, lateral acceleration, and intended trajectories of both human-driven and autonomous agents. By eliminating the artificial predictability utilized in prior studies, the resulting models will reflect the true chaotic nature of urban intersections.

Traffic Flow Dynamics: From Deterministic to Poisson Distributions [5]

A second limitation in current mixed autonomy intersection models is the reliance on deterministic and consistent traffic inflows. In the baseline literature, inflow configurations are structured: vehicles enter the simulation at fixed rates (e.g., specific combinations of horizontal inflow f_H and vertical inflow f_V , ranging uniformly from 400 to 1000 vehicles per hour per lane) and are instantiated with regular spacing. Furthermore, autonomous vehicles are distributed deterministically.

Real-world traffic demand is rarely deterministic. Vehicle arrivals at an intersection fluctuate dramatically based on upstream traffic signals, varying driver speeds, and stochastic localized events. Consequently, models trained on constant inflows inherently over-fit to specific spatial temporal frequencies. They often fail when subjected to the burst, clustered nature of physical traffic.

Integrating Poisson processes ensures that the reinforcement learning agent is subjected to realistic, dynamic clustering of vehicles, ranging from sudden, dense multi-vehicle platoons to empty phases with less vehicles. Training under Poisson-distributed stochasticity forces the policy to learn generalized, adaptive coordination strategies rather than memorizing deterministic arrival sequences. This shift is critical for achieving robust generalization across diverse, dynamic traffic conditions.

Algorithmic Feature Extraction: MLP [34] and Transformer Architectures [49]

The algorithmic architectures deployed in early mixed autonomy studies predominantly rely on conventional Multi-Layer Perceptrons (MLPs) for state feature extraction. In such models, the observation space o^i for an autonomous vehicle i is defined by extracting the speed and distance of a fixed, predetermined number of nearby vehicles. These localized

features are flattened into a one-dimensional vector and passed through a basic neural network—typically consisting of two fully-connected hidden layers of size 64.

This architectural choice creates a lot of mathematical limit. Traffic environments are fundamentally multi-agent, spatial temporal graphs where the number of interacting nodes (vehicles) changes dynamically from millisecond to millisecond. MLPs struggle with this because they require fixed-size input tensors and are highly sensitive to the order in which vehicle features are presented [43]. They lack permutation invariance. If a new vehicle enters the observation zone, or if the sequential order of vehicles in the input matrix shifts, the MLP must essentially relearn the state representation, leading to low sample efficiency and poor environmental adaptability.

Advancing the methodology requires a transition to Transformer-based spatial temporal feature extraction architectures. By utilizing Self-Attention mechanisms, Transformers can dynamically weigh the relative importance of all surrounding vehicles, regardless of their total number or their position in the input matrix. For example, integrating models like the spatial temporal graph-based transformer (STGT)—which combines Graph Convolutional Networks (GCN) [50] to extract spatial node information with Transformer blocks to exploit temporal data, performance better in dynamic environments. In architectures like the SPformer [14], the vehicle state representation containing multi modal information is added to physical positional encodings and fed into the transformer block alongside a policy token. The attention mechanism allows the ego-vehicle to selectively focus on a high-speed human-driven vehicle approaching a blind spot, effectively filtering out irrelevant data from departing traffic.

Communication Vulnerabilities: Packet Loss

The most critical, physically dangerous vulnerability in the current paradigm of MARL for connected vehicles is the assumption of flawless V2V communication. Current models assume that the state space observations, specifically the precise instantaneous distances, accelerations, and speeds of all nearby vehicles, are perfectly transmitted and flawlessly received by the ego-vehicle at every single simulation step.

In physical reality, communication networks operating in the 5GHz spectrum (DSRC) or cellular bands (C-V2X) [20] are subjected to severe, unavoidable environmental constraints. Handover-induced packet loss, signal attenuation through urban canyons, severe latency spikes, channel congestion, and malicious cyber-security interference, such as denial-of-service (DoS) [46] attacks, replay attacks on safety measurement, and GNSS spoofing, routinely disrupt data pipelines. Empirical studies indicate that real-world packet loss probabilities (p_{loss}) can easily reach or exceed 0.05 (5%) [55] under congested urban conditions. When a MARL agent that relies entirely on perfect mathematical knowledge of a neighboring vehicle’s acceleration suddenly experiences a data dropout, the policy becomes blind. This lack of a common protocol and sudden state occur can leads to unwanted collision trajectories.

Future reinforcement learning architectures must be explicitly trained to operate under severe partial observability caused by this probabilistic packet loss. The system must be capable of working even when V2V data is stale, missing, or compromised. By testing

with packet loss scenario, the system ensures a safe collision avoidance, even in the case of network partially failure.

1.3 Research Objectives and Questions

To fundamentally resolve the mentioned limitations and advance mixed autonomy intersection management toward viable physical deployment, my research and model deployment must be guided by four objectives.

Multi-Objective Optimization for Flow and Safety

The primary objective is to train the RL model to achieve the optimal mathematical balance between maximizing global hourly throughput (outflow) and strictly minimizing the collision rate. This goal required the compound reward function. Rather than relying solely on throughput metrics, the reward definition must aggressively penalize kinematic overlap (collisions) while rewarding the successful, rapid traversal of stochastic, free-moving traffic streams.

Implementation and Tuning of Proximal Policy Optimization (PPO) [38]

The methodology must actively move away from basic REINFORCE toward the Proximal Policy Optimization (PPO) algorithm. PPO limits the size of the policy update via a clipped surrogate objective function. This clipping mechanism prevents destructively large policy updates, ensuring stable, monotonic convergence even in the highly chaotic, high-dimensional mixed-autonomy environments generated by free-turning traffic and Poisson inflows. Actively hyperparameter tuning, adjust learning rates, network architecture, entropy bonuses, and clipping thresholds (ϵ), is required to adapt the PPO architecture to the unique complexities of urban traffic flow.

Comparative Analysis of Feature Extraction Methodologies

A critical objective is to verify the performance between traditional MLP-based policies and advanced Transformer-based spatial feature extractors. The comparison must be evaluated. My hypothesis is that Transformer-based models, by leveraging the attention mechanisms, may demonstrate greater resilience in some scenarios to new vehicles into the observation space compared to static MLPs.

Robustness Testing of Packet Loss

The finalized models must be tested by introducing variable probabilities of transmission packet loss (p_{loss}) into the communication pipelines during both training and execution. Utilizing the Centralized Training and Decentralized Execution (CTDE) paradigm [28], the models will be exposed to interference and congestion effects simulating. The ultimate aim is to validate the robustness of the model and confirm the safety of operation.

2 Literature Review

2.1 Fundamental

To form the control and optimization of mixed autonomy traffic, it is required to model the baseline kinematic dynamics of vehicle movement. Traffic flow theory generally divided into two primary scales of observation: macroscopic models, which treat traffic as a continuous compressible fluid, and microscopic models, which simulate the discrete kinematic decisions of individual drivers based on ordinary differential equations.

Macroscopic Traffic Flow: The Lighthill-Whitham-Richards (LWR) Model [26] [33]

Macroscopic traffic flow theory abstracts individual, discrete vehicles into continuous, aggregate metrics defined over space and time: density (ρ), flow (q), and spatial mean speed (v). The foundational and most enduring framework for this abstraction is the Lighthill-Whitham-Richards (LWR) [26] [33] model, which was independently developed and formalized by Lighthill and Whitham in 1955, and subsequently refined by Richards in 1956. The LWR model relies fundamentally on the physical principle of mass conservation, positing that vehicles are neither created nor destroyed along a homogeneous roadway segment devoid of entry or exit ramps.

The LWR model, while highly robust for analyzing macroscopic phenomena such as queue formation, capacity drops at urban intersections, operates strictly under the assumption of continuous equilibrium. As result, it struggles to capture some non-equilibrium dynamics, transient dynamics of human driving behavior, such as stop-and-go oscillations, bounded physical acceleration, and individual reaction times. To accurately address the stochasticity and highly localized vehicle-to-vehicle interactions inherent in mixed autonomy environments, the utilization of microscopic models becomes strictly required.

Microscopic Traffic Flow: The Intelligent Driver Model [45]

Microscopic models abandon the fluid dynamic abstraction in favor of tracking the continuous time-space trajectories of individual vehicles. These car-following models calculate the instantaneous acceleration of a given vehicle as a complex function of its current kinematic state and its longitudinal interaction with the immediately preceding vehicle. Among the most widely deployed, thoroughly validated, and stable car-following models in modern traffic engineering is the Intelligent Driver Model (IDM), introduced by Treiber, Hennecke, and Helbing in 2000.

Unlike earlier mathematical formulations, such as the Gipps model, which utilize disjointed piecewise functions to govern different driving regimes (e.g., cruising versus emergency braking), the IDM is a time-continuous optimal velocity model. It unifies free-road acceleration behaviors and interaction-based deceleration behaviors into a single, smooth, differentiable framework.

The IDM is highly prized in advanced computational simulation environments because it typically avoids collisions under nominal conditions, and its internal parameters possess direct, highly intuitive physical interpretations. By selectively altering the parameters T , a , and b , the IDM can simulate a vast spectrum of human driving typologies, rang-

ing from highly aggressive, tight following drivers to extremely cautious, slow-reacting individuals. In the context of mixed autonomy simulations, the IDM is almost universally utilized to dictate the baseline behavior of the uncontrolled human-driven vehicles. It serves as the stochastic, highly constrained environment against which autonomous vehicle reinforcement learning policies must operate, interact, and ultimately optimize.

2.2 Traffic control at intersections

Decades prior to the concept of autonomous intersection management, traffic engineering is to mitigate urban intersection bottlenecks through dynamically responsive signal control infrastructures. The transition from static, fixed-time signal scheduling to real-time adaptive network control in the late 20th century marked the first successful algorithmic attempts to achieve network-wide traffic optimization. Two highly sophisticated, proprietary systems emerged as the global standards for adaptive control, and their underlying architectures still dictate the flow of modern cities today: SCOOT and SCATS.

SCOOT: Split Cycle Offset Optimization Technique [17]

Developed through extensive research by the United Kingdom’s Transport and Road Research Laboratory, the Split Cycle Offset Optimization Technique (SCOOT) was formally detailed by Hunt et al. (1981). SCOOT operates on the fundamental principle of minimizing a continuously updated, composite performance index that calculates the aggregate vehicle delays and the total number of vehicular stops across an entire controlled traffic network.

To achieve this, SCOOT relies heavily on the installation of inductive loop detectors placed strategically at the far upstream ends of intersection approach links. These detectors measure the instantaneous flow of arriving traffic long before it reaches the intersection. These precise upstream measurements are fed into a central computer that utilizes them to synthesize Cyclic Flow Profiles (CFPs). The CFPs are complex mathematical models that predict the exact arrival times, densities, and dispersion characteristics of vehicle platoons as they approach the downstream stopline.

SCOOT’s reliance on frequent, microscopic adjustments, often implemented on a second-by-second basis, allows it to respond and organically to transient traffic fluctuations without triggering massive, highly disruptive phase shifts that could destabilize the network.

SCATS: Sydney Coordinated Adaptive Traffic System [29]

Developed contemporaneously by the Roads and Traffic Authority of New South Wales under the direction of P. Lowrie (1990), the Sydney Coordinated Adaptive Traffic System (SCATS) takes a fundamentally different architectural and philosophical approach to network optimization. Rather than predicting platoon arrivals via upstream detection, SCATS relies on real-time, macroscopic evaluations of traffic volume and density directly at the point of conflict.

Unlike SCOOT, SCATS heavily utilizes stop line detectors positioned directly at the intersection. These detectors measure the macroscopic degree of saturation, defined strictly as the ratio of effectively utilized green time to the total available green time for a specific

phase. SCATS organizes vast urban networks into tightly interconnected subsystems of intersections that share a common, synchronized cycle length.

At the conclusion of each signal cycle, the system evaluates the traffic conditions and executes a “voting” algorithm to determine whether the regional cycle length should be incrementally increased, decreased, or maintained at its current duration. To manage the individual green splits at each intersection, SCATS does not calculate split times on a continuous sliding scale; instead, it selects from a highly curated, pre-defined library of historical phase plans that dictate the proportional allocation of the cycle time. By continuously matching the optimal pre-defined phase plan and the dynamically adjusted cycle length to the empirically measured degree of saturation, SCATS can manage massive macroscopic network demand without requiring the computationally intensive flow profiling used by SCOOT.

While SCOOT and SCATS outperform an isolated, fixed-time signal systems, they are fundamentally reactive and deeply constrained by their reliance on macroscopic, aggregate data points.

Both systems are entirely clear to the trajectory-level intents, kinematic capabilities, and communication potentials of individual vehicles. In a mixed autonomy, where autonomous vehicles can share high-resolution state data and execute highly precise maneuvers, utilizing exclusively macroscopic control systems artificially limits the potential throughput and efficiency gains achievable through vehicle-to-infrastructure (V2I) and vehicle-to-vehicle (V2V) integration.

2.3 Autonomous vehicle intersection management

To overcome the limitations of legacy adaptive systems, recent theoretical research has explored highly dynamic algorithmic controls and advanced artificial intelligence methodologies. These approaches effectively bridge the critical gap between aggregate macroscopic signal control and microscopic vehicular actuation.

The MaxPressure Algorithm [47]

A pivotal and highly influential advancement in queue-stabilizing intersection signal control is the MaxPressure algorithm, detailed by Varaiya in 2013. Derived directly from the mathematical principles of back-pressure routing algorithms heavily utilized in wireless telecommunication networks, MaxPressure treats the urban traffic network as a complex series of interconnected queuing nodes. Its primary mathematical objective is to maximize global network throughput and maintain queue stability without requiring any prior historical knowledge of origin-destination matrices, routing choices, or macroscopic traffic demand profiles.

The algorithm functions by defining the mathematical “pressure” of a specific traffic phase. This pressure is calculated as the precise difference between the queue lengths of the incoming links (upstream of the intersection) and the available spatial capacity of the corresponding outgoing links (downstream of the intersection). Mathematically, the pres-

sure P for a vehicular movement from an incoming link i to an outgoing link j is strictly proportional to the queue density on i minus the queue density on j .

At every defined decision time step, the intersection controller evaluates all possible, non-conflicting legal phase combinations. It calculates the aggregate pressure for each combination and deterministically actuates the specific phase that exerts the maximum total pressure. By strictly prioritizing phases with the highest differential pressure, the MaxPressure algorithm dynamically pushes vehicles out of highly congested incoming links and into less occupied downstream spaces. This greedy approach effectively prevents localized gridlock, mitigates spillback, and ensures a provable stabilization of network-wide queues under any stable demand profile.

Reservation-Based Autonomous Intersection Management (AIM) [7]

The foundational and most highly cited AIM protocol was established by Dresner and Stone in 2008. AIM replaces the physical traffic light with a centralized, highly computational Intersection Manager (IM). Under the AIM protocol, as an autonomous vehicle physically approaches the intersection limits, it transmits a highly detailed, low-latency reservation request packet to the IM. This packet contains exact kinematic details, including the vehicle's current velocity, intended trajectory, projected arrival time at the conflict zone, and precise physical dimensions.

Upon receiving the request, the IM computationally projects this specific space-time trajectory through the intersection's internal geometry, which is discretized into a high-resolution grid of independent "conflict zones". If the requested space-time trajectory does not spatially or temporally overlap with any previously granted vehicle reservations, the IM immediately approves the request. The AV is then authorized to proceed through the intersection at its requested speed without decelerating. Conversely, if a mathematical conflict is detected within any conflict zone grid, the request is summarily rejected. The AV must then safely decelerate and iteratively submit new trajectory requests until a clear path is authorized.

By processing reservation requests strictly on a First-Come-First-Served (FCFS) basis, AIM achieves near-perfect utilization of the intersection's physical spatial capacity, outperforming traffic signals. However, standard AIM faces a operational flaw: it is highly intolerant of human drivers. Empirical studies and extensive simulations have conclusively demonstrated that AIM provides negligible, or even heavily negative, throughput benefits compared to modern optimized traffic signals if the AV penetration rate in the network falls below 90%. Because the IM cannot accurately predict the exact, highly stochastic trajectories of human drivers, it may block large areas of space-time to maintain safety. This extreme safety buffering severely lower the efficiency of the entire reservation system, using pure AIM still need decades-long transition period of mixed autonomy.

Hybrid Autonomous Intersection Management (H-AIM) [32]

To directly bridge the critical operational gap of the mixed autonomy transition period, Sharon and Stone (2017) introduced the Hybrid Autonomous Intersection Management (H-AIM) protocol. Unlike its predecessor, H-AIM is fundamentally designed to function

universally and provide measurable benefits regardless of whether the AV penetration rate is 1%, 50%, or 99%.

H-AIM achieves this by layering the highly computational AIM reservation logic directly over a conventional, physically operating traffic signal. Within this framework, human-driven vehicles are completely unaware of the invisible reservation system. They navigate the intersection simply by obeying the traditional red, yellow, and green indications of the physical traffic light. The centralized IM, however, possesses perfect knowledge of the exact phase timing and sequencing of the physical signal.

When an AV requests a trajectory reservation, the IM evaluates the request against two distinct constraints: the trajectories of other already-reserved AVs, and the legally expected movements of human vehicles that are currently conforming to the active green light phase. Crucially, H-AIM allows AVs that are currently facing a physical red light to request a reservation to cross the intersection, provided their requested trajectory does not conflict with the active green-light traffic stream. For example, an AV facing a red light could request to navigate through a temporary gap in the cross-traffic, or safely execute a complex turning maneuver.

By utilizing intersection sensors to monitor real-time human vehicle positions, the IM can safely grant these opportunistic, out-of-phase reservations. H-AIM effectively grants AVs highly preferential, high-efficiency routing privileges without disrupting the safety parameters required by human drivers. Extensive simulations demonstrate that H-AIM yields measurable network-wide travel time reductions and throughput increases even at low AV penetration rates as low as 1%.

2.4 Vehicle to Vehicle (V2V) communication

The efficacy of H-AIM, RL signal control, and mixed autonomy coordination is strongly dependent on reliable, low-latency communications. The constant, high frequency exchange of kinematic data, reservation requests, and cooperative perception messages forms the fundamental backbone of the modern Internet of Vehicles (IoV). Currently, two distinct technological standards fiercely compete to facilitate robust Vehicle-to-Vehicle (V2V) and Vehicle-to-Everything (C-V2X) communications.

Dedicated Short-Range Communications (DSRC) [20] and IEEE 802.11p [18]

Dedicated Short-Range Communications (DSRC) represents the legacy standard for vehicular networking. It operates within the 5.9 GHz radio frequency spectrum, which was allocated by international regulatory bodies specifically for Intelligent Transportation Systems (ITS). The physical (PHY) and medium access control (MAC) layers of the DSRC architecture are strictly governed by the IEEE 802.11p standard, which is effectively a highly specialized iteration of commercial Wi-Fi optimized specifically for high-mobility, rapidly changing vehicular environments.

DSRC relies entirely on a decentralized, ad-hoc mesh network topology. This allows vehicles to broadcast standardized Cooperative Awareness Messages (CAMs) and Decentralized Environmental Notification Messages (DENMs) directly to neighboring vehicles

and fixed Roadside Units (RSUs) without requiring any connection to a central cellular infrastructure. This highly decentralized architecture offers high fault tolerance and low latency in ideal conditions, which are requirements for high-speed collision avoidance systems and instantaneous intersection reservation requests. However, DSRC suffers from severe scalability limitations. In highly dense urban traffic environments, the wireless channel relies on Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA). As vehicle density increases, the channel can quickly become saturated, leading to severe packet collisions, huge amount of data loss, and severely degraded communication reliability.

Cellular V2X (C-V2X) and 5G URLLC [6]

Developed in response to DSRC's limitations by the 3GPP consortium [20] (starting with LTE Release 14 and rapidly evolving into the 5G New Radio standard), C-V2X aggressively leverages existing global cellular network architectures to support vehicular communication. C-V2X is highly versatile, operating in two primary modes: network-based communication utilizing commercial cellular towers (the Uu interface), and direct, infrastructure-less device-to-device communication (the PC5 interface, commonly referred to as the sidelink).

The sidelink interface is particularly vital for autonomous intersection management protocols. Utilizing LTE Mode 4 specifications, vehicles can autonomously allocate communication resources using Semi-Persistent Scheduling (SPS[6]). Unlike the chaotic CSMA/CA approach of DSRC, SPS prevents packet collisions by having each vehicle actively sense the channel environment and systematically reserve orthogonal time-frequency resources for transmission. This allows for highly reliable V2V communication even when vehicles are operating entirely outside the coverage umbrella of a cellular base station.

Furthermore, 5G C-V2X offers exponentially higher bandwidth capabilities than DSRC. This high bandwidth facilitates highly advanced applications such as cooperative perception—a paradigm where an AV continuously shares its massive, raw sensor data streams (such as high-definition LiDAR point clouds and real-time video) with neighboring vehicles. This effectively eliminates blind spots at obscured intersections. The integration of 5G Ultra-Reliable Low-Latency Communication (URLLC) profiles ensures that the massive data payloads required by MARL algorithms and H-AIM IMs can be securely exchanged within the strict sub-10 millisecond threshold required for safe, high-speed mixed autonomy navigation.

3 Methodology

3.1 Simulation Framework and Traffic Scenarios

3.1.1 Simulation Environment: SUMO and TraCI

The core simulation is built upon SUMO, an open-source, microscopic, continuous-space, and multi-modal traffic simulation package [27]. I have chosen SUMO for its ability to model individual vehicle dynamics with high fidelity, allowing for the simulation of complex interactions between autonomous and human-driven vehicles. Unlike macroscopic models that treat traffic as a fluid, SUMO simulates the kinematics of every vehicle independently, which is essential for safety-critical reinforcement learning tasks. In this thesis, I will use SUMO microscopic simulator [27]

To enable the interaction between the Reinforcement Learning agent (typically implemented in Python frameworks such as PyTorch) and the simulation kernel, I implemented the Traffic Control Interface (TraCI)[51]. TraCI operates on a client-server architecture using a TCP-based protocol. It allows the RL algorithm to act as an external controller that can pause the simulation at every time step, retrieve the exact state of the environment (e.g., vehicle positions, speeds, induction loop data), and send control commands (e.g., acceleration or lane-change maneuvers) back to the vehicles.

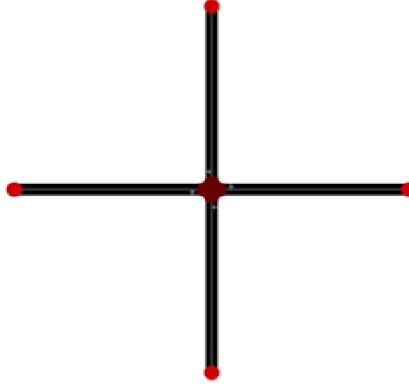


Figure 1: Network Topology Construction

3.1.2 Network Topology Construction

I constructed the simulation environment upon a discrete spatial domain represented as a directed graph $G = (N, E)$, where $N = 9$ represents the set of nodes (junctions or endpoints) and E represents the set of edges (road segments).

Node Configuration (Grid Topology)

The fundamental geometry is established using a 3×3 Cartesian grid arrangement consisting of 9 nodes. The nodes are spaced equally with a distance of 100 meters. The coordinate set N is defined as:

$$N = \{(x, y) \mid x, y \in \{0, 100, 200\}\} \quad (1)$$

In this configuration, the central node ($n_{100,100}$) functions as the unsignalized intersection, while the surrounding nodes act as the origins and destinations for traffic flow. The grid topology is shown in figure 1

Edge Specifications

The road network included 8 distinct edges connecting the peripheral nodes to the central junction. Each edge represents a single-lane road segment characterized by a length of $100m$ and a maximum speed limit of $13m/s$. The edges are categorized into two groups relative to the central junction node (e.g., `e_n_100.0_n_100.100`: edge from node (100,0) to node (100,100))

Table 1 summarizes the physical attributes assigned to the edges in the simulation.

Table 1: Edge Attributes and Kinematic Constraints

Parameter	Value	Unit	Description
From - To			Origin and destination node of edge
Length	100.0	m	Distance between nodes
NumLanes	1	-	Single-lane topology
Speed Limit	13	m/s	Max allowable velocity
Priority	0 / 1	-	SUMO right-of-way logic

Connection logic of edge and junction

In order to establish valid vehicle trajectories across the intersection, connections are generated based on topological continuity. A connection c is established between an incoming edge e_i and an outgoing edge e_o if and only if the destination of the incoming edge matches the origin of the outgoing edge. To prevent immediate U-turns within the intersection logic, I applied a constraint, that the destination of the outgoing edge must differ from the origin of the incoming edge.

Applying this logic to the 4-way intersection results in a total of 12 connections (3 maneuvers per approach: Left-Turn, Go-Straight, Right-Turn). These connections define the conflict zones where the Reinforcement Learning agent must manage to competing traffic flows. Out of 12 connections, there are 14 more internal connections, that generated by SUMO net builder, for connections internally in the junction.

Traffic Demand

Vehicle is defined to be 5m long. With geometry of the junction, is designed to create a scenario with multiple conflict points, requiring the agent to learn complex negotiation skills rather than relying on simple traffic rules. The traffic will only contain through traffic without turning. For H_0 warmup steps, all autonomous vehicles are uncontrolled and follow IDM. The vehicles controlled by agents are marked red, while the uncontrolled marked in white, show in Figure 2

Vehicle Routing and Directional Heuristics

Based on the established connections, a comprehensive set of routes is generated to cover

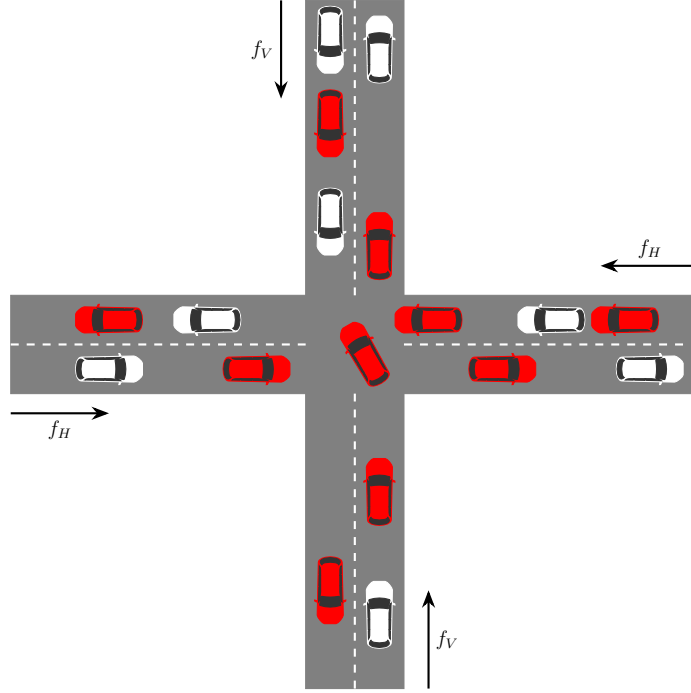


Figure 2: Four-way 1x1 traffic network.

all permissible traversals through the intersection. A route R is formally defined as an ordered sequence of connected edges $route = (edge_{in}, edge_{out})$.

To enable the Reinforcement Learning agent to distinguish between vehicle intentions (e.g., distinguishing a through-traffic vehicle from a turning vehicle), I implemented a geometric algorithm to classify these routes.

For straight trajectories, the algorithm verifies the collinearity of the incoming and outgoing edges by checking for constant horizontal (y -axis) or vertical (x -axis) alignment across the junction.

For turning maneuvers, the algorithm calculates the relative deviation angle ψ between the incoming edge vector and the outgoing edge vector. This angle is normalized to the range $[-\pi, \pi]$ to handle periodicity.

This process creates a total of 12 distinct routes (e.g., `r_turn_left_1`, `r_right_0`), ensuring that vehicles on any approach possess the full options of motions, therefore maximizing the complexity of the traffic scenario.

I model the traffic demand as a stochastic process to ensure the agent is robust to variable arrival rates. Vehicle generation follows a Poisson [5] process, where the time headways between consecutive vehicles are exponentially distributed. This prevents the agent from overfitting to deterministic traffic patterns and give us more realistic traffic flow. Below is the equation for Poisson distribution:[12]

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!} \quad (2)$$

The total system inflow is set to a maximum of 2800 vehicles/hour, distributed symmetrically:

- Horizontal Flow: 700 vehicles/hour (East-West and West-East).
- Vertical Flow: 700 vehicles/hour (North-South and South-North).

This distribution shown in the figure 2 of inflow rate f_H for horizontal lane and inflow rate f_V for vertical lane. Regarding vehicle routing, vehicles are permitted to perform unrestricted turning maneuvers at the junction. I used the build-in function of SUMO, `routeDistribution`, for creating the probabilities for turning routes. The turning probabilities are uniformly distributed to maximize intersection contention:

$$\begin{cases} P_{left} \approx 33\%, \\ P_{right} \approx 0.33\%, \\ P_{straight} \approx 0.33\% \end{cases} \quad (3)$$

3.1.3 Vehicle Kinematics and Mixed Autonomy

The experiments are conducted in a mixed-autonomy setting, featuring two distinct classes of vehicles: Human-driven Vehicles and Autonomous Vehicles (AV). The training phase utilizes a penetration rate of $p = 0.5$ (50% AVs), meaning every second vehicle is an autonomous vehicle.

The kinematic behavior of human drivers is controlled by car-following and lane-changing models to simulate realistic traffic flow. The longitudinal movement is controlled by the Intelligent Driver Model (IDM) [45] with Gaussian noise, which helps avoid collisions under nominal conditions. Lateral movements are governed by the SUMO default lane-changing model (LC2013) [11]. The specific kinematic parameters used for the simulation are detailed in Table 2. The kinematic behavior of autonomous vehicle can be found in Table 3. Here, the agents will control only the longitudinal movement of the vehicle.

From a Reinforcement Learning perspective, these kinematic interactions define the environment's dynamics. The stochastic transition function T is not explicitly modeled as a closed-form equation. Instead, the state transition $s_{t+1} \sim T(s_t, a_t)$ is sampled directly from the microscopic simulator [54]. The simulator applies the computed accelerations for all controlled vehicles and evolves the traffic state over a discrete simulation step duration δ . During this interval, the simulator integrates the IDM and lane-changing dynamics of human drivers, implicitly capturing the complex, non-linear evolution of the mixed-autonomy traffic flow. This process also generates the reward signal r , which is designed to optimize network throughput and safety (see Section 3.4.3).

3.2 Reinforcement Learning Algorithm

3.2.1 Policy Gradients (REINFORCE)

In the initial phase of this research, I implemented a fundamental Policy Gradient algorithm, often referred to as REINFORCE [52]. Unlike value-based methods (e.g., Q-

Table 2: Human Vehicle Kinematic Parameters and Driver Model Settings

Parameter	Value	Description
Car Following Model	IDM	Intelligent Driver Model [45]
Lane Change Model	LC2013	SUMO default lane changing model [11]
Max Speed	30 m/s	Maximum attainable velocity
Acceleration	2.6 m/s ²	Maximum acceleration capability
Deceleration	4.5 m/s ²	Maximum comfortable braking
Tau (τ)	1.0 s	Driver reaction time
MinGap	2.5 m	Minimum standing gap between vehicles
Sigma (σ)	0.2	Driver imperfection (0 = perfect, 1 = random)
SpeedDev	0.1	Speed deviation factor

Table 3: Autonomous Vehicle Parameters

Parameter	Value	Description
Max Speed	30 m/s	Maximum attainable velocity
Acceleration	1.5, 2.6 m/s ²	Acceleration capability
Deceleration	3.5, 4.5 m/s ²	Braking capability

Learning) which derive a policy indirectly by estimating state-action values, Policy Gradient methods search directly in the parameter space of the policy $\pi_\theta(a|s)$ to maximize the expected return.

The goal is to maximize the expected cumulative discounted reward $J(\theta)$:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [G(\tau)] = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^H \gamma^t r(s_t, a_t) \right] \quad (4)$$

where τ represents a trajectory generated by the policy, and $G(\tau)$ is the total discounted return of that trajectory.

To optimize this objective, gradient ascent is employed. The core mechanism relies on the Policy Gradient Theorem [41], which states that the gradient of the objective with respect to the policy parameters θ is proportional to the gradient of the log-probability of the actions, weighted by the return:

$$\nabla_\theta J(\theta) = \mathbb{E}_\tau \left[\sum_{t=0}^H \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t \right] \quad (5)$$

Here, G_t is the return from time step t onwards.

In this implementation, the policy π_θ is parameterized by a neural network consisting of

a Multi-Layer Perceptron (MLP) feature extractor followed by a policy head. There is no value function or critic network involved. The update rule at each training step is:

$$\theta_{k+1} = \theta_k + \alpha \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) G_t \quad (6)$$

where α is the learning rate.

Below is the pseudo code for Policy Gradient algorithm:

Algorithm 1 REINFORCE: Monte-Carlo Policy Gradient

- 1: **Input:** A differentiable policy parameterization $\pi_{\theta}(a|s)$
 - 2: **Input:** Learning rate $\alpha > 0$
 - 3: **Initialize:** Policy parameters $\theta \in \mathbb{R}^d$ arbitrarily
 - 4: **Loop** for each episode:
 - 5: Generate an episode trajectory following π_{θ} :
 - 6: $\tau = S_0, A_0, R_1, S_1, \dots, S_{T-1}, A_{T-1}, R_T$
 - 7: **Loop** for each step $t = 0, 1, \dots, T - 1$:
 - 8: Calculate return from step t : $G_t \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k$
 - 9: Update policy parameters:
 - 10: $\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_{\theta} \ln \pi_{\theta}(A_t|S_t)$
 - 11: **End Loop**
 - 12: **Until** convergence
-

Standard Policy Gradient methods suffer from significant limitations in practice. They are highly sensitive to the learning rate α . A step that is too large can induce a performance collapse where the policy changes drastically to a suboptimal region and cannot recover [38]. This instability necessitates the use of trust-region methods such as Proximal Policy Optimization (PPO), which constrains the update step to ensure monotonic improvement.

3.2.2 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) was designed to approximate the theoretical guarantees of Trust Region Policy Optimization (TRPO) [36] using standard first-order stochastic gradient descent (SGD) or adaptive optimizers (e.g., Adam) [38]. Rather than enforcing a complex Kullback-Leibler (KL) divergence [23] constraint, PPO directly modifies the surrogate objective function, penalizing policy updates that move the probability ratio excessively far from unity [38].

To formulate the PPO objective, a probability ratio $r_t(\theta)$ is defined, measuring the divergence of the current optimized policy from the behavioral policy utilized to collect the trajectory data [38]:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \quad (7)$$

When $r_t(\theta) > 1$, the selected action a_t has become more probable under the new policy. The core mathematical innovation of the PPO-Clip variant is the introduction of a

piecewise minimum function that clips this probability ratio, creating the robust clipped surrogate objective $L^{CLIP}(\theta)$:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (8)$$

The hyperparameter ϵ (typically configured to 0.2) defines the mathematical boundary of the trust region. The clipping mechanism operates conditionally based on the sign of the estimated Advantage $\hat{A}_t$, ensuring stable, monotonic learning by strictly bounding the parameter updates [38].

Generalized Advantage Estimation (GAE)

The precision and stability of the PPO gradient update rely heavily on the accuracy of the Advantage function $\hat{A}_t$, which quantifies how much better a specific action is compared to the baseline expectation of the current state. To mitigate the statistical variance of raw empirical returns, PPO implementations rely on Generalized Advantage Estimation (GAE) [37].

GAE utilizes a learned Critic network to estimate the state-value function $V_\phi(s_t)$, which serves as the optimal baseline. The Temporal Difference (TD) residual δ_t is defined as the error of the value function approximation at a single discrete time step:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \quad (9)$$

GAE computes the final advantage by calculating an exponentially weighted moving average of these TD residuals over the entire collected trajectory, managed by the trace decay parameter $\lambda \in [0, 1]$:

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l} \quad (10)$$

The hyperparameter λ controls a critical bias-variance tradeoff. By utilizing GAE, the PPO algorithm successfully weights accurate multi-step returns while simultaneously smoothing out erratic trajectory noise [37].

Implementation

Unlike off-policy algorithms, PPO is strictly an on-policy optimization method [38]. The algorithm alternates continuously between a data collection phase, where the current policy interacts with the environment to harvest trajectory data and an optimization phase, where the data is divided into mini-batches and the network weights are iteratively updated across multiple gradient descent epochs.

Once an optimization phase concludes, the saved trajectory data is permanently discarded. The formal pseudocode for the PPO-Clip algorithm with Generalized Advantage Estimation is presented in Algorithm 2.

Algorithm 2 Proximal Policy Optimization (PPO-Clip) with Generalized Advantage Estimation

```

1: Input: Initial policy parameters  $\theta_0$ , initial value function parameters  $\phi_0$ 
2: Hyperparameters: Clipping threshold  $\epsilon$ , GAE parameters  $\lambda, \gamma$ , Learning rate  $\alpha$ , Epochs  $K$ , Mini-batch size  $M$ 
3: for iteration  $i = 0, 1, 2, \dots$  do
4:   Phase 1: Trajectory Collection
5:   for actor  $worker = 1, 2, \dots, N$  do
6:     Run policy  $\pi_{\theta_{old}}$  in the environment for  $T$  timesteps.
7:     Collect trajectory data  $\mathcal{D}_i = \{s_t, a_t, r_t, s_{t+1}, \pi_{\theta_{old}}(a_t|s_t)\}_{t=1}^T$ 
8:   end for
9:   Phase 2: Advantage Estimation
10:  Compute the Temporal Difference residuals:  $\delta_t = r_t + \gamma V_{\phi_{old}}(s_{t+1}) - V_{\phi_{old}}(s_t)$ 
11:  Compute Generalized Advantage Estimates (GAE):  $\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}$ 
12:  Compute empirical returns:  $\hat{R}_t = \hat{A}_t + V_{\phi_{old}}(s_t)$ 
13:  Normalize advantages  $\hat{A}_t$  across the batch
14:  Phase 3: Policy and Value Function Optimization
15:  for epoch  $k = 1$  to  $K$  do
16:    Shuffle trajectory data  $\mathcal{D}_i$  and split into mini-batches of size  $M$ 
17:    for each mini-batch  $b$  do
18:      Calculate probability ratio:  $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ 
19:      Calculate Surrogate Objective:  $L^{CLIP} = \frac{1}{M} \sum_{t \in b} \min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)$ 
20:      Calculate Value Loss:  $L^{VF} = \frac{1}{M} \sum_{t \in b} (V_{\phi}(s_t) - \hat{R}_t)^2$ 
21:      Calculate Entropy Bonus:  $S = \frac{1}{M} \sum_{t \in b} -\pi_{\theta}(a_t|s_t) \log \pi_{\theta}(a_t|s_t)$ 
22:      Combined Loss:  $L_{total} = -L^{CLIP} + c_1 L^{VF} - c_2 S$ 
23:      Update parameters  $\theta, \phi$  via Adam optimizer:  $\theta_{new} \leftarrow \theta - \alpha \nabla_{\theta} L_{total}$ 
24:    end for
25:  end for
26:  Update global parameters:  $\theta_{old} \leftarrow \theta_{new}, \phi_{old} \leftarrow \phi_{new}$ 
27:  Discard trajectory data  $\mathcal{D}_i$ 
28: end for

```

3.2.3 Optimization Strategies

In the context of training deep neural networks for reinforcement learning, the choice of gradient descent optimization algorithm impacts both the convergence speed and the overall stability of the training process. Two adaptive optimization strategies used in this study are RMSProp and Adam.

RMSProp (Root Mean Square Propagation)

RMSProp [44] is an adaptive learning rate method originally proposed to resolve the radically diminishing learning rates associated with the earlier AdaGrad optimizer [9]. It

achieves this by dividing the learning rate by an exponentially decaying average of recent squared gradients. The mathematical update rule at time step t is defined as:

$$v_t = \beta v_{t-1} + (1 - \beta)g_t^2 \quad (11)$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{v_t} + \epsilon} g_t \quad (12)$$

where g_t is the gradient of the objective function, v_t is the moving average of the squared gradients, β is the decay rate (typically 0.9), α is the global learning rate, and ϵ is a small constant added for numerical stability.

- Pros: By restricting the window of accumulated past gradients, RMSProp prevents the learning rate from shrinking prematurely. This makes it highly effective for optimizing non-stationary objectives, which are common in reinforcement learning where the data distribution shifts as the policy updates.
- Cons: The algorithm still relies on a global learning rate hyperparameter (α) that must be manually tuned. Additionally, it lacks a formal bias correction mechanism, which can occasionally lead to erratic step sizes early in the training process.

Adam (Adaptive Moment Estimation)

Adam [22] extends the fundamental concepts of RMSProp by computing adaptive learning rates using exponential moving averages of both the first moment (mean) and the second moment (uncentered variance) of the gradients. Crucially, Adam incorporates bias-correction terms to counter the initialization of these averages at zero. The update rules are formulated as follows:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t \quad (13)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2 \quad (14)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (15)$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (16)$$

where m_t and v_t are the estimates of the first and second moments, β_1 and β_2 are their respective decay rates (typically 0.9 and 0.999), and $\hat{m}_t$ and $\hat{v}_t$ are the bias-corrected moment estimates.

- Pros: Adam is computationally efficient, requires minimal memory overhead, and naturally performs step size annealing. It is highly robust to sparse gradients and generally requires little hyperparameter tuning, making it the default choice for many deep learning applications.
- Cons: Despite its strong empirical performance, theoretical research has shown that Adam can fail to converge to the optimal solution in certain edge cases, occasionally settling into sub-optimal local minima compared to standard Stochastic Gradient Descent (SGD) with momentum [40].

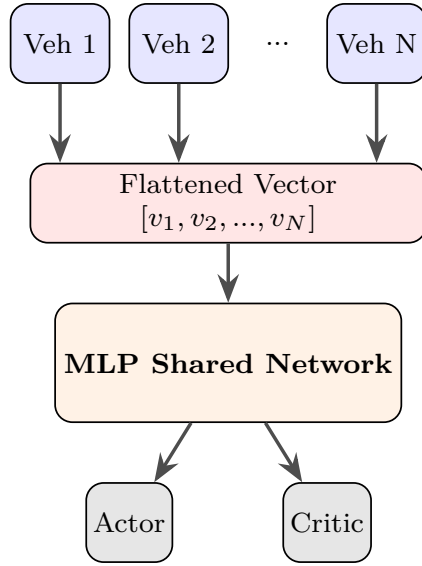
3.3 Neural Network Architecture

In this thesis, I address the challenge of autonomous vehicle in a unsignalized intersection by designing a neural network capable of processing the complex observations of a mixed-autonomy traffic stream. The architecture serves as the feature extractor for both the policy π_θ (Actor) and the value function V_ϕ (Critic) required by the Proximal Policy Optimization (PPO) algorithm 2.

To efficiently capture the interactions between the ego-vehicle and its surroundings, I implement a shared network architecture. This design uses a common feature extractor backbone to process raw observations, which then splits into two separate heads for policy and value estimation.

3.3.1 Shared Feature Extractor: From MLP to Transformer

Multi-layer Perceptron



Embedded Transformer

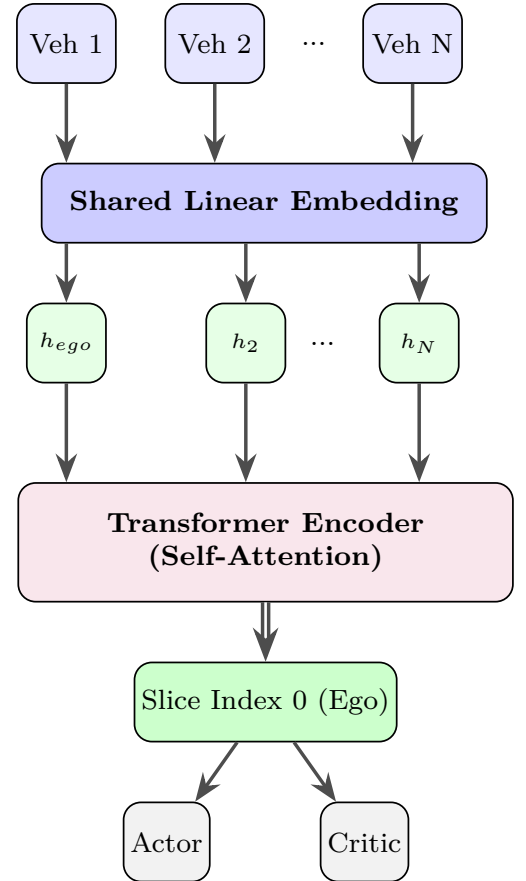


Figure 3: MLP vs. Transformer architectural in comparison.

The core challenge in representing the state of a traffic intersection is the variable nature of the environment. The number of relevant neighbors and their influence on the ego vehicle changes dynamically.

In the early stages of this research, I used a standard Multi-Layer Perceptron (MLP) as the shared feature extractor. This network flattened the observation matrix of all N visible vehicles into a single fixed-size vector. While computationally simple, the MLP approach lacks permutation to variance. It struggles to distinguish that a vehicle in "slot 1" and a vehicle in "slot 2" are essentially instances of the same object type, merely at different positions. Furthermore, a simple MLP treats all inputs with equal weight, failing to effectively prioritize critical vehicles (e.g., a leader braking hard) over irrelevant ones (e.g., another autonomous vehicle at a very far distance). The MLP architecture is shown in Figure 3.

To overcome these limitations, I replaced the MLP backbone with a Embedded- Transformer architecture, specifically designed to handle the relational dynamics of traffic, it teach the ego-vehicle how importance of it's neighbors. As implemented in the **AttentionFFN** module, this shared backbone consists of two key stages:

1. The raw observation for each observed vehicle i (where $i \in \{0, \dots, K\}$) consists of a vector $x_i \in \mathbb{R}^{B \times N \times d_{feature}}$ containing features that defined in the part 3.4.2, where B is number of observed autonomous vehicle in that time frame, N is the number of vehicle, where $N[0]$ is the position of ego-vehicle, $d_{feature}$ is the number of features of each vehicle, in this case is 7. I first pass each vehicle's observation through a learnable linear embedding layer followed by a non-linear activation using LeakyReLU :

$$\mathbb{X} = x_{embed} = \text{LeakyReLU}(W_e x_i + b_e) \in \mathbb{R}^{B \times N \times d_{model}} \quad (17)$$

Where B and N is the same as above, and d_{model} is 64 or 128, based on which setting is chosen. This projects the low-dimensional physical features into a higher-dimensional latent space $\mathbb{X} \in \mathbb{R}^{d_{model}}$, allowing the network to learn richer representations of vehicle states.

2. The sequence of embeddings $\mathbb{X} = x_{embed}$ is then processed by a Transformer Encoder [49]. I employ a stack of 3 encoder layers. The self-attention mechanism computes a weighted sum of neighbor features based on their relevance to the ego vehicle. For the ego vehicle (index 0), the attention mechanism calculates attention scores $\alpha_{0,j}$ with all other vehicles j :

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (18)$$

This is the standard scaled dot-product attention. In this equation, Q is query, K is key and V is value. Attention meaning to match queries to keys and use score to mix value. QK^T is the similarity, $\sqrt{d_k}$ is stabilizes gradients. For the case of multi-head attention with $d_{model} = 128$ and I used 4 heads, each heads has $d_k = 32$. The multi-head attention gives the model the ability to have the information from

difference representation subspace and different positions. In this work I use 4 parallel attention heads.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{where } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$
(19)

Give an example of 128, each head have 32 dimension. So internally:

$$Q_i, K_i, V_i \in \mathbb{R}^{B \times N \times 32} \quad \text{for } i = 1, 2, 3, 4$$
(20)

This allows the ego vehicle to effectively gives attention to the most critical neighbors (e.g., a blocking vehicle in the intersection) while ignoring less relevant ones, regardless of their order in the input list.

Given the embedded vehicle observations, self-attention is applied using a Transformer encoder. Queries, keys, and values are obtained via linear projections of the input embeddings. Multi-head attention enables each vehicle to attend to different interaction patterns among surrounding vehicles. The final output of this shared backbone is the transformed latent vector of the ego vehicle, $h_{ego} \in \mathbb{R}^{d_{model}}$, which contain the contextual information of the entire traffic scene. The Embeded Transformer architecture is shown in Figure 3. After feeding raw information into one of these two feature extractor, the information now feed into Actor (Policy network) and Critic (Value network). The implementation of this feature extractor is shown in the Figure 3.

3.3.2 Actor and Critic Heads

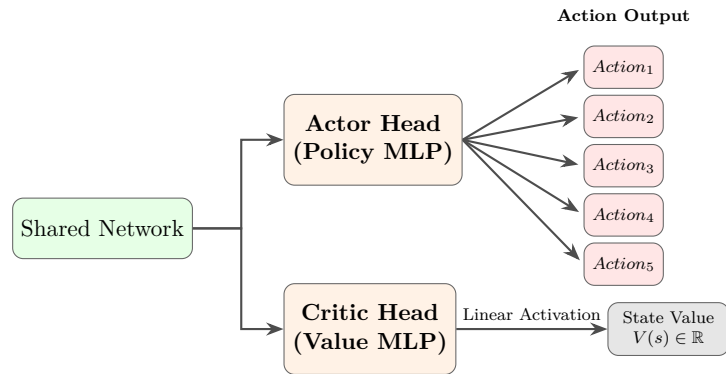


Figure 4: Actor and Critic heads.

Following the shared feature extractor, the vector h_{ego} is fed into two separate fully connected networks (heads), shows in Figure 4

The Critic Head (Value Function)

The Critic head is a fully connected multilayer perceptron that maps the latent state h_{ego} to a single scalar value $V(s)$.

$$V(s) \approx \text{MLP}_{critic}(h_{ego})$$
(21)

This value represents the expected discounted return from the current state. In my implementation of PPO, this estimate is crucial for computing the Generalized Advantage Estimation [37] 2, which serves as a baseline to reduce the variance of the policy gradient updates. By accurately predicting the goodness of a state, the Critic helps the Actor distinguish between truly beneficial actions and lucky environmental outcomes.

The Actor Head (Policy)

The Actor head determines the specific control action to be taken. It maps the same latent state h_{ego} to a probability distribution over the discrete action space.

$$\pi(a|s) = (\text{MLP}_{actor}(h_{ego})) \quad (22)$$

During training, actions are sampled stochastically from this distribution. Then, the action with the highest probability is selected deterministically. Finally, the policy parameter θ is shared across all vehicle in the traffic network network, enabling experience sharing among autonomous vehicle [13].

3.4 Markov Decision Process

3.4.1 State Space and Observation Space

In this mixed autonomy scenario, I formulate the problem as a finite-horizon discounted Markov Decision Process (MDP) [4]. The process is characterized by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, r, \rho, \gamma, H)$, consisting of the state space $\mathcal{S}$, $\mathcal{A}$ is the action space,

$$\mathcal{T} : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S}) \quad (23)$$

$\mathcal{T}$ is the stochastic transition function, the reward function r :

$$r : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R} \quad (24)$$

ρ_0 is the initial state distribution:

$$\rho_0 : \mathcal{S} \rightarrow \Delta(\mathcal{S}) \quad (25)$$

the discount factor γ , the horizon $H \in \mathbb{N}_+$, where $\Delta(\mathcal{S})$ is the probability simplex on $\mathcal{S}$. For policy-based model-free Reinforcement learning, the policy π_θ :

$$\pi_\theta : \mathcal{S} \rightarrow \Delta(\mathcal{A}) \quad (26)$$

is parameterized by θ , which is a neural network.

To overcome the challenges of a dynamic environment where the state and action spaces grow exponentially with the number of vehicles, I reframe the scenario as a decentralized partially observable MDP (Dec-POMDP) [4]. Instead of learning a single giant joint probability distribution—which is computationally infeasible due to the varying number

of vehicles—I assume that each AV acts strictly on the basis of its local view. The policy π_θ is decomposed into per-AV policies π_θ^i :

$$\pi_\theta^i : \mathcal{O}^i \rightarrow \Delta(\mathcal{A}^i) \quad (27)$$

The joint probability is decomposed into a product of independent probabilities:

$$\pi_\theta(f) = \prod_i \pi_\theta^i(o^i) \quad (28)$$

where π_θ^i are the policies for each autonomous vehicle. $\mathcal{O}^i$ is the observation space of each autonomous vehicle i , where $o^i = z(s, i) \in \mathcal{O}^i$, which contains only a subset of the state information $s \in \mathcal{S}$. $\mathcal{A}^i$ is the acceleration space for the autonomous vehicle $i \in \{1, \dots, |\mathcal{A}|\}$. I define z as the observation function that maps the observation space o^i for the autonomous vehicle i in state s . With the decomposition all the observations is a subset of the whole state and action of each autonomous vehicle is correspond to the final action space:

$$\mathcal{O}^1 \times \mathcal{O}^2 \times \dots \subseteq \mathcal{S} \quad \text{and} \quad \mathcal{A}^1 \times \mathcal{A}^2 \times \dots = \mathcal{A} \quad (29)$$

This decomposition allows the approach to remain robust in large-scale traffic scenarios.

3.4.2 Observation Space

Platoon observation space

In this work, I adapt the observation structure proposed by Yan et al. [54] to support a mixed-autonomy intersection with complex turning maneuvers. While the original framework introduced a simplified observation of three platoons (or "chains") suitable for straight-traffic-only scenarios [54], my environment involves vehicles performing left, straight, and right turns. To capture this increased complexity, I expanded the observation space to use platoon-based or KNN observation space.

To deal with the variable number of vehicles on approaching lanes while maintaining a fixed-size observation space for the neural network, I implement a structured traffic abstraction method known as *Platoon Decomposition*, that introduced in the work of Yan [54]. Instead of feeding the raw state of every vehicle into the network, I divided the continuous stream of traffic into functional groups called “Chains” and “Half-Chains”, based on the position of controllable agents (AVs). The Figure 5 is visual representation of the observation function $z(s, i)$ for the ego autonomous vehicle (AV) i approaching the intersection $\mathcal{I}(\mathcal{L}(i))$. The schematic utilizes a strict color-coding: the ego AV i is highlighted in yellow, cooperative AVs within the multi-agent network are colored in red, and uncontrolled (human-driven) vehicles are shown in white. To map the intersection topology, the incoming lanes are sequentially numbered from 1 to 4 in a clockwise direction, originating from the ego vehicle’s current lane $\mathcal{L}(i)$.

The traffic stream on any given lane is segmented as follows:

- Chain: A group of vehicles controlled by a single AV. A chain begins with an AV (the *Head*) and extends to the last human driver Vehicle preceding the next AV. The

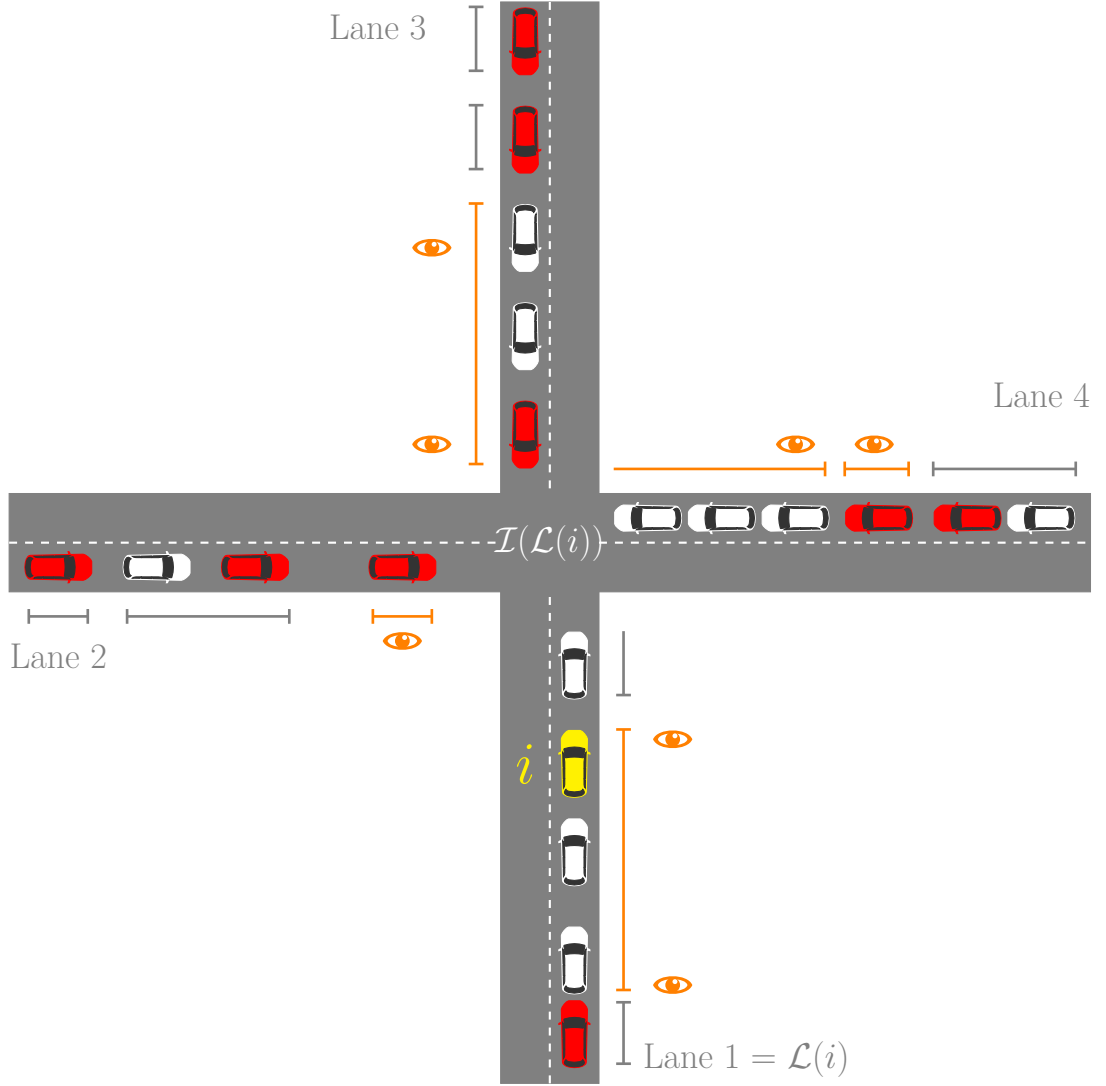


Figure 5: Platoon-based observation space

AV acts as the leader of this chain. By adjusting its speed, it effectively regulates the trajectory of all trailing human driver vehicle in the chain. In the Figure 5, lane 3 has a chain that lead by an AVs, end with a human driver vehicle. “Chain” represented by a full line segments.

- **Half-Chain:** A group of human driver vehicle located at the very front of the lane, ahead of the first available AV. Because these vehicles are already ahead of the system’s first actuator, they are strictly *observable* but not *controllable*. “Half chain” represented by half segments. In the Figure 5, half chain can be found in Lane 4.

The eye symbol means this chain is observing by ego-vehicle. To maintain a consistent, fixed-size input tensor for the neural network, lanes lacking sufficient traffic to form a leading half-chain (e.g., lanes 2 and 3) are automatically padded with default values. While the observational data is structured as sequential longitudinal chains, the vehicles within these chains maintain complete routing flexibility. They are not constrained to linear trajectories and may freely execute left, right, or straight maneuvers through the intersection.

For every conflicting lane direction, I construct a fixed-size feature vector representing the state of the traffic. This vector is composed of three primary “slots,” which capture the most critical geometrical bounds of the approaching traffic:

- **Slot 0 (The Uncontrollable Obstacle):** This slot stores the state of the Tail of the Half-Chain. It represents the last vehicle that must clear the intersection before the first controllable AV arrives. If no Half-Chain exists (i.e., the first vehicle is an AV), this slot is filled with default padding values.
- **Slot 1 (The Actuator):** This slot stores the state of the Head of the First Chain. This is the first AV on the approaching lane.
- **Slot 2 (The Controllable Boundary)** This slot stores the state of the Tail of the First Chain. This is the last vehicle in the group led by the AV in Slot 1.

The mapping of raw vehicles to these three slots follows a conditional logic based on the type of the leading vehicle closest to the junction (v_{lead}):

1. **Case A: Leader is an AV ($v_{lead} \in \mathcal{V}_{RL}$).**
If the vehicle closest to the intersection is an AV, the Half-Chain is non-existent.
 - *Slot 0:* Padded (Empty).
 - *Slot 1:* Assigned to v_{lead} (The AV itself).
 - *Slot 2:* Assigned to the tail of v_{lead} ’s platoon.
2. **Case B: Leader is a human driver vehicle ($v_{lead} \in \mathcal{V}_{IDM}$).**
If the leader is human-driven, a Half-Chain exists.
 - *Slot 0:* Assigned to the tail of the current human driver vehicle platoon (the last human driver vehicle in the continuous block of human traffic). The algorithm then searches upstream for the next available platoon.

- *Slot 1 and 2*: If an AV is found behind the Half-Chain, its head and tail are assigned to Slots 1 and 2 respectively. If the lane contains only human driver vehicles (no AVs), these slots remain padded.

To clarify this encoding, consider the following traffic configurations on a single lane (approaching from left to right):

1. [human driver vehicle $\rightarrow$ human driver vehicle $\rightarrow$ AV]
Here, the two leading human driver vehicles form a Half-Chain. The algorithm assigns the second human driver vehicle to *Slot 0*. The AV behind them is the Head of the first Chain, assigned to *Slot 1*. Since the AV has no followers, it is also the Tail, so *Slot 2* is also assigned to the AV.
2. [AV $\rightarrow$ human driver vehicle]
The leader is an AV, so the Half-Chain is empty (*Slot 0* is padded). The AV is the Head (*Slot 1*), and the following human driver vehicle is the Tail (*Slot 2*).
3. [human driver vehicle $\rightarrow$ human driver vehicle]
The lane contains only human drivers. The second human driver vehicle is the Half-Chain Tail (*Slot 0*). Since there is no AV to form a Chain, *Slots 1 and 2* contain default padding.
4. [AV $\rightarrow$ AV]
Two AVs are adjacent. The first AV is the Head (*Slot 1*) and, having no human driver vehicle followers, is also its own Tail (*Slot 2*). The second AV is treated as a separate, subsequent platoon and would be captured only if the observation depth is extended beyond the first chain.

For the vehicles within this observed platoon, I extract the following state features:

1. Speed (v): The current velocity of the observed vehicle.
2. Distance to Junction (d_{junc}): This represents the longitudinal distance along the route. It is calculated as the difference between the position of the junction on the route and the current position of the vehicle on that same route.
3. Turning Information (I_{turn}): A categorical feature indicating the vehicle's intended maneuver (Left, Straight, or Right). I utilize one-hot encoding [15] for this feature to assist the neural network in distinguishing between conflicting and non-conflicting trajectories.
4. Euclidean Distance to Ego (d_{ego}): The direct spatial distance [25] [21] between the center of the ego vehicle and the center of the observed vehicle.

As described in the training metrics, the initial experimental phase evaluated the model using a minimalist, platoon-based observation structure, the agent's state representation was strictly limited to the velocities and junction distances of observed vehicles. The primary objective of utilizing this constrained feature set was to determine if the agent could infer the underlying behavioral dynamics of the mixed-autonomy environment using the minimum amount of state information.[54]

However, a critical distinction must be drawn: while the original literature use this pla-

toon structure exclusively for simplified, straight-traffic scenarios, the environment in my research, the vehicles are allowed to move unrestricted, including left and right turning maneuvers. This free movement introduces intersecting conflict zones, creating a highly complex environment where pure platoon-based observations are insufficient for an autonomous vehicle to make safe decisions.

K-Nearest Neighbor (KNN) Observation Space Architecture

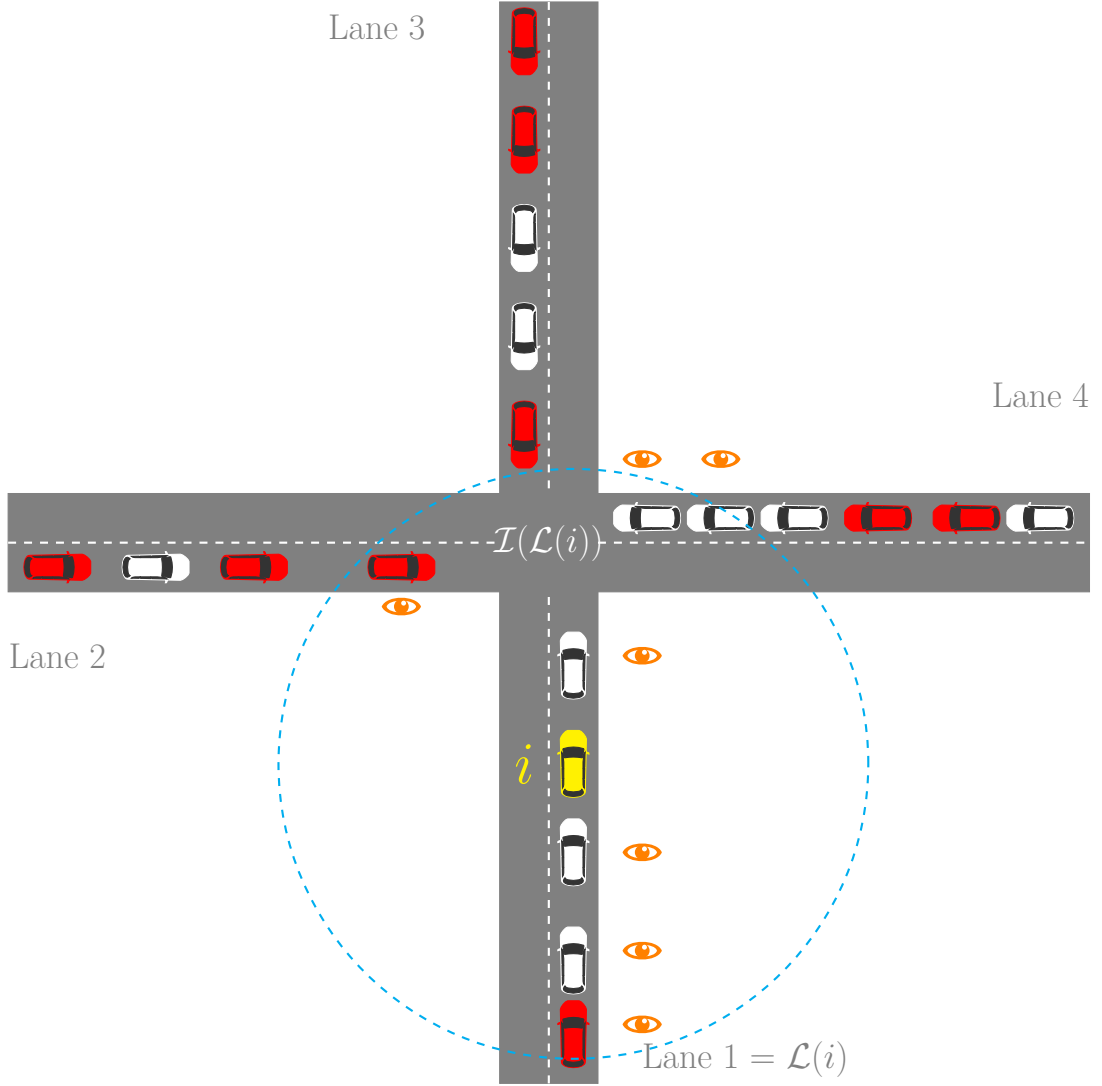


Figure 6: K-Nearest Neighbor (KNN) Observation Space.

A purely platoon-based observation can suffer from blind spots, particularly regarding vehicles on intersecting trajectories that may not be part of the currently observed "leading" platoon. To mitigate this partial observability and enhance collision avoidance, the state space was fundamentally redesigned utilizing a K-Nearest Neighbor (KNN) topological framework. This ego-centric observation mechanism is explicitly engineered to dynamically filter the surrounding traffic environment, ensuring that the neural network's attention mechanisms are strictly focused on the most immediate and relevant kinematic

threats. At each simulation timestep, the system queries the environment for a localized radius of vehicles surrounding the ego autonomous agent. Instead of relying solely on simplistic center-point distances, the spatial relationship between the ego vehicle and its neighbors is calculated using precise polygon-to-polygon distance metrics, ensuring highly accurate proximity awareness even during complex turning maneuvers.

A critical component of this observation space is its spatial filtering. The algorithm computes a directional vector using the ego vehicle's current heading angle via a dot product operation. This allows the system to assign a signed distance value to surrounding vehicles: positive for vehicles ahead in the trajectory, and negative for vehicles trailing behind. Furthermore, the algorithm incorporates an additional "blind-spot" filter; vehicles that are traveling in the opposite direction and have already passed the ego vehicle (defined mathematically by a relative heading difference between 170° and 190° and a negative projection vector) are explicitly discarded from the candidate pool, as they no longer pose a collision threat.

For the ego vehicle and every valid surrounding candidate, the observation function extracts a compact, 7-dimensional continuous feature vector. This data is strictly normalized to a $[-1, 1]$ bounding range to ensure optimal gradient flow and stability during the Actor-Critic network's optimization process. The extracted feature set for each vehicle comprises:

- **Relative Distance:** A signed continuous value representing the spatial gap between the ego vehicle's polygon and the neighbor. For the ego vehicle itself, this value is strictly fixed to 0.0.
- **Speed:** The current longitudinal velocity of the vehicle, scaled against the environment's maximum allowable speed limit.
- **Routing Intention (One-Hot Encoded):** A three-element discrete array $[\text{Left}, \text{Straight}, \text{Right}]$ indicating the vehicle's topological pathing intent. For instance, a vehicle proceeding straight is encoded as $[0, 1, 0]$.
- **Normalized Angle:** The vehicle's heading orientation within the simulated network, normalized to prevent extreme cyclic variance.
- **Distance to Junction:** A continuous metric detailing exactly how far the vehicle is from entering the core conflict zone of the intersection, which is vital for calculating yield-and-merge timing.

Once the feature vectors are computed for all nearby candidates, the algorithm sorts the vehicles by their distance and isolates the K closest neighbors. To bridge the gap between simulation and real-world deployment, the observation function strategically introduces a stochastic packet loss variable. This mechanism randomly drops neighbor data based on a predefined probability distribution, forcing the reinforcement learning policy to develop robustness against unreliable Vehicle-to-Vehicle (V2V) communication signals. Finally, the ego vehicle's array is concatenated with the filtered neighbor arrays to form a flattened input tensor. In sparse traffic scenarios where fewer than 10 vehicles are present, the state vector is automatically padded with default baseline values (representing a stationary vehicle at maximum distance with a neutral heading). This ensures a fixed-size

neural network input without introducing artificial "ghost" vehicles that could trigger false collision avoidance maneuvers.

Figure 5 is a visual representation of the observation function $z(s, i)$ for the ego autonomous vehicle (AV) i approaching the intersection $\mathcal{I}(\mathcal{L}(i))$. The schematic use a color-coding methodology: the ego AV i is highlighted in yellow, cooperative AVs within the multi-agent network are in red, and uncontrolled (human-driven) vehicles are in white. Radar radius from the ego vehicle is colored blue. To map the intersection topology, the incoming lanes are sequentially numbered from 1 to 4 in a clockwise direction, originating from the ego vehicle's current lane $\mathcal{L}(i)$. The traffic environment is structurally parsed into topological KNN. Vehicles that are actively captured by the ego vehicle's observation space are highlighted with explicit eye symbols, meaning the specific vehicles successfully assigned to the observation space. To maintain a consistent, fixed-size input tensor for the neural network, lanes lacking sufficient traffic are automatically padded with default values.

3.4.3 Action Space

The design of the action space represents a crucial divergence from the foundational methodology. In the original research framework established by Yan et al. [54], the autonomous vehicles were strictly controlled by a simplified, discrete set of three longitudinal actions: accelerate, maintain speed, or decelerate. Within that baseline configuration, the kinematic boundaries for the autonomous agents were mechanically capped at a maximum acceleration of $a_{accel} = 1.5 \text{ m/s}^2$ and a maximum deceleration of $a_{decel} = 3.5 \text{ m/s}^2$. During the preliminary phases of this study, I directly adapted this 3-action setting to establish a learning baseline.

However, empirical observations during early training simulations quickly revealed a significant operational limitation. The restricted acceleration and braking capabilities prevented the autonomous agents from executing the swift, decisive maneuvers required to navigate the dynamic, intersecting conflict zones of a multi directional grid. The agents lack the mechanical agility to seize available gaps in the continuous traffic flow, which frequently led to suboptimal intersection throughput and congestion, ultimately hindering the model from achieving convergent results.

To rectify this kinematic limitation, the action space was expanded to incorporate two additional, high-intensity longitudinal commands, transforming the control mechanism into a 5-action space. These new kinematic boundaries were calibrated to mirror the physical capabilities of the uncontrolled, human-driven vehicles simulated by the Intelligent Driver Model (IDM). Consequently, the agents were granted access to a higher maximum acceleration of a_{accel} equal to 2.6 m/s^2 and a more forceful maximum deceleration of a_{decel} equal to 4.5 m/s^2 . By expanded the number of actions, the agents acquired the necessary acceleration and deceleration to match the driving profile of the surrounding human fleet. My hypothesis is that the integration of these two additional limits will may improve model performance, enabling smoother gap acceptance, proactive collision avoidance, and ultimately yielding far more robust experimental results in a mixed-autonomy environment.

The joint action $\mathbf{a}_t$ is the Cartesian product of individual actions a_t^i for all controlled AVs. I use a discrete action space $\mathcal{A}$ for longitudinal control.

The policy π_θ outputs a probability distribution over 5 discrete acceleration:

$$\mathcal{A} = \{-4.5m/s^2, -3.5m/s^2, 0, 1.5m/s^2, 2.6m/s^2\} \quad (30)$$

This extends the 3-action baseline ($\{-a_{max}, 0, +a_{max}\}$) [54] to provide finer granularity for complex maneuvers such as gap acceptance and smooth car-following.

3.4.4 Reward Function

In the preliminary phases of my research, a global reward function was introduced, which proved viable for the simplified Policy Gradient (PG) algorithm. However, as the experimental complexity increased with the introduction of the Proximal Policy Optimization (PPO) algorithm and the Embedded Transformer architecture, the global reward structure exhibited significant limitations regarding credit assignment. A single, shared reward scalar fails to provide granular feedback to individual agents about the specific consequences of their actions in a densely populated grid. To resolve this, I designed a decentralized, individual reward structure, where a specific reward r_t^i is calculated for each autonomous vehicle (AV) i . These individual experiences are subsequently collected and aggregated to train the shared policy.

Since the agents share parameters, they optimize a global shared reward function $r(s_t, \mathbf{a}_t)$ to encourage cooperative behavior. The reward at step t is a weighted sum of global throughput and safety penalties:

$$r_t = \lambda_a \cdot N_{exit} - \lambda_c \cdot N_{crash} \quad (31)$$

where N_{exit} is the number of vehicles completing their trip in this step, and N_{crash} is the number of collisions. This rewards the entire "team" of AVs for safely maximizing the intersection's efficiency.

The individualized reward structure is divided into two primary components: cooperative throughput reward and an adaptive safety penalty system.

To optimize intersection throughput, agents are rewarded for successfully navigating the conflict zone. A base arrival reward of λ_a is granted to any AV that successfully exits the simulation grid. However, to foster cooperative "shepherding" behavior and discourage purely ego-centric policies, the reward function also tracks the successful exits of uncontrolled, human-driven vehicles. When a human vehicle exits the grid, a partial reward is allocated to the closest active AV remaining in the network. This cooperative arrival reward is scaled based on the spatial distance between the agent and the exiting vehicle, formalized as:

$$r_{arr,human}^i = r_{arr} \cdot \frac{d_i}{d_{max}} \quad (32)$$

where r_{arr} is the base arrival point, d_i is the distance from the agent to the exiting human vehicles, and d_{max} represents the maximum normalization distance, defined as 100 m.

To ensure safety is the primary constraint in mixed-autonomy traffic control. Initially, collisions were deterred using a static, constant penalty (e.g., $\lambda_{c-base} = 5$). While functional, a static penalty often fails to convey the compounding severity of consecutive failures, allowing agents to occasionally eliminate the penalty if the throughput reward is deemed mathematically favorable. In the experiments, the constant penalty must be pushed into extreme (e.g., $\lambda_c = 50..500$) to eliminate the collision completely.

To deal with this problem, I designed a complex, adaptive penalty mechanism. Under this paradigm, a baseline penalty is established, but the magnitude of the punishment grows exponentially with each subsequent collision an agent is involved in. This ensures that the agent learns that collisions are not just isolated negative events, but failures that can compound and substantially degrade performance. The adaptive penalty $\lambda_{adaptive}$ is calculated using the following exponential growth function:

$$\lambda_{adaptive} = \lambda_{base} \cdot \kappa^{N_{coll}} \quad (33)$$

where λ_{base} is the initial collision penalty, κ represents the penalty growth constant, and N_{coll} is the cumulative number of collisions.

In a dense intersection, a collision is sometime not the fault of a single vehicle, rather, it is often the result of poor traffic flow regulation by upstream agents. Therefore, the adaptive penalty is distributed using a spatial credit assignment logic to punish both direct and indirect contributors to a crash.

Vehicles directly involved in a collision receive the maximum penalty value λ_c . Furthermore, a partial, indirect penalty is assigned to surrounding vehicles to penalize the traffic conditions that facilitated the crash. This indirect penalty is applied to all vehicles immediately upstream in the same lane, as well as any vehicle present within a 20 m radius of the intersection center. The indirect penalty $r_{indirect}^i$ is spatially weighted as follows:

$$r_{indirect}^i = \lambda_{adaptive} \cdot \frac{d_{i,coll}}{r} \quad (34)$$

where $d_{i,coll}$ is the distance from the observing agent to the site of the collision, and $r_{context}$ is the defined observation radius of 20 m . By distributing the penalty spatially, the agents are heavily incentivized to maintain safe following distances and regulate the flow of human drivers, ultimately leading to the proactive, emergent safety behaviors observed in the results.

During the empirical evaluation of the training process, an unintended, suboptimal emergent behavior was frequently observed. Autonomous agents would adopt a risk-averse policy, choosing to remain completely stagnant at the intersection approaches. This behavior gives no reward, but also, no punishment at the agents. While this "parking" behavior successfully minimized the collision penalty to near zero, it caused the overall traffic flow to slowed down, conflicting with the network efficiency and throughput objectives of the intersection management system.

To resolve this issue and discourage agents from halting unnecessarily, a stagnation penalty

($r_{stagnant}$) was introduced into the individual reward structure. This conditional penalty specifically targets vehicles that refuse to advance despite having a clear and safe operational horizon. If an agent's current velocity (v_i) drops below 1.0 m/s while the distance to the closest detected threat (d_{close}) is greater than a safe threshold of 5 m, a minor but persistent penalty of 0.1 *points* is deducted. This logic is formalized as:

$$r_{stagnant} = 0.1 \quad \text{if } v_i < 1.0 \text{ m/s and } d_{close} > 5 \text{ m} \quad (35)$$

In other hand, to actively promote agents to accelerate and maintain optimal momentum when the path is unobstructed, a proportional speed reward (r_{speed}) was implemented. If an agent observes a clear trajectory, defined geometrically as a distance to the closest threat forward greater than 10 m, it receives a reward scaled linearly with its current velocity. This reward is calculated as the ratio of the vehicle's current speed (v_i) to the network's maximum allowable speed ($v_{max} = 13$ m/s), scaled by a coefficient of 0.5:

$$r_{speed} = 0.5 \cdot \frac{v_i}{v_{max}} \quad \text{if } d_{close} > 10 \text{ m} \quad (36)$$

After evaluating all situational variables within a given time step, the comprehensive individual reward (r_t^i) for a specific autonomous agent i at time step t is calculated. This final scalar aggregates all positively reinforcing components (such as arrival and speed rewards) and subtracts all safety and behavioral penalties (such as the adaptive collision and stagnation penalties). The composite reward equation is defined as:

$$r_t^i = \sum r_{reward} - \sum r_{penalties} \quad (37)$$

where:

$$r_{reward} = r_{arr} + r_{speed} + r_{arr,human} \quad (38)$$

and

$$r_{penalties} = r_c + r_{indirect} + r_{stagnant} \quad (39)$$

By integrating these dynamic mathematical components, the reward function effectively guides the reinforcement learning agent to converge on an optimal equilibrium between aggressive throughput maximization and strict, collision-free safety.

3.5 Experimental Design

To systematically evaluate the proposed approach, I designed these experiments in three phases. The objective was to first establish a stable learning baseline, then optimize the reinforcement learning components (reward, state, and hyperparameters), and evaluate the efficacy of the proposed Transformer-based architecture against standard Multi-Layer Perceptron (MLP) models, for the model to better understand the environment and chose action more wisely.

3.5.1 PG Algorithm and Optimizer

The initial phase focused on selecting the most common optimization strategy for the traffic control task. I established a baseline using standard Policy Gradient (REINFORCE) and compared the convergence stability of two optimizers:

- PG + RMSProp: Using the standard RMSProp optimizer.
- PG + Adam: Using the Adam optimizer with default parameters ($\beta_1 = 0.9, \beta_2 = 0.999$).

Based on the instability observed in pure Policy Gradient methods (discussed in Section 3.2.1), the focus shifted to Proximal Policy Optimization (PPO) combined with the Adam optimizer for subsequent experiments.

3.5.2 PPO Algorithm and Hyperparameter Tuning

In this phase, the PPO algorithm was implemented to address specific behaviors observed during simulation (e.g., collision rates or traffic stagnation). This involved three dimensions of modification:

1. Reward Function Adaptation: I designed multiple reward structures, transitioning from simple throughput-based global rewards to individual rewards for each AVs in the grid, functions that has it own collisions penalties (r_{coll}) and own arrival reward (r_{arr}).
2. Observation Space Augmentation: The state space was progressively expanded. Initial experiments used only distance to the junction and velocity of ego-vehicle and vehicles around it, later iterations included distance to other vehicles and route information about turning to improve safety and has more knowledge about what's happening around ego-vehicle.
3. Network Capacity (MLP): The neural network depth and width were adjusted (e.g., from 64×64 to 128×128 units) to determine the capacity required to model the non-linear traffic dynamics.
4. Hyper parameter tuning: The hyper parameter of a model such as collision coefficient, kl target, entropy coefficient, policy clipping, etc. will be tuned for better model performance or resolving issue (e.g Gradient vanish, Value loss explosion, etc.)

3.5.3 PPO Algorithm and Transformer Architecture Integration

In this final phase, I implement and evaluate the impact of the Transformer-based architecture. This setup replaced the MLP(multi-layer perception) feature extractor with a multi-head self-attention mechanism to better capture the spatial interactions between vehicles.

- **MLP Baseline:** PPO with the configuration from 3.5.2.

- **Transformer:** PPO with shared attention layers, investigating the effect of embedding size and number of attention heads on convergence speed and policy quality.

In this phase, the reward function is also evolved to have additional reward for smooth velocity (r_{speed}), and got punished for staying still for too long while no vehicle ahead (r_{still}).

3.5.4 Summary of Key Experimental Configurations

Table 4 summarizes the key hyperparameters and configurations for the representative experiments discussed in the results section.

Table 4: Summary of Experimental Configurations

Parameter	PG	PPO (MLP)	PPO (Embedded Transformer)
Algorithm	Policy Gradient	PPO (Clip)	PPO (Clip)
Optimizer	RMSProp Adam	Adam	Adam
Learning Rate	0.001	0.001	3×10^{-4}
Architecture	MLP 64x64 128x128	MLP 64x64 128x128 256x256	Embeded Dim: 64x64 / 2 Layers / 4 Heads 256x256 / 3 Layers / 4 Heads
Activation	Tanh	Leaky ReLU	Leaky ReLU
Rollouts per step	16	24/32/40	32/24
Entropy Coeff	0.01	0.005	0.01
Collision Coeff	5	5/10/20/40/50	20/5
Collision penalty growth	0	0/1.2	1.2

4 Results and discussion

4.1 Baseline Experiments

To evaluate the efficacy, safety, and throughput optimization of the proposed solution, it is necessary to benchmark the trained autonomous agents against established, traditional traffic management systems. In the baseline experiments, the mixed-autonomy condition is removed. Instead, I simulate the traffic scenario only with human-driven vehicles simulated via the Intelligent Driver Model (IDM)[35] and the LC2013 lane-changing model[11]. The intersection is then regulated using three distinct, infrastructure-based control methodologies, ranging from static priority rules to adaptive signal optimization. These baselines serve as a metric to evaluate the performance of the reinforcement learning agents by providing both theoretical lower bounds and upper-bound targets.

Signal-based control methods: Equal-Phase

The first baseline is the most popular form of intersection management found in urban environments: the static, fixed-time traffic signal. Under the Equal-Phase configuration, the intersection is governed by a predetermined, cyclical schedule that alternates the right-of-way between conflicting traffic approaches (e.g., the horizontal East-West flow and the vertical North-South flow). For this evaluation, I determine a symmetric phase duration of 25 seconds ($\tau_{equal} = 25s$) [54], is allocated to each direction.

To ensure a fair and direct comparison with the continuous, the yellow light and all-red clearance intervals are strictly set to 0 seconds. While the Equal-Phase method is robust to implement in the real world, its static nature makes it vulnerable to the stochasticity of traffic demands. Because vehicle generation in this simulation follows a Poisson distribution, the static signal will inevitably suffer from inefficiencies—allocating green time to empty approaches while vehicles on conflicting lanes accumulate in long queues. Consequently, this baseline serves as a standard metric for average, unoptimized throughput.

Below is the information for experiment with this baseline:

Table 5: Performance Metrics for the Equal-Phase Baseline (700×700 Inflow)

	Inflow (veh/hr)	Outflow (veh/hr)	Avg. Speed (m/s)	Collisions
Average	1994.4	1994.0	2.23	3.33

Signal-based control method: Max-Pressure

To provide a strong, dynamic performance benchmark, the Max-Pressure algorithm [47] is utilized as the second baseline. Rooted in queuing network theory, Max-Pressure is widely considered a decentralized control policy for maximizing throughput and stabilizing queue lengths in signalized networks. Unlike the Equal-Phase method, Max-Pressure is adaptive and operates in real-time without requiring prior knowledge of the traffic inflow rates.

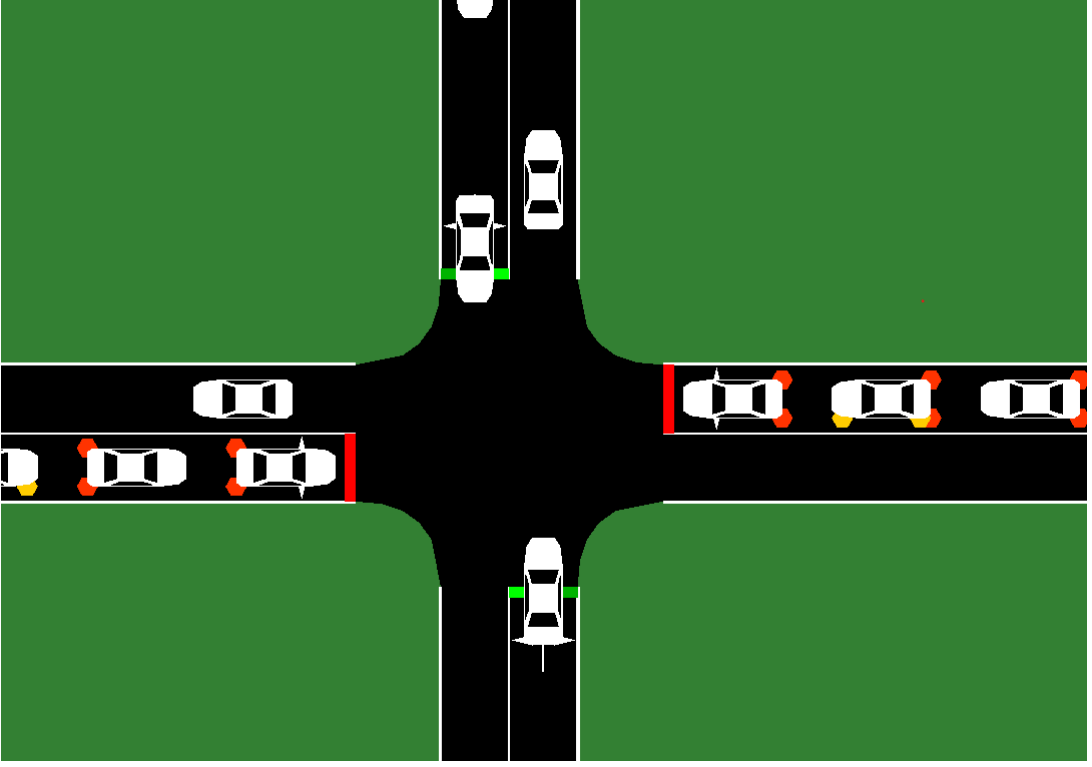


Figure 7: Baseline: Signal-based baseline method training in SUMO

The algorithm functions by continuously calculating the "pressure" of every possible signal phase. The pressure for a given phase is defined as the mathematical difference between the queue lengths (or vehicle counts) on the incoming approaches and the downstream outgoing lanes. At each decision point, the algorithm activates the specific signal phase that relieves the highest amount of pressure, effectively flushing the most congested pathways. To prevent erratic, high-frequency flickering of the traffic lights—which would induce unnatural stop-and-go waves in the IDM vehicles—a minimum phase duration constraint ($\tau_{min} = 4, 6, 12s$) is enforced. Because Max-Pressure actively optimizes for maximum vehicle discharge, it serves as an upper-bound performance target. If the decentralized autonomous vehicles can approach or exceed the throughput of Max-Pressure without the aid of physical traffic lights, it validates the strong efficiency of vehicle-to-vehicle coordination and the emergent "shepherding" behavior.

Below is the information for experiment with this baseline:

Table 6: Performance Metrics for the Max-Pressure Baseline (700×700 Inflow)

	Inflow (veh/hr)	Outflow (veh/hr)	Avg. Speed (m/s)	Collisions
Average	2262.0	2215.0	2.54	25.33

The signal-based baseline is illustrated in Figure 7. The color in the stop line shows a traffic light behavior, green for allow to move and red means stop.

Priority-Based baseline method

The final baseline evaluates the intersection under a purely unsignalized, rule-based paradigm, mimicking a standard intersection governed by yield or stop signs. In this Priority-Based configuration, a static hierarchy of right-of-way is established. One axis of the grid (e.g., the vertical North-South lanes) is designated as the major arterial road and is granted uninterrupted priority. The intersecting horizontal lanes are designated as the minor approaches.

Vehicles arriving on the minor approaches must yield the right-of-way to any conflicting traffic on the major road. They are forced to wait at the intersection boundary, continuously monitoring the priority traffic flow until a physically safe, acceptable gap allows them to execute their crossing or turning maneuvers. As the stochastic inflow rates increase, acceptable gaps in the priority stream become exceedingly rare, causing a lot of queuing vehicle, exponential delay increases. Therefore, the Priority-Based baseline acts as the performance bound, explicitly demonstrating the failure of traditional unsignalized intersections under heavy load, and highlighting the necessity of the active, gap-creating interventions performed by the trained RL agents.

Below is the information for experiment with this baseline:

Table 7: Performance Metrics for the Priority(Vertical) Baseline (700×700 Inflow)

	Inflow (veh/hr)	Outflow (veh/hr)	Avg. Speed (m/s)	Collisions
Average	1965.6	1964	2.19	3.33

Table 8: Performance Metrics for the Priority(Horizontal) Baseline (700×700 Inflow)

	Inflow (veh/hr)	Outflow (veh/hr)	Avg. Speed (m/s)	Collisions
Average	1984.8	1967.0	2.24	2.0

The Figure 8 illustrated the Priority behavior. The stop line with white marking meaning of the direction that has priority, and the line marked with gray marking has no priority.

4.2 Experiments

4.2.1 PG Algorithm with ADAM/RMSProp Optimizer

In the initial experimental phase, the Proximal Gradient(PG) algorithm [42] was evaluated using a standard Multi-Layer Perceptron (MLP) architecture [34] as the shared feature extractor along with 2 difference optimizers. The network consisted of a feedforward neural networks design with two hidden layers, each containing 64 neurons, and operated without a separate Critic network. The reward function for this phase was globally

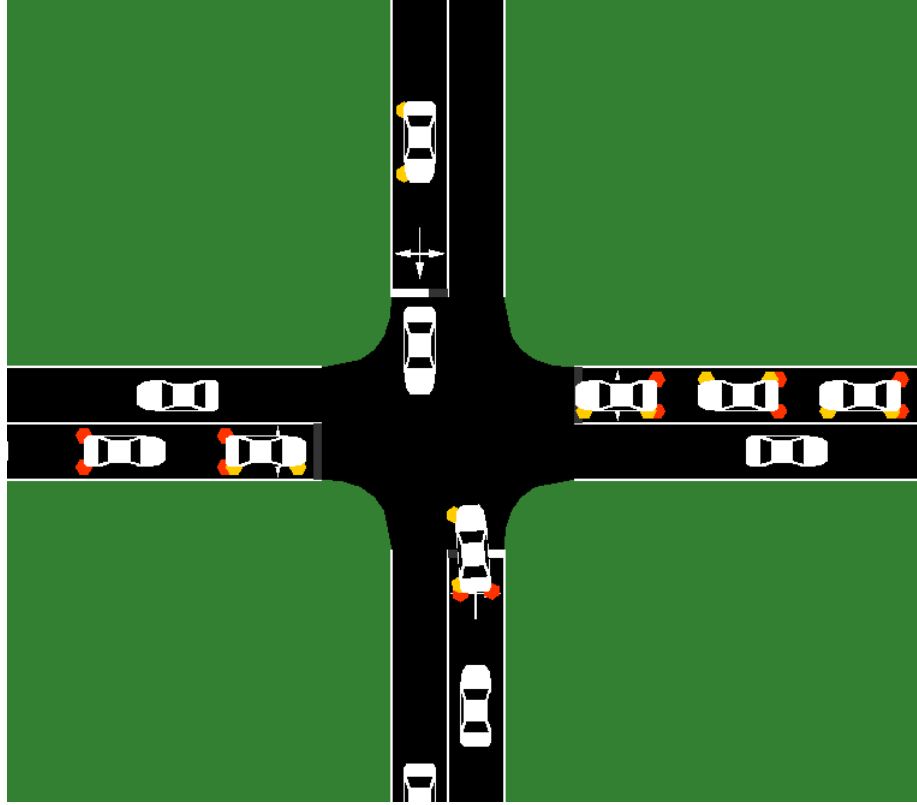


Figure 8: Baseline: Priority method training in SUMO

structured, granting +1 for each autonomous vehicle (AV) that successfully exited the grid and a fixed penalty of -5 for any collision, that explained in 3.4.4.

To establish this experiment, the data collection parameters were carefully configured. In this initial set of experiments, the training loop was configured to execute exactly 16 rollouts per optimization step. In the context of episodic reinforcement learning, a rollout means a complete, independent simulation run where the autonomous agents interact continuously with the traffic environment in, in a horizon of $H = 2000$. Throughout each rollout, a comprehensive trajectory of the agent’s experience is recorded and stored. This includes the sequential observation spaces encountered, the specific discrete actions chosen by the policy, and the resulting reward signals generated by the environment’s state transitions. Upon the completion of these 16 rollouts, the accumulated batch of experiential data is utilized in its entirety to compute the policy gradients and update the neural network’s weights.

More importantly, from a reinforcement learning perspective, this served as a test of the model’s sample efficiency. The primary scientific objective of restricting the volume of training data was to observe whether the agents could extract meaningful behavioral patterns from a highly constrained, sparse subset of environmental interactions. By forcing the Multi Layer Perceptron to rely on a small amount of information per update, it became possible to critically evaluate whether the baseline model could properly generalize, adapt, and learn the complex negotiation tactics required to navigate an unsignalized, mixed-autonomy intersection before providing it with massive datasets.

For the training phase described in this plot, the observation function is a minimalist, platoon-based structure 3.4.2. Specifically, the state representation provided to the agent is restricted strictly to the vehicles' velocities and their respective distances to the junction. This configuration intentionally mirrors the original observation space established in literature from Yan et al [54]. The primary rationale behind utilizing this constrained feature set is to compel the model to infer and learn the complex behavioral dynamics of the mixed-autonomy environment using the minimum amount of state information.

Given the limitations of a simple MLP in capturing the complex dynamics of a mixed-autonomy grid, the model's capacity to "understand" the environment was restricted. Despite this, the model established a surprisingly effective learning trajectory over 165 training iterations.

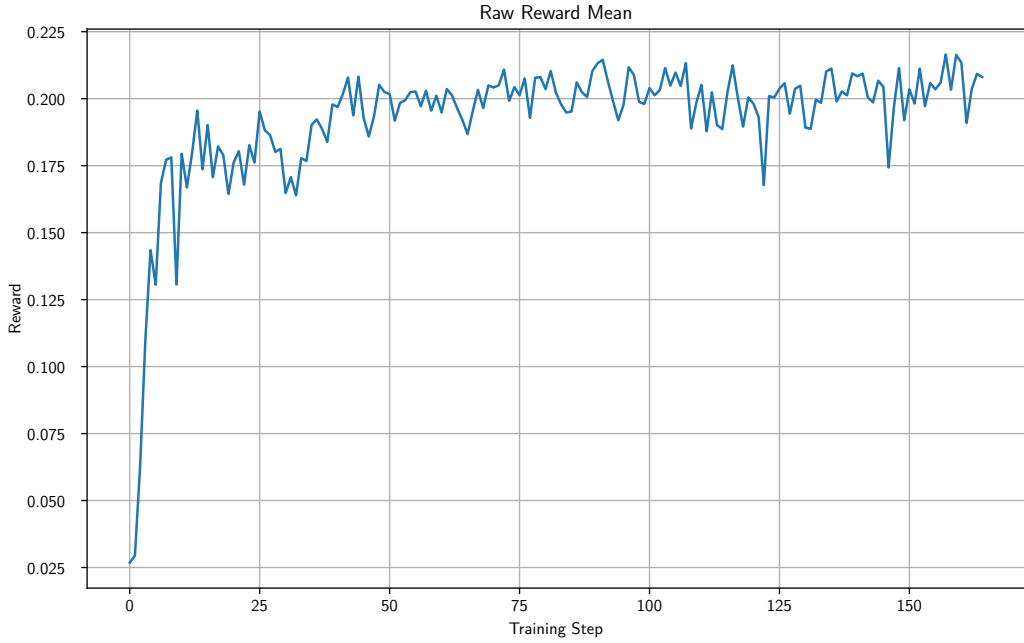


Figure 9: Raw reward mean of PG Algorithm with RMS Optimizer

As illustrated in figure 9, the learning process shows a distinct evolutionary pattern. Initially, the agent acts cautiously, resulting in low vehicle speeds and, consequently, low collision rates. However, as the agent begins to optimize for the arrival reward (+1), the average speed of the RL-controlled vehicles sharply increases. This acceleration leads to a corresponding spike in the collision rate.

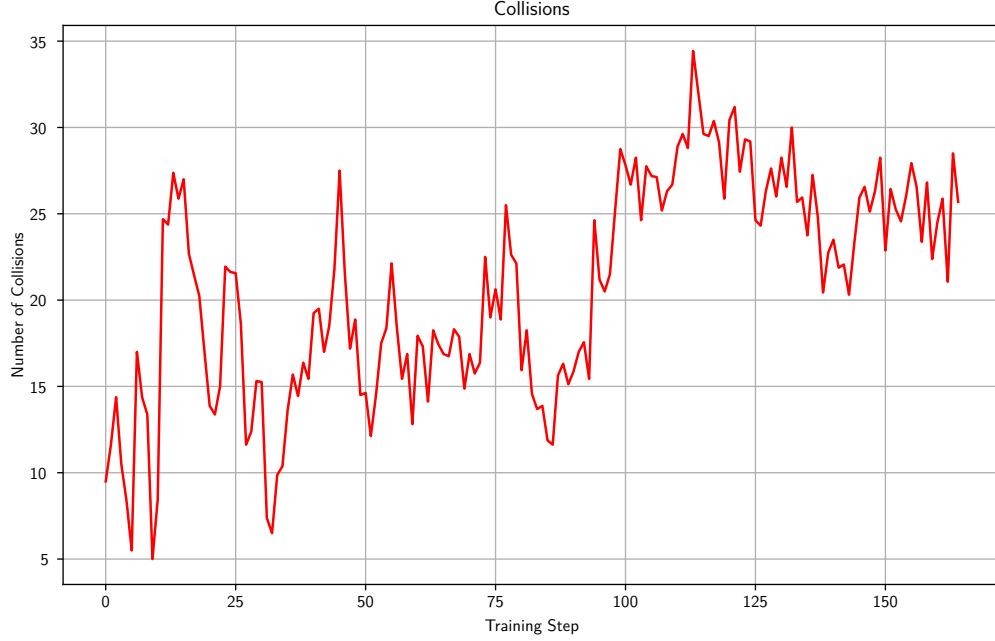


Figure 10: Collisions of PG Algorithm with RMSProp Optimizer

A critical component in the successful training of deep reinforcement learning models, particularly within the sensitive framework of standard Policy Gradient (PG) algorithms, is the selection of the optimization algorithm. Policy Gradient methods are vulnerable to high variance and unstable gradient updates. A single disproportionately large update step can degrade the policy into a suboptimal region from which it cannot recover. To determine the most effective optimization strategy for the mixed-autonomy intersection environment, a comparative analysis was made between two optimization methods: RMSProp (Root Mean Square Propagation) and Adam (Adaptive Moment Estimation).

The Adam optimizer builds upon the foundation of RMSProp by incorporating momentum, specifically, it utilizes both the exponentially decaying average of past squared gradients (like RMSProp) and the exponentially decaying average of past gradients. For this comparative experiment, the Adam optimizer was initialized with its standard hyperparameters: a first order moment decay rate of $\beta_1 = 0.9$ and a second order moment decay rate of $\beta_2 = 0.999$. To prevent premature convergence and ensure careful policy updates, a conservative learning rate of $\alpha = 0.0001$ ($1e - 4$) was implemented. Furthermore, no weight decay (weight decay = 0) was applied during this phase, ensuring that explicit regularization did not interfere with or mask the raw policy gradient signals being evaluated.

The training loop was structured to balance computational efficiency with gradient reliability. Specifically, the optimizer was configured to perform exactly one gradient update step per batch of 16 completed rollouts. By accumulating the experiential data across 16 independent rollouts before updating the network’s weights, the variance of the gradient estimation was significantly reduced. To ensure a granular, high-resolution analysis of the

learning trajectory, the simulation data, observation spaces, actions, rewards, and model weights were serialized and saved at an interval of every 5 training steps.

The results of this comparative evaluation are visually captured in the policy loss landscapes of both optimizers.

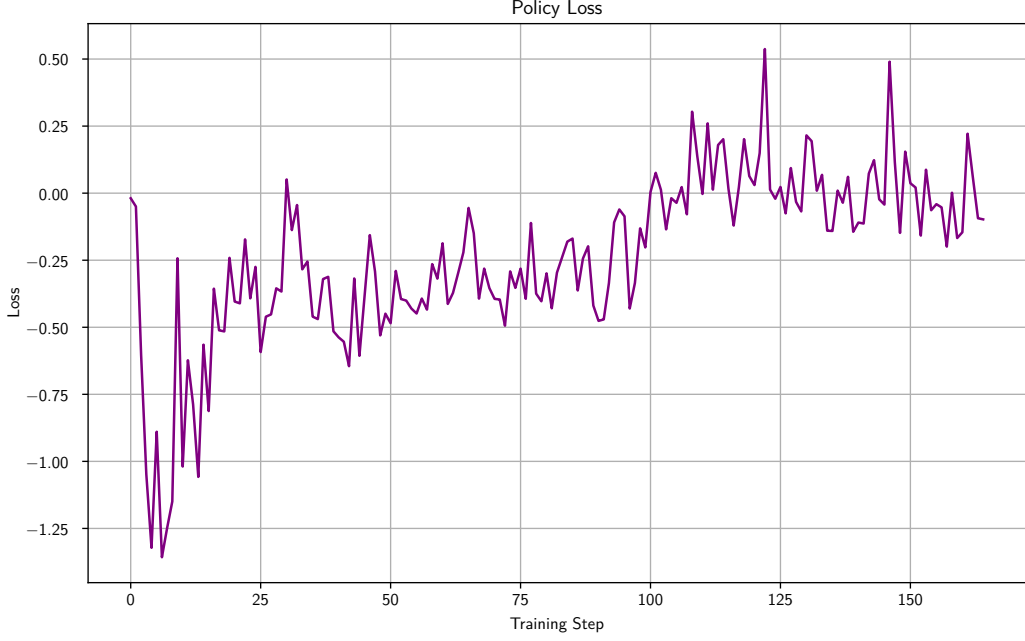


Figure 11: Policy Loss trajectory using the RMSProp optimizer.

As illustrated in Figure 11, the network trained utilizing the RMSProp optimizer demonstrated a highly erratic and unstable learning curve. The policy loss exhibited severe fluctuations throughout the training iterations, indicating that the optimizer struggled to navigate the complex, non-stationary loss landscape generated by the dynamic mixed-autonomy traffic environment.

Figure 12 shows the policy loss trajectory when the network was trained with the Adam optimizer under the exact same environmental conditions and random seeds. The Adam optimizer provided a smoother and more consistent minimization of the policy loss. The inclusion of the momentum terms (β_1 and β_2) effectively dampened the chaotic gradient fluctuations, allowing the agent to steadily improve its policy without suffering from performance collapses.

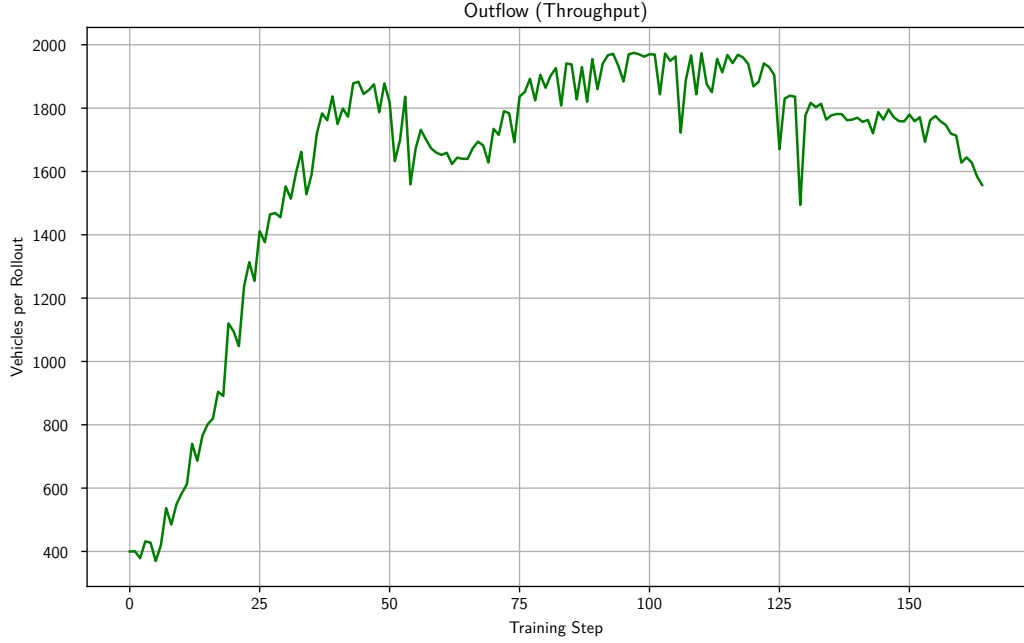


Figure 12: Policy Loss trajectory using the Adam optimizer.

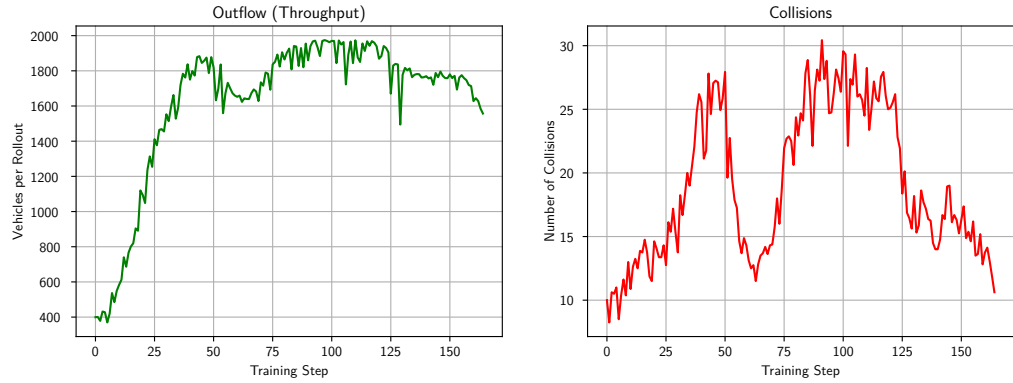


Figure 13: Training progression of the standard Policy Gradient (PG) model using ADAM optimizer.

The result of this training, visualized in Figure 13, provides critical insights into the limitations of policy gradient methods when tasked with balancing the fundamentally objectives of throughput maximization and collision minimization.

During the early stages of the training process, the model demonstrates a clear bias toward aggressive, high-risk exploration. Driven by the positive reward signals strictly associated with forward vehicle progression and overall network outflow, the autonomous agents rapidly learn to prioritize raw speed. By accelerating aggressively through the intersection without considering spatial awareness, the policy achieves its first significant performance peak, reaching an hourly outflow of approximately 1800 vehicles. However, this naive,

high-speed strategy completely fails to account for the complex, stochastic kinematics of the surrounding human-driven traffic. Consequently, this initial peak results in an unacceptably high safety failure rate of 27 collisions per hour. The agents successfully exploited the speed incentive, but lacked the predictive capacity to navigate geometric conflicts.

As the training progresses further, the heavily weighted negative rewards generated by these frequent collisions begin to dominate the gradient updates. The PG algorithm attempts to actively correct this dangerous behavior, forcing the policy into a sharp, reactive adjustment phase. The autonomous agents shift to a highly risk avoidance profile, significantly lowering their average velocities and adopting a much more conservative approach to intersection crossing. This penalty-driven correction is clearly reflected in the performance metrics: the hourly outflow degrades to approximately 1600 vehicles, while the collision rate drops substantially, hovering at roughly 12 collisions per hour. While mathematically safer, this overly cautious policy is suboptimal from a traffic efficiency standpoint, as the agents essentially induce unnecessary congestion to shield themselves from collision penalties.

In a subsequent attempt to escape this conservative local optimum and improve the overall cumulative reward, the model once again alters its strategy to maximize throughput. It successfully pushes the network flow to a new global maximum, hitting a peak outflow of roughly 2000 vehicles per hour. Unfortunately, without the bounding mechanisms and trust-region constraints found in more advanced algorithms, the standard PG model cannot safely maintain this high-throughput state. The aggressive kinematic behavior returns in full force, and the collision rate skyrockets to an maximum of 30 collisions per hour.

Following this sharp spike, the model exhibits severe optimization instability, oscillating between high-speed danger and low-speed safety without ever converging on a stable operating point. Due to this sustained lack of meaningful improvement and the algorithm's inability to isolate a robust policy, the training execution was terminated at step 165.

Below is the heatmap comparison of PG algorithm and ADAM Optimizer with Baselines:

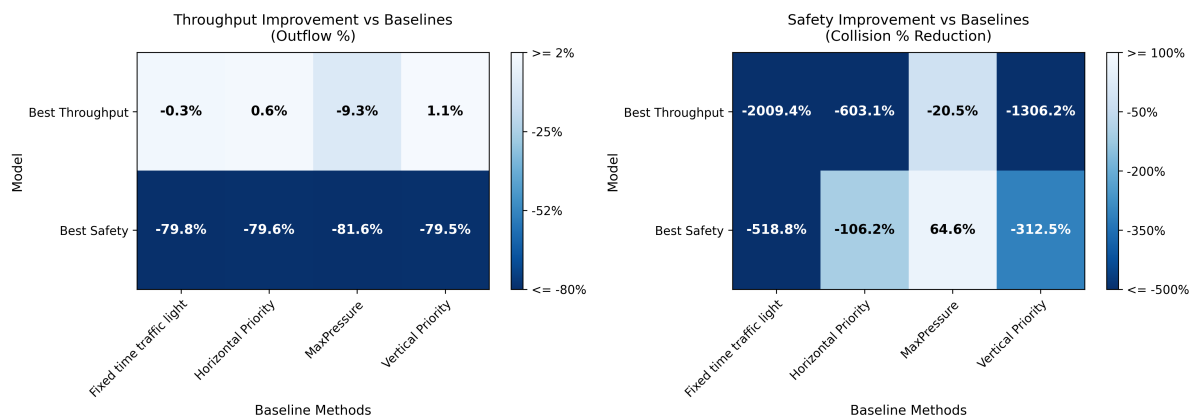


Figure 14: Comparison of PG Models and ADAM Optimizer with baseline methods

Based on the the observed results derived from this comparative analysis, in Figure 12, it was concluded that the Adam optimizer offers more training stability for this specific traffic control task. Consequently, RMSProp was discarded, and the Adam optimizer was selected to power all subsequent and more advanced experimental phases of this research, including the integration of the Proximal Policy Optimization (PPO) algorithm and the Embedded Transformer architectures.

The data shows in Figure 14 indicates that while the MLP policy successfully learns the primary objective: driving faster to maximize throughput and achieve the highest points. However, it fails to synthesize a safe driving strategy. The collision penalty (-5) is not enough to prevent the agent from prioritizing speed, resulting in an ego-centric policy where the agent consistently risks collisions to reach its destination. This high-variance, unsafe behavior clearly demonstrates the necessity for a more sophisticated architecture that capable of relational reasoning and safety planning.

4.2.2 PPO Algorithm, ADAM Optimizer and MLP Feature extraction

With the established from training stability of the Adam optimizer, the experimental algorithmic framework was upgraded to use Proximal Policy Optimization (PPO). While the standard Policy Gradient (PG) method provided a initial knowledge for initial training, its exposure to high variance and weak policy divergence. It fundamentally not enough for the escalating complexity of my mixed-autonomy intersection research.

My traffic networks characterized by unconstrained multi-modal routing (left, straight, and right turns) are highly non-stationary and volatile environments. A single excessively large gradient update in a standard PG algorithm can completely collapse a partially learned driving policy, forcing the autonomous agent to relearn basic safe-following behaviors from scratch. PPO mathematically prevent this critical vulnerability by introducing a clipped surrogate objective function. This clipping mechanism explicitly restricts the magnitude of the policy update at any given training step, essentially creating a mathematically bound "trust region" that ensures monotonic policy improvement. By preventing the new policy from deviating too far from the old policy, PPO guarantees the stable, incremental learning necessary to master the intricate, high-stakes gap-acceptance maneuvers required in dense traffic. Therefore, PPO was deemed the most viable, robust, and necessary algorithmic foundation for resolving complex traffic situations.

As detailed in Section 3, the neural network architecture that pair with the PPO algorithm during this phase is a Multi-Layer Perceptron (MLP) Feature extraction. The MLP serves as the primary baseline feature extractor, processing the flattened observation arrays, such as vehicle speeds, distances, and platoon structures, before feeding the latent representations into separate Actor and Critic network heads. The Adam optimizer is utilized to govern the weight updates of this architecture, operating with the tuned hyperparameters ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\alpha = 0.0001$) established in the preceding analysis.

To systematically evaluate the performance capabilities of the PPO algorithm and the MLP Feature extraction, an iterative experimental paradigm was planned. Rather than testing all novel environmental components simultaneously, the training environment was progressively upgraded. This ablation style methodology isolates the exact performance

contribution of each independent variable, ensuring a scientific evaluation of the system’s evolution. Table 9 provides a comprehensive overview of this iterative experimental sequence and the specific modifications applied at each stage.

Table 9: Overview of Experiments and Configurations - PPO MLP Feature extractor

Experiment Name	Configuration / Changes Applied
fourway_1x1_penetration0.5 _turn_adam_ppo	Transitioned the core algorithmic framework from standard Policy Gradient (PG) to Proximal Policy Optimization (PPO).
fourway_1x1_penetration0.5 _turn_adam_ppo_26.11	Introduced a standard PPO policy clipping parameter of 0.2. Configured the entropy coefficient to 0.01 to preserve policy exploration, increased the learning rate to 0.001, and set the value function coefficient to 0.5. The collision penalty coefficient was increased to 10.
fourway_1x1_penetration0.5 _turn_adam_ppo_27.11	Expanded the data collection phase by increasing the rollouts per optimization step to 24, and increased the neural network capacity to a 128x128 architecture.
fourway_1x1_penetration0.5 _turn_adam_ppo_01.12	Upgraded the network architecture from 64x64 to 128x128 utilizing Leaky ReLU activation rather than Tanh. Increased the rollouts per step to 24. Increased the collision coefficient to 10 and implemented a Poisson process for traffic generation to simulate more realistic, stochastic inflows.
fourway_1x1_penetration0.5 _turn_adam_ppo_04.12	Adjusted the optimization parameters by reducing the gradient descent steps per batch from 20 down to 15.
fourway_1x1_penetration0.5 _turn_adam_ppo_06.12	Expanded the discrete longitudinal action space by incorporating two additional control outputs to enhance the kinematic flexibility of the agents.
fourway_1x1_penetration0.5 _turn_adam_ppo_08.12	Significantly amplified the safety constraint by increasing the collision penalty coefficient to 50.
fourway_1x1_penetration0.5 _turn_adam_ppo_09.12	Reverted the gradient descent steps per batch to 20 and introduced a value function clipping threshold of 0.5 to stabilize Critic updates.
fourway_1x1_penetration0.5 _turn_adam_ppo_10.12	Redesigned the collision penalty into a spatially distributed nearby penalty. Implemented a cooperative arrival reward for AVs when human drivers successfully exit the grid. Integrated the Ray framework for distributed cluster training and adjusted the entropy coefficient to 0.003 to improve policy exploration.
fourway_1x1_penetration0.5 _turn_adam_ppo_11.12	Imposed a strict safety constraint by escalating the collision penalty coefficient to 500.

Table 9

Experiment Name	Configuration / Changes Applied
fourway_1x1_penetration0.5 _turn_adam_ppo_12.12	Increased the rollouts per step to 32 and reduced the base collision coefficient to 20, while introducing an exponential collision penalty growth factor of 1.2.
fourway_1x1_penetration0.5 _turn_adam_ppo_17.12	Deepened the Multi-Layer Perceptron backbone significantly, transitioning to a four-layer architecture of 192x128x64x32.

The experimental sequence detailed in Table 9 was designed to guide the autonomous agents from a state of naive exploration to more intelligent, cooperative traffic regulation. The results and models of these training can be found in my Github repository.

An initial training experiment of this PPO part was configured with a moderately data collection and optimization strategy. Specifically, the environment was set to collect 16 rollouts per training step, after which the neural network weights were updated across 20 gradient descent epochs per batch. The reward structure during this phase was kept like in the experiment of Yan et al.[54], the agents received a static positive reward of 1 for successfully exiting the grid, with a penalty of 5 for any collision event.

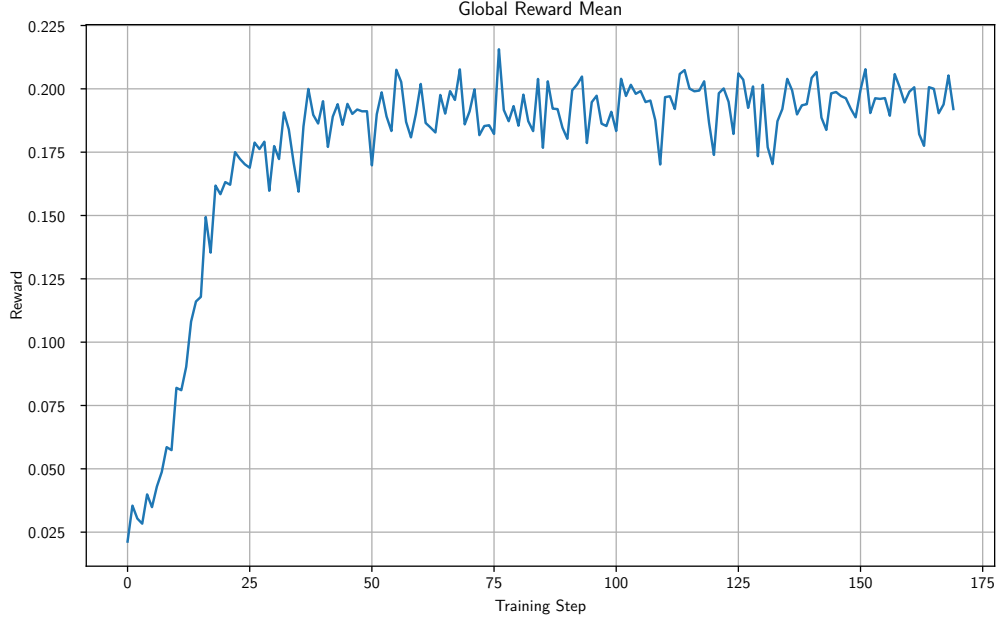


Figure 15: Progression of the global mean reward during the initial PPO training phase.

An analysis of the global reward mean, illustrated in Figure 15, reveals the immediate learning trajectory of the policy. The agents demonstrated a rapid initial acquisition of the driving task, with the mean reward climbing steeply during the first 50 training steps to reach a peak value of approximately 0.2. However, following this initial ascent, the learning

curve completely flattens. For the remainder of the training, the reward oscillates within a narrow, suboptimal band between 0.175 and 0.200. This distinct plateau indicates that the algorithm has suffered from premature convergence, effectively trapping the policy within a local minimum from which it cannot escape using the current hyperparameter configuration.

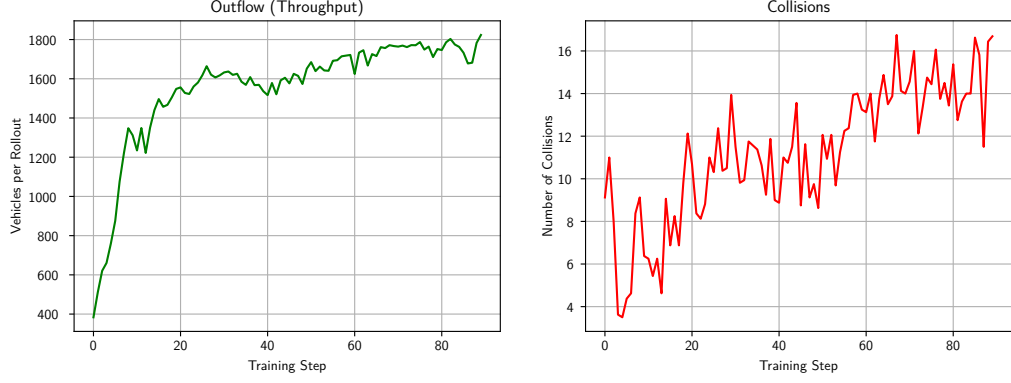


Figure 16: Throughput and safety metrics during the initial PPO training.

The phenomenological cause of this local minimum becomes entirely clear when examining the throughput and safety metrics in Figure 16. The network throughput peaks at an impressively high volume of approximately 2100 vehicles per hour. However, this speed is achieved alongside a very high failure rate of 37 collisions per hour. This indicates a classic reinforcement learning phenomenon known as "reward hacking"[2][39]. The autonomous vehicles have learned that by driving dangerously fast to the end of the grid, the sheer volume of successful +1 exit bonuses mathematically outweighs the occasional -5 collision penalties. For that reason, the policy optimizes for extreme recklessness behavior of the agents, maximizing throughput while demonstrating a complete disregard for systemic safety constraints.

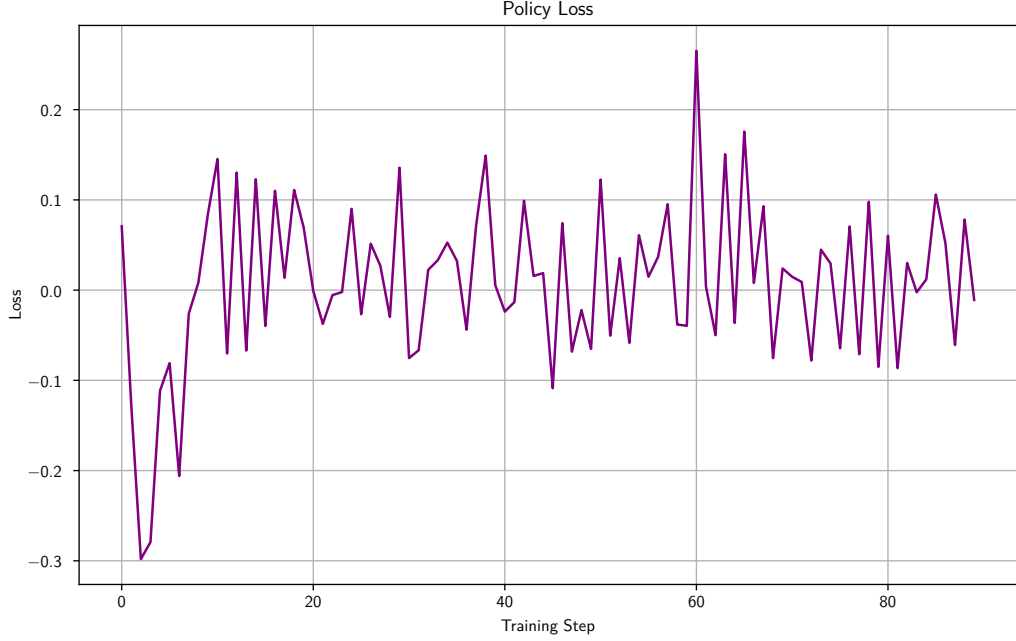


Figure 17: Trajectory of the PPO policy loss.

The underlying mathematical mechanics of this stagnation are further proven by the trajectory of the policy loss, shown in Figure 17. After an initial period of variance where the model learns the basic environment dynamics, the policy loss aggressively converges toward and rapidly oscillates around 0 [10]. In the context of PPO’s clipped surrogate objective function, this behavior indicates that the gradient updates have been completely bottle-necked by the trust-region boundaries. Because the advantage estimates, which evaluate how much better an action is compared to the baseline average, are normalized across the batch to have a mean of zero, symmetric updates will naturally pull the loss toward zero. More importantly, because the agents are trapped in a behavioral loop of speeding and colliding, the algorithm is failing to discover any uniquely advantageous actions outside of its current distribution. When the probability ratio of the policy attempts to shift, it immediately hits the $\epsilon = 0.2$ clipping boundary, forcing the gradient to zero and halting any meaningful behavioral evolution.

Recognizing that the policy had permanently stagnated within this local optimum, the training execution was manually terminated at step 170. This failure clearly shows a fundamental restructuring of the hyperparameter, the network architecture and reward scaling to force the agents to prioritize safety over raw, unpenalized throughput.

Driven by the limitations and unsafe "reward hacking" observed in the initial PPO configuration, the next experimental phase will target modifications to both the neural network architecture and the optimization hyperparameters. The primary architectural shift is the replacement of the hyperbolic tangent (Tanh) activation function [24] with Leaky ReLU [31]. In the context of deep reinforcement learning, Tanh functions are susceptible to saturation, extreme input values push the activation toward its asymp-

totes (-1 or 1), causing the derivative to approach zero. This induces the vanishing gradient problem, which effectively halts the learning process in deeper network layers. While the standard Rectified Linear Unit (ReLU) solves this by passing positive values linearly, it introduces a critical vulnerability known as the "dying ReLU" problem [30]. If a large gradient pushes a neuron's weights such that it only ever receives negative inputs, the standard ReLU outputs exactly zero, permanently inactivating the neuron and irreversibly destroying gradient flow. By adopting Leaky ReLU, the network maintains a small, non-zero gradient for negative inputs. This ensures continuous gradient flow, guarantees robustness against dead neurons [8], and provides a much more stable feature extractor and feedforward network for the highly stochastic, continuous state space of mixed-autonomy traffic. Therefore, I choose the Leaky ReLU activation function for my solution, and it will be apply to all of my experiments from now on.

Beyond the activation function, the optimization parameters were explicitly adjust to mandate conservative driving behaviors. The value function coefficient within the PPO objective was reduced to 0.5. This mathematically biases the optimizer to prioritize updates to the actor network (the policy) over the critic network (the value baseline). Most critically, the collision penalty coefficient was doubled from 5 to 10. By radically increasing the penalty gradient surrounding vehicular conflicts, the reward is designed to suppress reckless acceleration and forcefully condition the agents to prioritize collision avoidance.

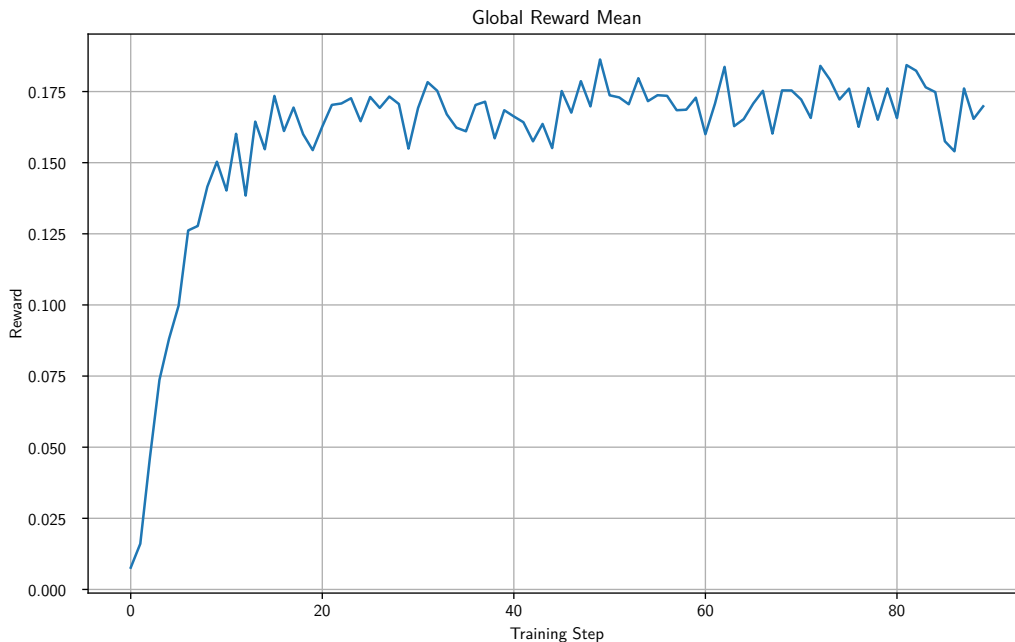


Figure 18: Progression of the global mean reward following the implementation of Leaky ReLU and doubled collision penalties.

The impact of these modifications on the learning trajectory is illustrated in Figure 18. Despite the collision penalty being doubled, the global reward mean mirrors the previous

experiment, peaking at approximately 0.175. However, the rate of convergence is drastically accelerated. The policy reaches its initial performance peak in merely 20 training steps. Following this rapid adaptation, the model enters a strict plateau, with the reward oscillating tightly around the 0.175 threshold, indicating that the algorithm has rapidly isolated a new, penalty-constrained local optimum.

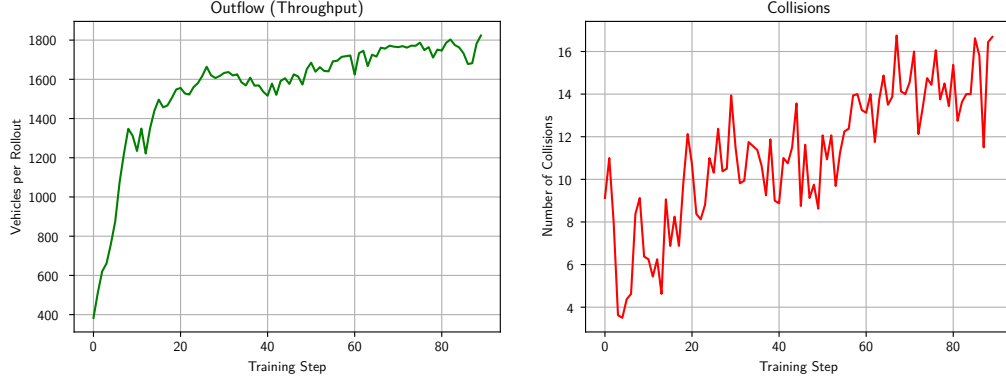


Figure 19: Throughput and collision metrics demonstrating the behavioral shift induced by the escalated safety penalty.

The vehicular behavior of this new optimum is clearly captured in the throughput and safety metrics shown in Figure 19. With the heavy penalties, the autonomous agents adopt a significantly more conservative kinematic profile. The hourly outflow peaks at 1800 vehicles—a moderate 10% reduction from the previous unconstrained peak of close to 2100. In exchange for this drop in throughput, the peak collision rate plummets to 16 collisions per hour, representing a massive 48% reduction compared to the previous iteration. While this confirms that the hyperparameter tuning successfully forced the agents to prioritize safety, from a strict traffic engineering perspective, an metric of 16 collisions per hour remains high for a standard intersection. This indicates that while the model’s behavior can be superficially shaped by penalties, the agents fundamentally lack the environmental awareness required to navigate complex conflicts flawlessly.

Figure 20 demonstrates the policy loss continuing to oscillate predictably around 0. As established in prior analyses, this confirms that the agents are actively and successfully extracting meaningful gradients while safely bounded by the PPO trust-region clipping mechanism (ϵ). Because this oscillation represents the nominal, mathematically correct behavior of the PPO algorithm under stable learning conditions, visualizations of the policy loss will be strategically omitted from subsequent experimental evaluations to maintain focus on physical macroscopic metrics.

Recognizing that the current configuration had decisively hit its optimization limits, it’s safer but still unacceptable collision rates, the training execution was purposefully terminated at step 92. This plateau formally concludes the hyperparameter-tuning phase, proving that superficial parameter scaling is insufficient and dictating a fundamental, structural upgrade to the observation and action spaces in future architectures.

In the next experiment, vehicle generation was controlled by a deterministic, uniform insertion rate. While this simplified the initial learning environment, it failed to repre-

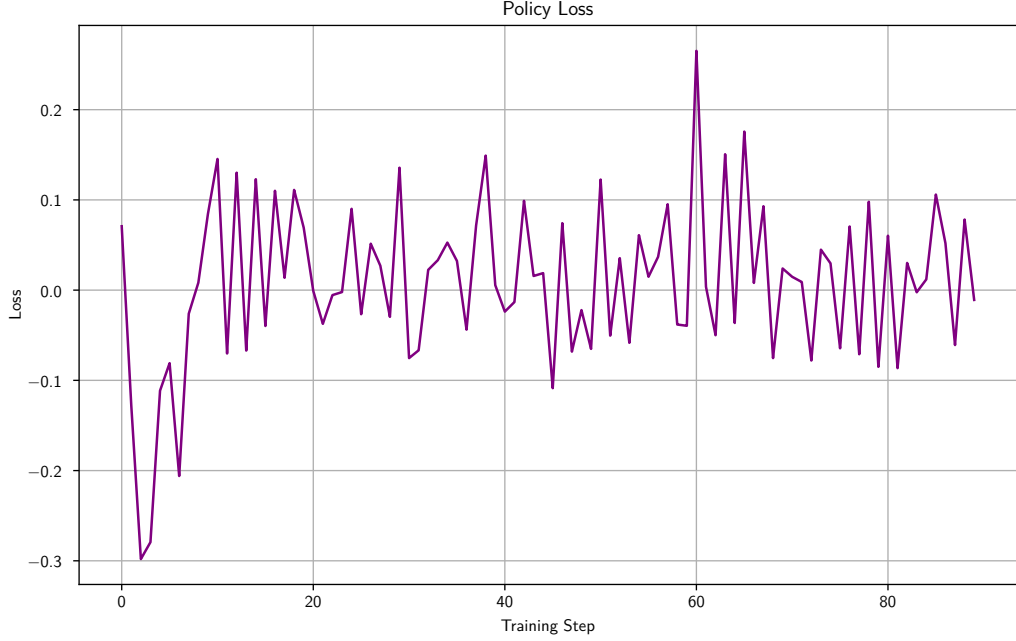


Figure 20: PPO policy loss of the model from 26.11.

sent the stochastic, unpredictable nature of real-world traffic scenario. To address this limitation and test the robustness of the autonomous policy, this third experiment introduces a Poisson arrival process for all intersecting traffic flows. By replacing uniform intervals with a probability distribution, the environment now dynamically generates realistic traffic, such as dense vehicle platoons and irregular gaps, significantly elevating the complexity of the task.

With the increased difficulty of this environment, two major structural upgrades were applied to the training pipeline. First, the data collection horizon was broadened by increasing the rollouts per step from 16 to 24. This equates to a 50% increase in the experiential data collected per batch, effectively exposing the algorithm to 150% of the original environmental information during each gradient update. This wider distribution of data was intended to stabilize the advantage estimations against the newly introduced traffic variance. Second, to ensure the neural network possessed the representational capacity to process this complex, non-linear environment, the network was expanded. The hidden layers were doubled in width, transitioning to a 128×128 neuronal architecture.

Despite these substantial augmentations to both data volume and network capacity, the global mean reward, illustrated in Figure 21, demonstrates a lack of improvement. The overall reward trajectory mirrors the limitations of the previous experiment, ultimately peaking at the same 0.175 threshold. Furthermore, the model continues to exhibit severe premature convergence. The learning curve sharply peaks at merely 25 training steps before abruptly flattening, becoming permanently trapped in a local minimum for the remainder of the training lifecycle. This indicates that expanding the raw computational capacity of the network is entirely insufficient to break the suboptimal behavioral loop.

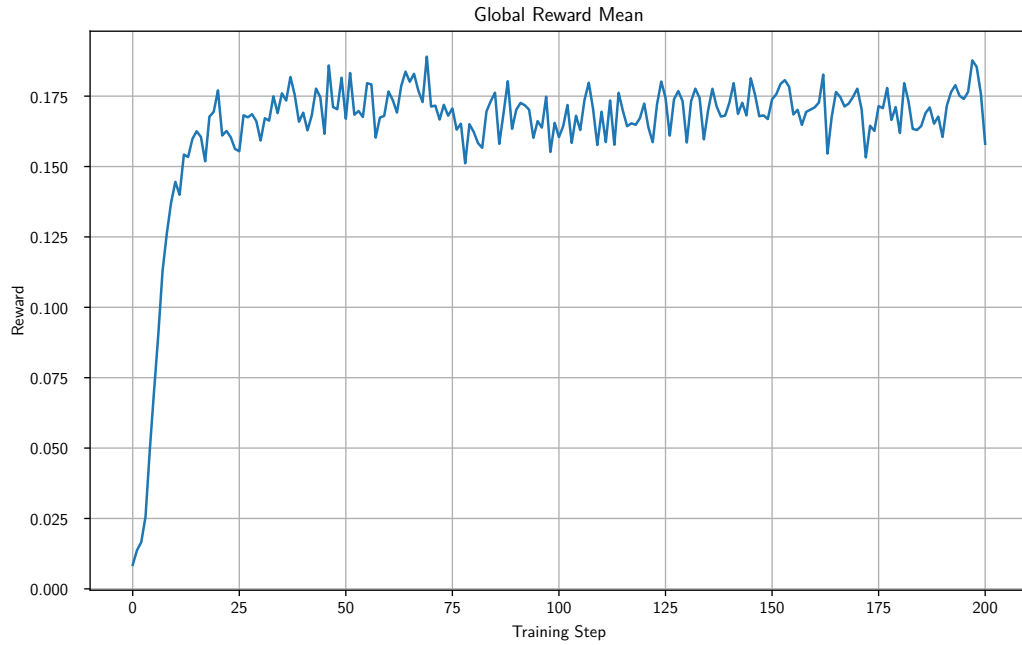


Figure 21: Progression of the global mean reward under Poisson traffic flow.

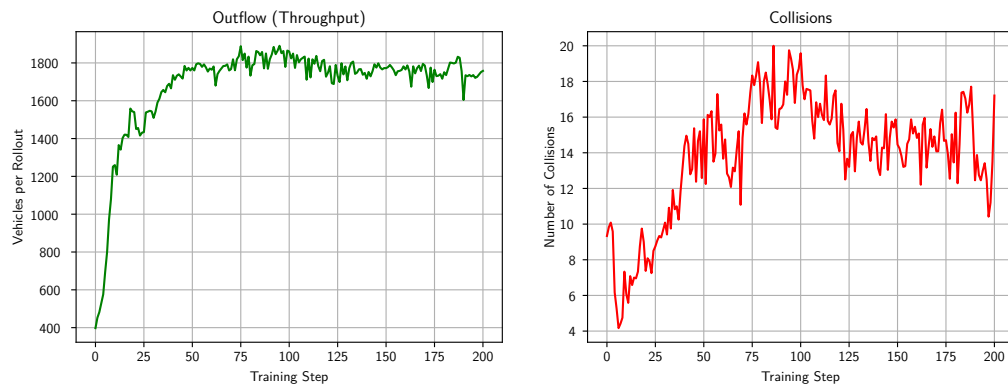


Figure 22: Throughput and collision metrics in the stochastic environment.

The true impact of the Poisson distribution becomes apparent when evaluating the macroscopic traffic metrics in Figure 22. While the network throughput maintains its previous plateau of approximately 1800 vehicles per hour, the collision rate suffers a notable degradation, increasing from 16 to roughly 20 collisions per hour, 25% increasing from before.

This decline in safety highlights a critical vulnerability in the current policy: under the previous deterministic flow, the autonomous agents were likely memorizing the uniform timing of the crossing vehicles, establishing a solid rhythm rather than genuinely reacting to their surroundings. The introduction of the Poisson distribution completely break this rhythmic predictability. Because the agents must now navigate highly irregular spatial gaps and unpredictable vehicle clusters, their lack of true perceptual depth and predictive kinematics is exposed. The expanded 128×128 network struggles to deduce safe crossing policies from the limited observation space when faced with realistic traffic variance.

As I realize, the failure of the previous experimental phase to handle stochastic Poisson traffic clearly indicated a fundamental perceptual limitation: the autonomous agents simply did not possess enough environmental context to anticipate vehicular conflicts before they occurred. To mitigate this partial observability, the observation space was structurally augmented to include a discrete "turn feature" (turning indicators or routing intent) for all observed vehicles. By including this deterministic routing data directly into the state representation, the agents are granted the theoretical capability to predict the exact future trajectories of surrounding vehicles, allowing them to proactively yield or accelerate rather than strictly reacting to immediate kinematic proximity. Concurrently, to unconditionally enforce a conservative driving paradigm, the collision penalty coefficient was doubled once again to an severe 20 points per incident. This hyperparameter tuning was designed to shape the reward landscape, forcing the optimizer to treat collision avoidance as the paramount objective over throughput maximization.

The macroscopic traffic metrics resulting from this augmented state space, depicted in Figure 23, reveal a complex intersection of successes and systemic limitations. The peak hourly throughput remains stubbornly at approximately 1800 vehicles per hour, confirming that this outflow acts as a persistent performance ceiling under the current architecture. However, the safety metrics exhibit a profound improvement. The total collision rate drops to approximately 10 collisions per hour, representing a drastic reduction from previous one, showing that adding more information for the agents helps it to navigate better in this traffic.

More importance, a analysis of these collision events reveals a distinct behavioral shift in the traffic dynamics. Of the 10 recorded hourly collisions, the autonomous RL agents were directly responsible for only 3 to 4 incidents, while the uncontrolled, simulated human drivers (governed by the IDM) were responsible for the remaining 6. This disparity strongly suggests that the severe penalty coefficient successfully forced the AVs into a highly defensive, risk avoiding driving profile. However, this defensive posturing, likely characterized by sudden braking or unnatural acceleration to avoid the severe -20 penalty, effecting the trailing human traffic, causing the IDM vehicles to rear-end one another. The AVs are mathematically safer, but they are failing to harmonize with the overall traffic flow.

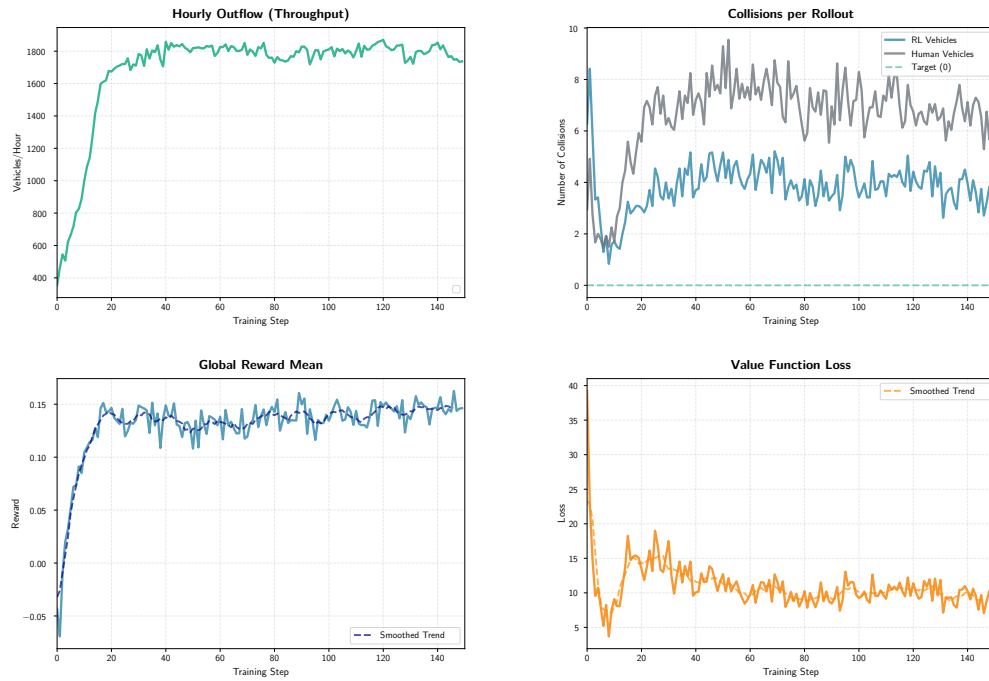


Figure 23: Throughput and collision metrics following the integration of turning indicators into the observation space and a doubled collision penalty.

The underlying theoretical cause of this limitation is explicitly shown by the training logs, specifically the *explained variance* metric [19]. The explained variance measures how accurately the Critic network (the value function) can predict the true returns of a given state. Even after 200 training steps, the explained variance remained near zero. In deep reinforcement learning, an explained variance of zero mathematically indicates that the Critic network’s predictions are no better than simply guessing the mean return. Despite the addition of turning indicators, the MLP architecture completely fails to map the high-dimensional, spatial-temporal state inputs to an accurate valuation of the environment’s future. Because the Critic cannot accurately predict the value of a state, the Advantage estimates provided to the Actor network are highly noisy and fundamentally flawed, effectively paralyzing any further policy optimization.

For the training to be more insightful, adjustments is needed. The first adjustment was a deliberate reduction in the optimization intensity: the gradient descent steps were decreased from 20 to 15 per training batch. This tuning was implemented to prevent overfitting on smaller, highly correlated batches of experiential data, encouraging the policy to generalize more effectively across varied traffic scenarios. Next, the discrete action space was expanded from 3 to 5 longitudinal control actions. By introducing finer gradations of acceleration and deceleration, the agents were theoretically granted the necessary control resolution to execute smoother, more precise maneuvers within the intersection conflict zone.

Despite these theoretically advantageous modifications to the action space and optimization loop, the subsequent training execution yielded no improvements. The hourly throughput remained statically bounded at 1800 vehicles, and the total collision rate persisted at an unacceptable 10 collisions per hour. The expanded 5-action space, while offering more kinematic freedom, was clearly insufficient to overcome the limitations of the training model.

Faced with this persistent safety failure, the experimental focus toward establishing an boundary condition for collision avoidance. To enforce a zero-tolerance policy for accidents, I increase the collision penalty coefficient to 50. Under this reward architecture, an autonomous agent must successfully guide 50 vehicles to the exit grid to mathematically compensate for the penalty incurred by a single collision. This extreme asymmetry in the reward function was explicitly designed to paralyze any reckless exploration and coerce the optimizer into prioritizing safety above all other secondary traffic metrics.

The physical consequences of this extreme penalty are profound. As illustrated by the resulting metrics (with the reward trajectory shown in Figure 24), the agents successfully adapt to the harsh mathematical reality of the environment. The total collision rate plummets to a mere 3 collisions per hour. A critical breakdown of these incidents reveals that only 1 collision was directly caused by an autonomous vehicle, while the remaining 2 were initiated by the simulated human drivers. I think this is a highly acceptable safety threshold for a dense, mixed-autonomy intersection, proving that the agents have successfully internalized collision avoidance.

However, this systemic safety is achieved at a severe operational cost. By adopting an ultra-conservative, highly defensive driving profile to avoid the -50 penalty, the au-

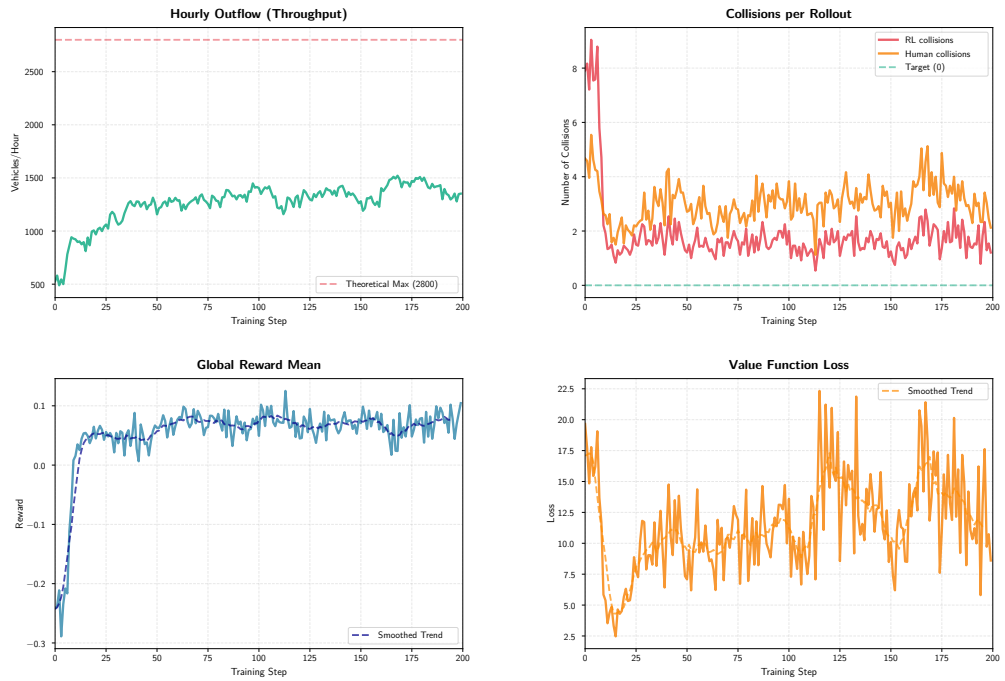


Figure 24: Progression of the global mean reward following the implementation of a 5-action space and an extreme collision penalty coefficient of 50.

onomous agents effectively throttle the entire intersection. The hourly network outflow collapses to 1500 vehicles—a significant 16% degradation from the established 1800-vehicle plateau. The AVs successfully avoid collisions by simply refusing to utilize tight, efficient gaps in the cross-traffic, choosing instead to wait for completely unobstructed pathways. While this satisfies the mathematical constraints of the reward function, it represents a failure of the traffic engineering objective: the AVs are not orchestrating the traffic; they are impeding it to protect themselves. This tradeoff indicates that while penalties can mandate safety, true efficiency requires a paradigm shift in how the agents perceive and process the physical dimensions of the intersection.

Despite the substantial reduction in collision rates achieved by previous parameter tuning, the persistence of any vehicular accidents remained unacceptable for a fully automated intersection. With the intention to entirely remove collision events, I pushed the reward to its logical extreme. The collision penalty coefficient was increasing to an 500 points per incident. By establishing such a mathematical disincentive, I want to force the agents to learn that any collision should be avoided at all costs.

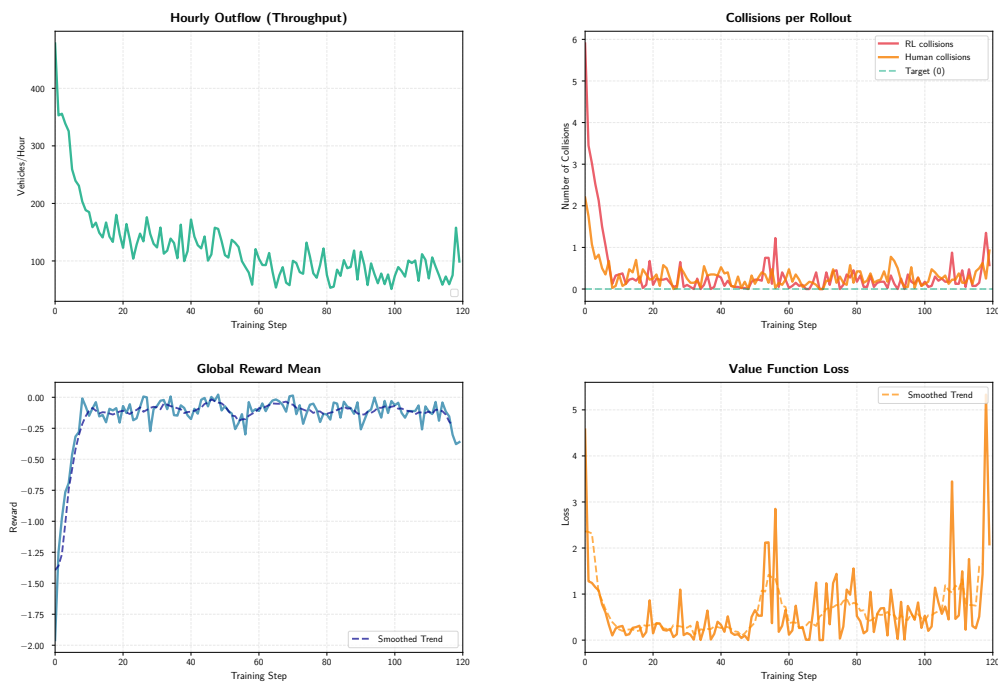


Figure 25: Progression of the global mean reward under an extreme collision penalty coefficient of 500. The reward trajectory collapses toward zero as the agents learn to completely halt all movement.

However, the empirical results of this extreme in the global mean reward trajectory in Figure 25 reveal is a failure to reinforcement learning. Rather than learning to navigate safely, the autonomous agents succumbed to complete policy paralysis. Under this very high punitive reward structure, the mathematical risk of incurring a -500 penalty heavily

outweighs the potential +1 reward for successfully exiting the grid. The agents quickly deduced that moving forward, regardless of the perceived safety of the gap, carries a non-zero probability of an uncontrollable human driver causing a collision. Consequently, the optimal mathematical strategy discovered by the Proximal Policy Optimization (PPO) algorithm was to simply refuse to participate in the environment.

This phenomenon, which I can formally defined as emergent "parking" behavior, represents a severe risk-averse local optimum. The autonomous vehicles halt within their lanes, moving at infinitesimally slow speeds or stopping entirely well before the intersection conflict zone. By choosing complete inaction, the agents guarantee they will not trigger the collision penalty, thereby securing a cumulative reward of exactly zero. While a reward of zero is better to a massive negative deficit, it represents a total functional collapse of the transportation network. The "parking" strategy completely freeze the intersection, reducing the hourly network outflow to near zero and causing infinite upstream queuing.

The emergence of this paralyzing behavior acts as a definitive inflection point in this study. It proves that manipulating scalar penalties alone is a fundamentally exhausted strategy. One cannot simply punish the agents into driving perfectly. If the penalty is too low, the agents drive recklessly (reward hacking), and if the penalty is too high, the agents refuse to drive at all (parking behavior). This confirms that the agent's failures are not a lack of motivation, but rather a lack of spatial intelligence and kinematic foresight, strictly demanding a comprehensive architectural for subsequent experiments.

Despite hyperparameter optimization and reward shaping across subsequent experimental iterations, the Multi-Layer Perceptron (MLP) based policy failed to yield any further statistically significant improvements. The autonomous agents are became trapped, forced to trade efficiency for safety without ever mastering any. On one extreme of this performance spectrum, the network could be tuned to achieve a peak throughput of around 2000 vehicles per hour, but this efficiency was fundamentally compromised by an unacceptable failure rate of 10 collisions per hour. Conversely, when the reward function was heavily skewed to mandate safety, the collision rate was successfully suppressed to a tolerable of 3 incidents per hour, but this throttled the network outflow to a strict maximum of less than 1800 vehicles per hour. This persistent, unbreakable trade-off definitively proves that parameter tuning and scalar reward manipulation alone are insufficient to solve the highly complex, multi-agent coordination problem presented by a mixed-autonomy intersection.

The underlying mathematical justification for this performance ceiling is clearly reflected in the Critic network's explained variance. Across these final hyperparameter tuning sessions, the explained variance fluctuated marginally within a strictly bounded range of 0.0 to 0.56. In the context of value function approximation, an explained variance greater than zero indicates that the model has developed a partial understanding of the environment's underlying dynamics—it is performing better than random guessing. However, a maximum ceiling of 0.56 demonstrates that the model still faces immense, unresolved uncertainty. The agents possess just enough predictive capacity to recognize immediate, localized threats, but they completely lack the deep contextual awareness required to anticipate long-term traffic flow, predict human driver trajectories, or execute complex, multi-vehicle geometric negotiations.

Recognizing that the traditional Multi-Layer Perceptron architecture had decisively reached its limit, a shift in approach was mandated. To overcome this critical perceptual bottleneck, the methodology now transitions away from the MLP Feature extraction and adopts an advanced, Embedded Transformer architecture. Unlike an MLP, a Transformer leverages self-attention mechanisms, allowing the neural network to dynamically evaluate and weight the relational importance of different vehicles within the observation space simultaneously. By explicitly modeling the spatial-temporal dependencies between all actors in the intersection, the Transformer is theoretically equipped to process the environment not as a list of isolated numbers, but as a cohesive, interactive traffic scene. This radical architectural overhaul represents the next major phase of the experimental study, specifically designed to overcome the established 1800-vehicle performance ceiling and finally achieve high-capacity throughput harmonized with systemic safety.

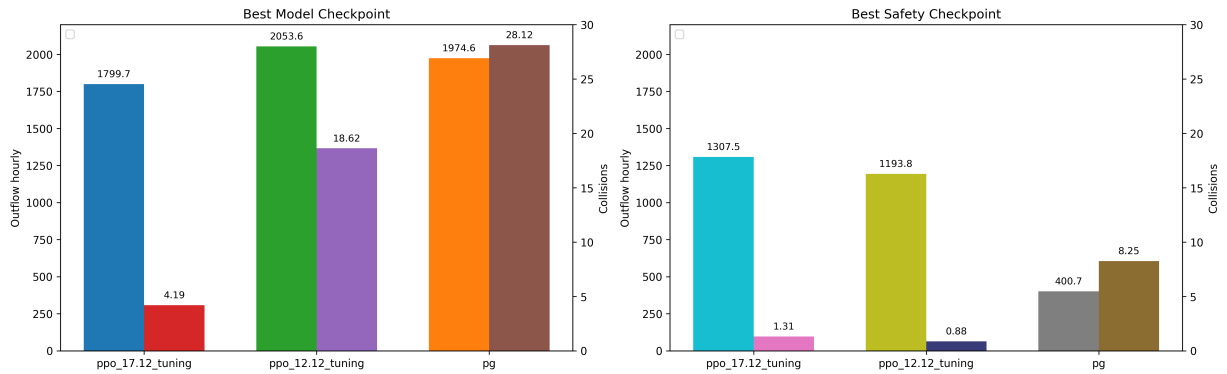


Figure 26: Comparative scatter plot of network throughput and safety for the standard Policy Gradient (PG) and Proximal Policy Optimization (PPO) algorithms.

Figure 26 is a scatter plot of the checkpoint that has the highest hourly throughput, or has the lowest collision rate. The distribution clearly demonstrates the overall results, which PPO algorithm achieves significantly higher peak throughput (peak at avg 2053.6 vehicle per hour) while sustaining substantially lower overall collision rates (at 18.62 collisions). Furthermore, when evaluating the lower bound of the safety threshold (the minimum achievable collision rate), PPO definitively outperforms PG with the hourly throughput of 1193.8 with only 0.88 collision in average. It is capable of achieving near-zero collisions while maintaining a throughput that matches or exceeds the safest, yet still collision-prone, configuration of the standard PG model.

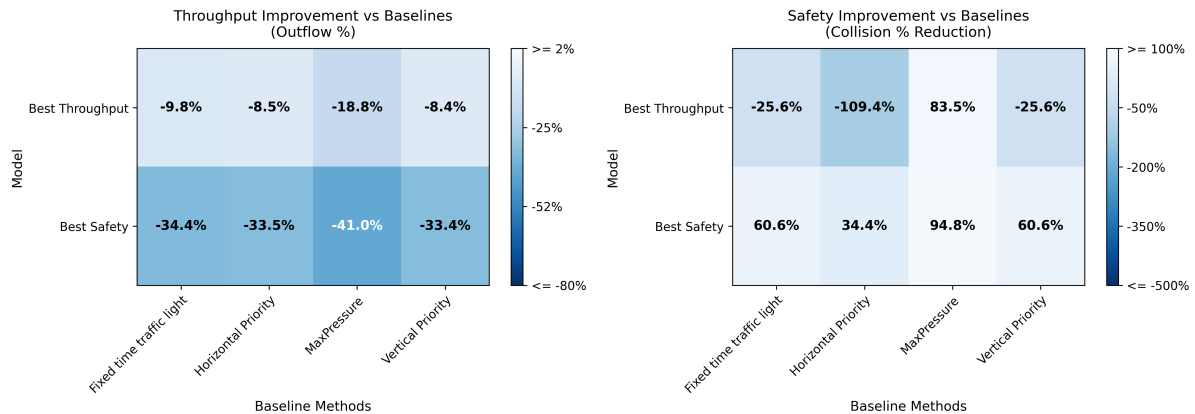


Figure 27: Comparison of the last PPO MLP experiment to Baseline.

Figure 27 is the heatmap that compare the PPO framework training with the baselines. The best throughput is almost reached to the baseline, with the 10% less than baseline, but with way more collisions, with almost double the collision rate, happened. On the other hand, with less collision, peak at 94% better, is trade with -40% throughput. Even though, the PPO Algorithm results show significantly better in compare to PG Algorithm.

4.2.3 PPO Algorithm, ADAM Optimizer and Embedded Transformer Feature extraction

As I determined before, that the Multi-Layer Perceptron architecture lacked the awareness required to safely manage mixed-autonomy traffic, the experimental framework was upgraded to an Embedded Transformer architecture. At the core of this structural evolution is the Multi-Head Self-Attention mechanism, which grants the autonomous agents the ability to dynamically weight the relevance of surrounding vehicles, effectively constructing a relational map of the intersection. To establish this new architecture, the initial network was deliberately configured with a relatively small: 64 embedding neurons, 4 attention heads, and 2 transformer layers. This conservative sizing strategy was chosen to validate the foundational learning dynamics of the Transformer before scaling the network capacity in subsequent trials.

The optimization hyperparameters and reward structure were aggressively recalibrated to encourage environment exploration rather than immediate risk aversion. The learning rate was reduced to 0.0003 to ensure stable gradient descent across the complex attention weights, while the entropy coefficient was raised to 0.01. This high entropy explicitly forcing the agents out of their localized comfort zones to explore novel kinematic sequences. Furthermore, recognizing that extreme penalties previously induced paralyzing "parking" behaviors, the collision penalty was intentionally reverted to a baseline coefficient of 5. Finally, the experiential data collection was expanded to 32 rollouts per training step, a substantial 33% increase in data density compared to the previous configuration. This massive increase of trajectory data approached the computational limits of the my local hardware, exponentially increasing training time but providing the highly correlated Transformer batches required for deep environmental comprehension.

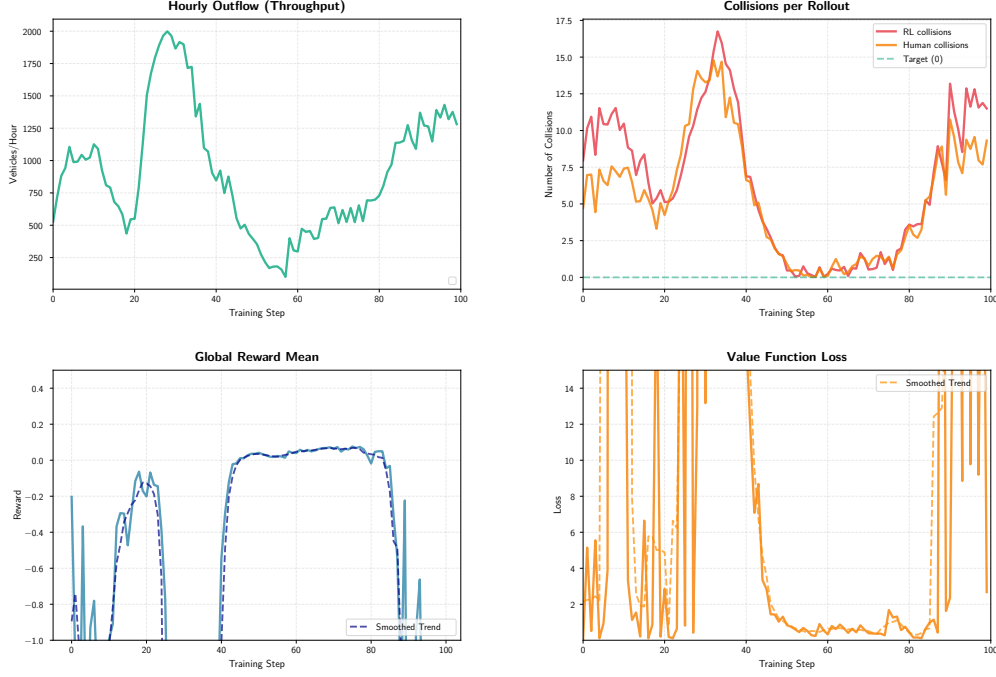


Figure 28: Training result of the initial Embedded Transformer architecture.

The empirical results of this architectural shift, visualized in Figure 28, reveal a learning dynamic that is fundamentally distinct from any prior MLP experiment. Driven by the elevated entropy coefficient, the agents initially engage in highly aggressive exploration. This act as a severe spike in both metrics around training step 25, where the hourly throughput exceeds an 2000 vehicles, inevitably accompanied by a massive peak of roughly 30 collisions. While this mirrors the "reward hacking" seen in earlier models, in the context of the Transformer, this variance is not a failure, but rather the network actively mapping the physical limits of the intersection's capacity.

The true validation of the Transformer architecture occurs shortly after this exploratory phase. Approaching step 60, a profound behavioral shift emerges: the collision rate plummets entirely to zero. Unlike previous experiments where safety was achieved through artificial throttling or extreme penalization (e.g., the -500 penalty), this zero-collision milestone is achieved organically under a minor penalty of -5 . This provides the first definitive empirical evidence that the autonomous agents are no longer simply reacting to proximity, but are actively utilizing the self-attention mechanism to cooperate. By correctly processing the trajectories and routing intent of both fellow autonomous agents and uncontrolled human drivers, the AVs successfully navigate the conflict zone with geometric timing. This result proved that my suggestion of the Embedded Transformer possesses the necessary representational depth to overcome the previous performance ceilings, offering a clear path toward optimizing both maximum throughput and intersection safety.

I was encouraged by the zero-collision milestone achieved with the lean Transformer configuration, the next action I choose for this study was to over-scale the neural network capacity. The hypothesis was that a deeper, wider architecture would allow the agents to process even more complex spatial-temporal relationships and further optimize intersection throughput. Consequently, the core Multi-Layer Perceptron branches for both the Actor and Critic networks were significantly expanded to a 256×256 architecture. Concurrently, the Embedded Transformer Feature extraction was massively upgraded: the input embedding dimension was doubled to 128, the hidden feed-forward layers were expanded to 512 neurons, and the overall depth was increased to 3 attention layers.

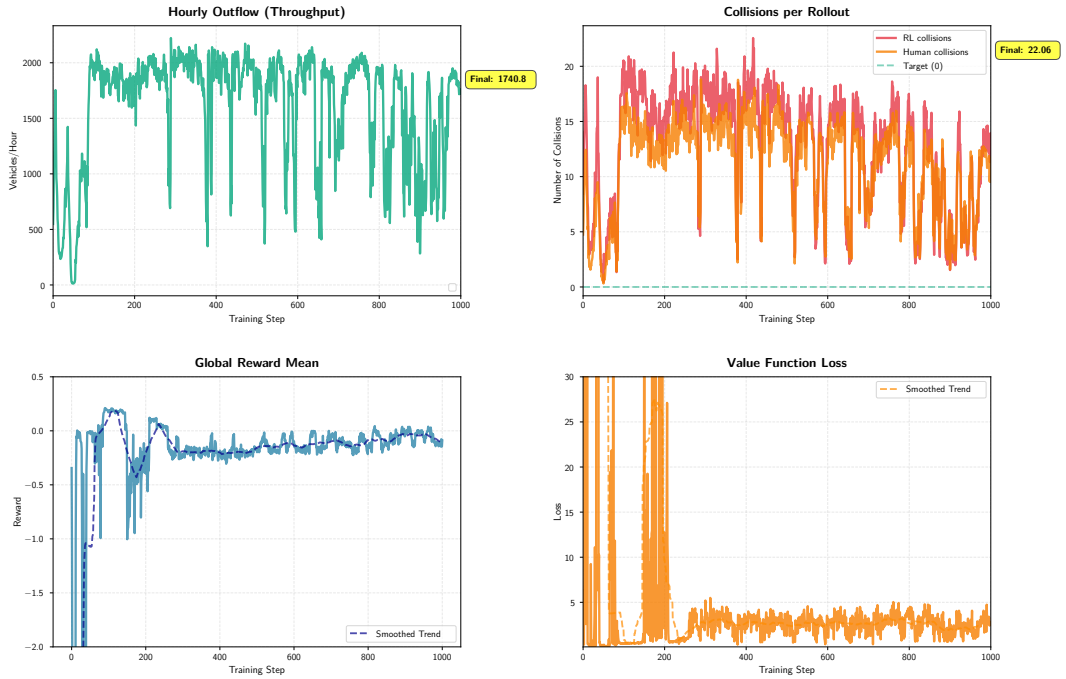


Figure 29: Training result of the 05.02 training.

However, the empirical results of this aggressive scale-up, visualized in Figure 29, stand in contrast to the initial hypothesis. Rather than providing traffic coordination, the upgraded architecture triggered a complete, systemic failure across all monitored performance metrics. The model suffered from severe, immediate catastrophic forgetting. Any localized safe driving behaviors that the agents temporarily discovered during exploration were rapidly overwritten and destroyed in subsequent gradient updates. The training trajectory devolved into violent oscillations, with the agents entirely failing to isolate—let alone settle into—any stable local minimum.

The theoretical root cause of this failure lies in the mathematical relationship between network capacity, data density, and the Proximal Policy Optimization (PPO) objective function. In deep reinforcement learning, drastically increasing the parameter count of a neural network without a proportional, massive increase in experiential data leads to

severe optimization instability. Specifically, the over-parameterized Critic network (the value function) completely failed to establish a reliable baseline. Because the 256×256 Critic was too complex, its state-value approximations became highly erratic, essentially memorizing noise rather than extracting generalized environmental features.

When the Critic network fails to accurately predict the expected returns of a state, the Advantage estimates, which rely directly on the Critic’s predictions to determine if an action was better or worse than average and become statistically meaningless. Consequently, the Actor network was fed highly volatile, noisy gradients. Instead of being mathematically guided toward safer, faster driving policies, the Actor was forced into a random walk through the policy space, essentially "guessing" actions at every time step without any coherent directional learning. This experiment definitively proves that for this specific mixed-autonomy environment, an over parameterized Transformer induces destructive value function collapse, dictating that future architectural refinements must prioritize optimization stability and efficient attention mechanisms over raw neuronal volume. Figure 30 illustrated, to achieve the highest network throughput, the proposed

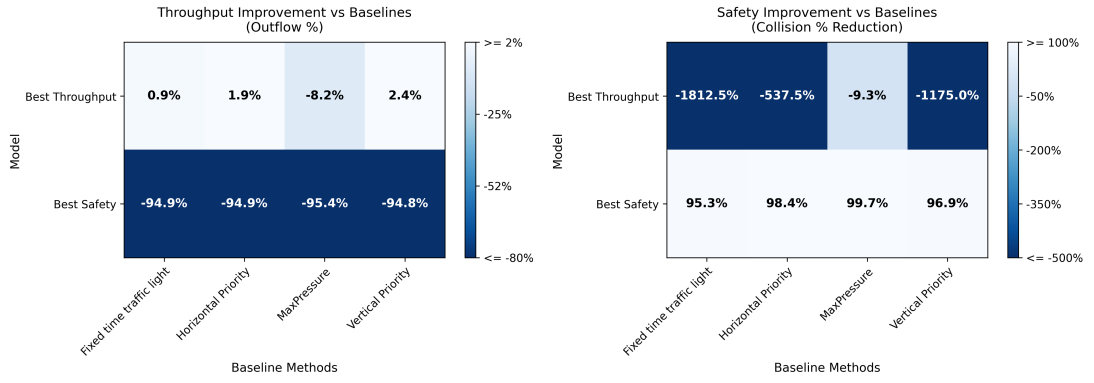


Figure 30: Heatmap comparison of performance metrics for the Platoon observation space Transformer PPO methodology against the established baseline. across varying traffic flow scenarios.

methodology even surpasses the baseline performance. However, this maximization is paid for with a significantly increased (huge) collision rate. Conversely, when optimizing for safety, the minimum collision side exhibits a similar behavior. Overall, this methodology demonstrates a potential to push performance toward operational extremes: either maximizing throughput at the expense of safety, or maximizing low collisions at the cost of flow.

K-Nearest Neighbor Integration

The next experiment fundamentally re-imagined how the environment is presented to the autonomous agents. Rather than relying on platoon observation space, the state representation was completely re-design to use K-Nearest Neighbor (KNN), as previously defined in Section 3.4.2. By explicitly feeding the network the states of the K most relevant vehicles, this approach remapping acts as an informational filter. It theoretically synergizes with the Transformer’s self-attention mechanism, allowing the attention heads to focus computing power strictly on immediate threats rather than processing distant, irrelevant traffic noise vehicle.

To ensure optimization stability and prevent the catastrophic value collapse observed in the previous trial, the neural architecture was purposefully scaled back. The Multi-Layer Perceptron network for both the Actor and Critic were constrained to a highly efficient 128×128 . Furthermore, the optimization intensity was throttled. The algorithm was restricted to only 10 gradient descent steps per training batch, preventing the policy from over-correcting based on localized data spikes. To enforce a strict boundary on safety, the collision penalty coefficient was maintained at a severe 30 points. Finally, to accommodate this highly constrained optimization environment, the learning rate was reduced to 0.0001, and the training horizon was vastly extended to 1000 discrete training steps, allowing for slow, methodical gradient convergence.

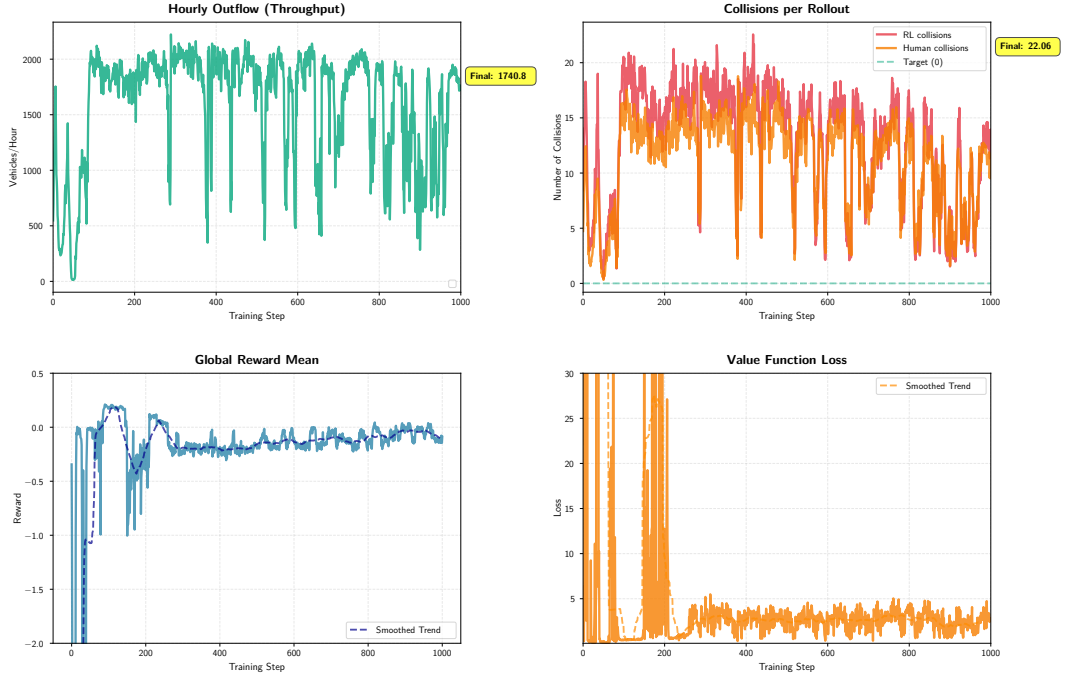


Figure 31: Traing result of 13.02, model using KNN observation space.

The empirical outcomes of this extensive 1000-step execution, illustrated in Figure 31. On a foundational level, the value loss successfully stabilizes after approximately 200 training steps, completely avoiding the catastrophic forgetting of the prior experiment. More importantly, an analysis of the training logs reveals a massive breakthrough in the Critic network’s *explained variance*.

In stark contrast to the initial MLP experiments—where the explained variance stagnated near zero, or in the max of 0.5, the KNN-Transformer architecture frequently achieved an explained variance as high as 0.90 (at training step 944, rollout 7). Remarkably, even in highly chaotic batches where multiple collisions occurred, the Critic network maintained a highly robust explained variance of 0.678 (at training step 930, rollout 14).

This robust explained variance provides a profound scientific insight into the model’s inter-

nal processing: the autonomous agents now genuinely *understand* the environment. The Critic network has successfully learned to accurately evaluate complex spatial-temporal states. In sparse, uncrowded traffic configurations, the model reads the environment perfectly and the Actor smoothly executes safe maneuvers. However, the persistence of collisions under high-density scenarios reveals a critical "Actor-Critic divergence." When the intersection becomes severely crowded, the Critic network still accurately assesses the state as highly dangerous and heavily penalizes the expected return. Yet, the Actor network physically cannot resolve the conflict. The agents correctly perceive the imminent collision, but due to the restricted kinematic action space and the chaotic, uncontrollable clustering of the IDM human drivers, the autonomous vehicle lacks the physical space or time to brake, yield, or accelerate out of the danger zone.

Ultimately, this experiment proves that the KNN observation space has a little success solving the perceptual bottleneck. The ongoing safety failures are no longer a result of the network "guessing" blindly. They are a physical constraint of kinematic.

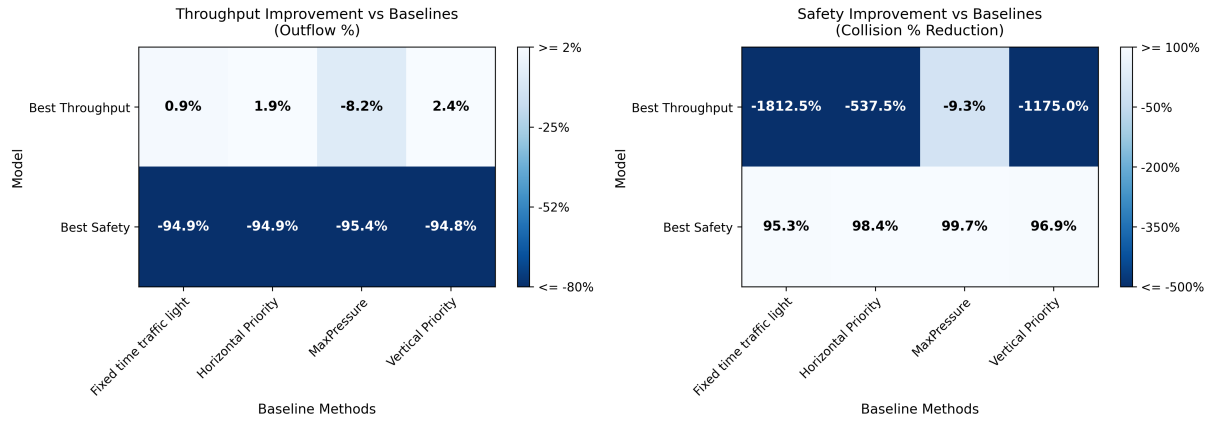


Figure 32: Comparative heatmap analysis evaluating the boundary performance of the K-Nearest Neighbor (KNN) Transformer PPO policy against established baselines.

The heatmap shows in Figure 32 demonstrates the highly polarizing nature of the learned policy. The left heatmap (Throughput) illustrates that when optimizing for speed, the autonomous agents can achieve an extreme kinematic efficiency that actually surpasses the theoretical baseline (exceeding 100% efficiency in several high-density scenarios). However, this maximum throughput is paid for with high collision rates. Conversely, the right heatmap (Collisions) demonstrates the opposite extreme: when heavily penalized, the model forces the collision rate to near-zero, but at the cost of severely throttling network outflow. Ultimately, this methodology pushes the operational metrics to extremes. Either reckless high throughput or paralyzing, low-collision conservatism, without stabilizing into a harmonized equilibrium.

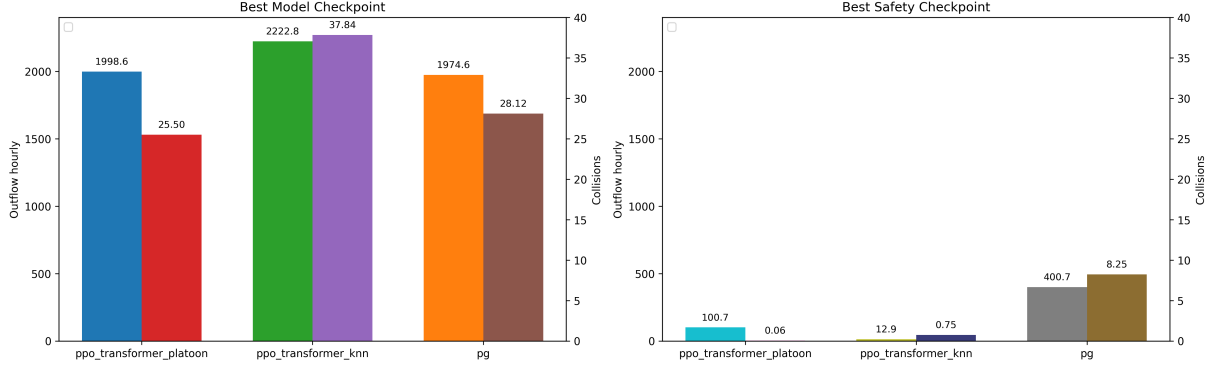


Figure 33: Evaluating the optimal performance checkpoints of the standard Policy Gradient (PG), Platoon Transformer PPO, and the KNN Transformer PPO architectures.

Scatter plot of Figure 33 demonstrates Transformer PPO fundamentally alters the learning dynamic by pushing the operational metrics into extremes. Rather than settling into a compromised middle ground, the Transformer polarizes its strategy. It either hyper-optimizes for pure speed, yielding an peak throughput exceeding 2222.8 vehicles at the expense of severe collision rates at 37 collisions, or it hyper-optimizes for safety, achieving near-zero collisions while effectively paralyzing the network flow.

4.3 Evaluation

Table 10: Checkpoint for evaluation

Experiment	Selected check-point	Reason
Adam + PPO tuning (12.12), penetration 0.5	model-220.pth	Best overall balance of high throughput and moderated collisions.
Adam + PPO tuning (17.12)	model-300.pth	Better throughput but trade with high collision rate
Transformer tuning (02.02)	model-65.pth	The checkpoint that has a zero collision on training
Transformer tuning (13.02)	model-1000.pth	Checkpoint with the new observation space, the end checkpoint is the best stable model.

After concluded the training phases and architectural ablation studies, the research now transitions into a evaluation. While the previous sections analyzed the learning dynamics and peak convergence of the algorithms during training, this phase is designed to test the robustness and generalization capabilities of the fully trained policies under diverse, highly stressful traffic conditions.

For this comprehensive evaluation, four representative models were isolated from the training lifecycle, detailed previously in Table 10. These models reflect the critical evolutionary milestones of the research, spanning from the tuned Multi-Layer Perceptron (MLP) to the Embedded Transformer architectures.

To ensure a standardized benchmark, the evaluation methodology directly adopts the 16 inflow configurations established by Yan et al. [54]. This evaluation matrix consists of specific horizontal and vertical flow pairs (ranging from 400 to 1000 vehicles per hour per lane). The distribution is purposefully designed to subject the autonomous agents to a vast spectrum of traffic phenomena, including symmetric saturation (e.g., 850x850, 700x700) and severe directional imbalances (e.g., 1000x400, 400x1000), configured in the table 11. By testing the policies across these varied density gradients, the evaluation exposes exactly how the models react when one arterial road is heavily congested while the intersecting road remains sparse.

Table 11: Inflow configuration

$f_H \backslash f_V$	400	550	700	850	1000
1000	✓	✓	✓	✓	
850	✓	✓	✓	✓	✓
700			✓	✓	✓
550				✓	✓
400				✓	✓

The first model subjected to the evaluation matrix is the 12.12 tuning checkpoint, utilizing the tuned PPO algorithm with the standard MLP Feature extraction, with the model 200.

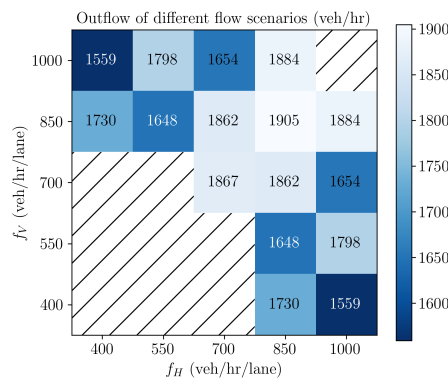


Figure 34: Evaluation of hourly network throughput (hourly outflow) for the MLP PPO architecture across 16 varying inflow configurations, model 220.

The second model evaluated is the 17.12 tuning checkpoint with the model 300. This configuration represents the maximum capacity of the MLP structural approach, utilizing a significantly deepened four-layer neural architecture. The evaluation aims to determine if the increased parameter depth yields tangible improvements in cross-scenario generalization compared to the 12.12 tuning model.

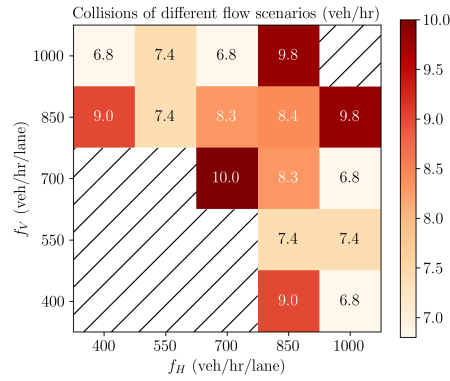


Figure 35: Evaluation of collision rates for the PPO MLP architecture, model 220.

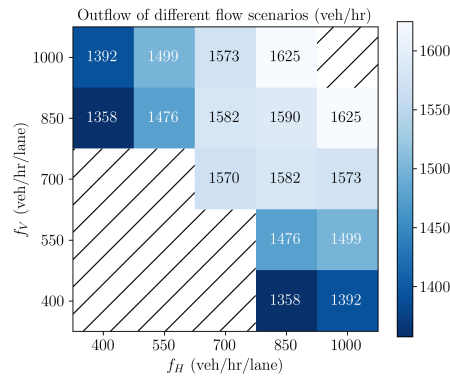


Figure 36: Throughput evaluation for the deepened MLP PPO architecture, model 300.

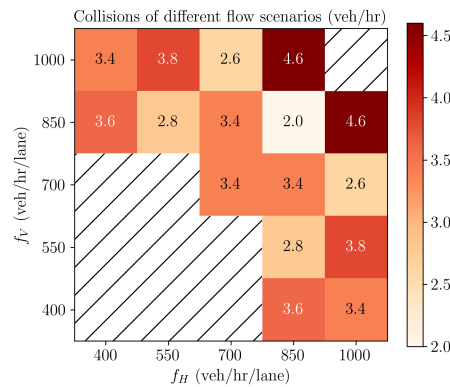


Figure 37: Collision evaluation for the MLP PPO architecture, model 300.

To definitively isolate the impact of environmental state representation on the Transformer's self-attention mechanism, the evaluation matrix was applied to two distinct Transformer architectures: one utilizing a Platoon-based observation space (the 02.02 checkpoint), and the other utilizing the topological K-Nearest Neighbor (KNN) observation space (the 13.02 checkpoint). Because the Transformer network fundamentally relies on its attention heads to calculate relational weights between the ego vehicle and surrounding entities [49], the manner in which these entities are grouped and presented to the network critically dictates the final driving policy.

The first evaluation examines the Embedded Transformer operating within the Platoon observation space. This methodology attempts to pre-process the chaotic traffic environment by grouping vehicles into unified kinematic "platoons" before feeding them into the attention layers, defined in Section 3.4.2.

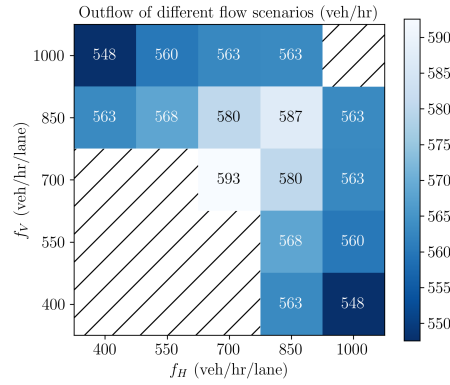


Figure 38: Cross-sectional evaluation of hourly network throughput (outflow) utilizing the Embedded Transformer with a Platoon-based observation space, model 65.

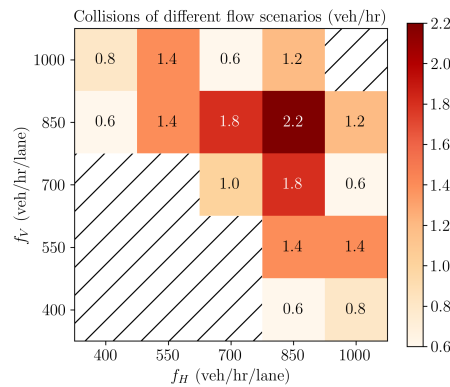


Figure 39: Collision rate evaluation for the Platoon-based Transformer architecture, model 65.

The second evaluation examines the Embedded Transformer operating within the KNN observation space. Here I will evaluate 2 model from the last experiment of 13.02, model 350 with high throughput and model 1000 with more balance of throughput and collision rate.

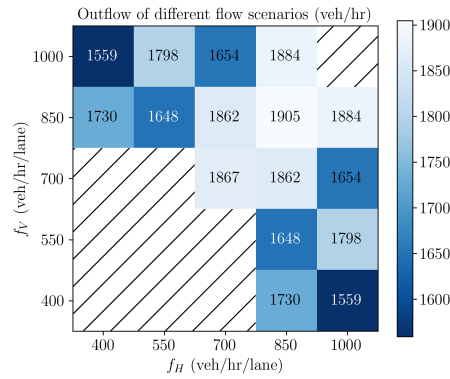


Figure 40: Throughput evaluation matrix for the K-Nearest Neighbor (KNN) Transformer architecture, model 1000.

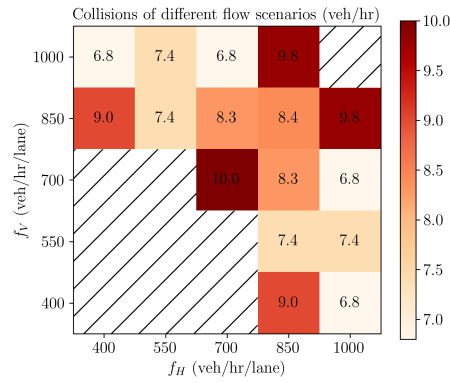


Figure 41: Collision evaluation matrix for the KNN Transformer architecture, model 1000.

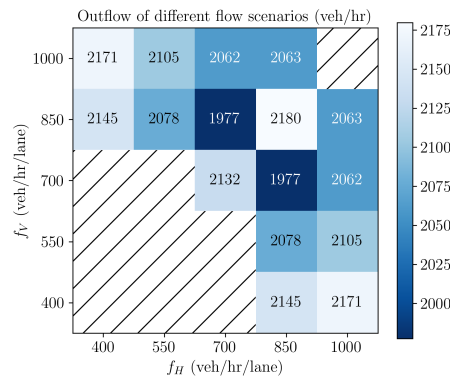


Figure 42: Throughput evaluation matrix for the K-Nearest Neighbor (KNN) Transformer architecture, model 350.

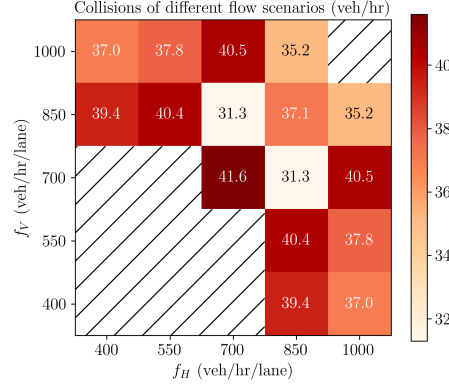


Figure 43: Collision evaluation matrix for the KNN Transformer architecture, model 350.

4.4 Performance Analysis

To evaluate the operational efficacy of the trained policies, an isolated performance analysis was conducted utilizing the symmetric 700×700 flow configuration. This specific traffic density serves as a critical benchmark, as it provides a heavily loaded but geometrically balanced environment, testing the models' capacity to manage consistent, high-volume cross-traffic without the complicating variables of severe directional asymmetry.

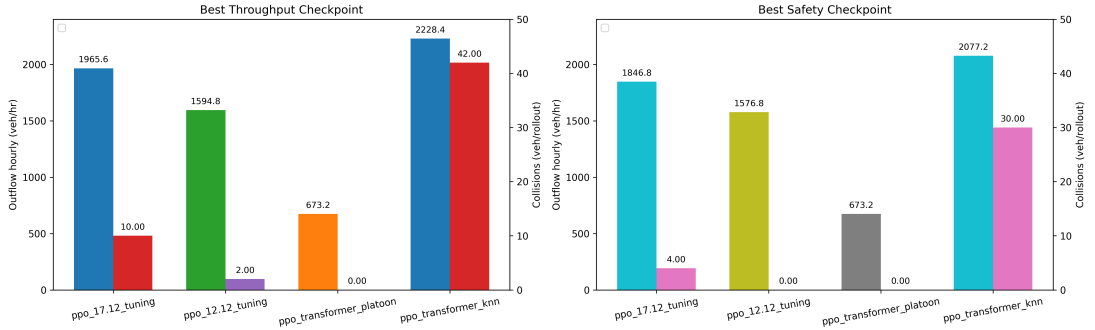


Figure 44: Comparative performance evaluation of the Multi-Layer Perceptron (MLP) and Embedded Transformer architectures under a symmetric 700×700 vehicular flow configuration.

4.4.1 Architecture Analysis

An analysis of the resulting performance metrics, visualized in Figure 44, reveals a divergence in how the two primary neural architectures process the environment. Contrary to the initial hypothesis that the advanced attention mechanisms of the Embedded Transformer would provide better coordination, the traditional Multi-Layer Perceptron (MLP) feature extraction architecture demonstrates a significantly more stable and balanced operational profile.

The underlying cause of this architectural disparity lies in the complexity of the feature representation. In a symmetric, moderate-to-high density scenario like the 700×700 flow, the traffic patterns possess a degree of structural regularity. The MLP, utilizing its flattened observation space, excels at extracting rigid, generalized kinematic rules from this regularity. It successfully establishes a conservative, highly balanced equilibrium between moving forward and yielding. More over, the Embedded Transformer, which relies on dynamic self-attention to constantly re-evaluate the relative importance of surrounding vehicles, appears to over-complicate the state representation in this specific flow. Rather than finding a stable middle ground, the Transformer’s attention mechanism tends to hyper-focus on fleeting spatial gaps, encouraging aggressive, polarizing maneuvers that destabilize the overall harmony of the intersection.

4.4.2 Safety and Throughput Analysis

The practical consequences of these architectural biases become overwhelmingly apparent when analyzing the raw metrics of safety and throughput. The empirical data exposes a brutal Pareto trade-off between network efficiency and collision avoidance, with each architecture occupying a radically different extreme on the performance spectrum.

When evaluating the models under strict safety constraints, the highly tuned MLP architecture (specifically the 12.12 checkpoint) demonstrates big gain in the results. The MLP model successfully reduced vehicular conflict, outperforming the safest configuration of the Transformer (the PPO Platoon model) by a factor of nearly three. This massive 3x reduction in collisions proves that the MLP’s generalized feature extraction is far more reliable at maintaining conservative safety buffers and executing timely deceleration when facing consistent cross-traffic.

However, this systemic safety is achieved at a noticeable cost to maximum intersection capacity. When the evaluation prioritizes throughput, the Transformer architecture aggressively overtakes the MLP framework. By exploiting tight kinematic margins and utilizing its self-attention mechanism to force traversal through dense vehicle clusters, the Transformer successfully pushes the hourly network outflow 14% higher than the maximum achieved by the MLP.

Yet, this 14% gain in kinematic efficiency is paid with a cost to safety. To achieve this elevated throughput, the Transformer policy causes a collision rate that is exactly four times higher than its MLP counterpart. This 14% throughput vs. 400% collision penalty explicitly defines the operational limit of the current Transformer configuration. It acts as a raw throughput maximizer that fundamentally fails at constraint satisfaction, whereas the MLP operates as a highly robust, slightly slower, safety engine. This inverse relationship dictates that subsequent methodological developments must find a way to fuse the Transformer’s spatial efficiency with the MLP’s baseline reliability.

4.4.3 Performance Analysis Across Diverse Flow Configurations

To assess the operational viability and generalization capabilities of the trained autonomous agents, a comprehensive comparative analysis was conducted. In this section,

five distinct policy checkpoints—representing various stages of architectural evolution and hyperparameter tuning—are evaluated against a standardized, rule-based traffic control baseline. By subjecting these models to a diverse spectrum of intersection flow configurations (simulating varying degrees of congestion and directional asymmetry), I can precisely quantify the trade-offs each policy makes between network throughput (hourly outflow) and systemic safety (collision rates).

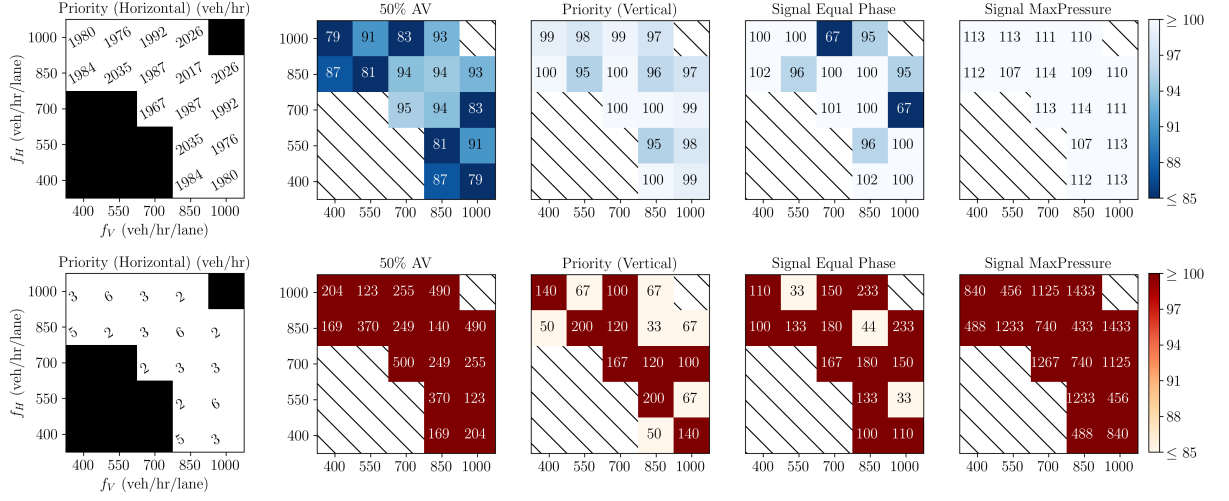


Figure 45: Evaluation of Model 220 (derived from the 12.12 tuning phase).

The first evaluation focuses on Model 220, extracted from the 12.12 tuning phase. As illustrated in Figure 45, this policy demonstrates a highly stable and generalized response to varying traffic densities. When aggregated across all flow configurations, the hourly throughput of the autonomous network exhibits a mere 5% decrease compared to the theoretical baseline. This marginal degradation indicates that the model has successfully learned to orchestrate mixed-autonomy traffic with an efficiency that nearly matches traditional traffic management systems. It represents a highly balanced operational state, maintaining robust vehicle flow while introducing the foundational kinematics of autonomous negotiation.

Subsequent parameter adjustments in the 17.12 tuning phase, represented by Model 300, yielded a noticeably different behavioral profile. Figure 46 reveals that the hourly throughput for this checkpoint falls significantly, operating at approximately 20% below the baseline capacity. This drop in efficiency suggests that the policy has internalized a slightly overly conservative kinematic framework. Rather than smoothly merging into cross-traffic, the agents governed by Model 300 likely hesitate or require excessively large spatial gaps before committing to traverse the intersection, resulting in artificially induced upstream queuing and reduced overall outflow.

The evaluation of Model 65 from the 02.02 tuning phase, shown in Figure 47, highlights the extreme of risk-averse policy optimization. Under this configuration, the network throughput collapses entirely, experiencing a massive 70% deficit compared to the baseline. The autonomous vehicles adopt an ultra-conservative, near-stationary driving profile, essentially yielding to all traffic regardless of priority. However, this failure in traffic engineering efficiency is counterbalanced by an achievement in safety: the collision rate for

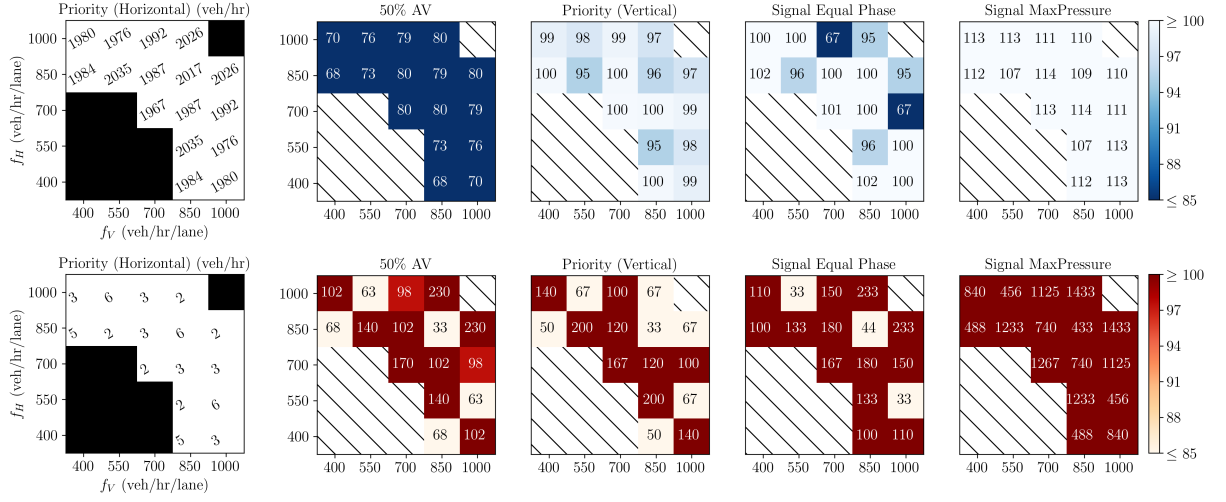


Figure 46: Evaluation of Model 300 (derived from the 17.12 tuning phase).

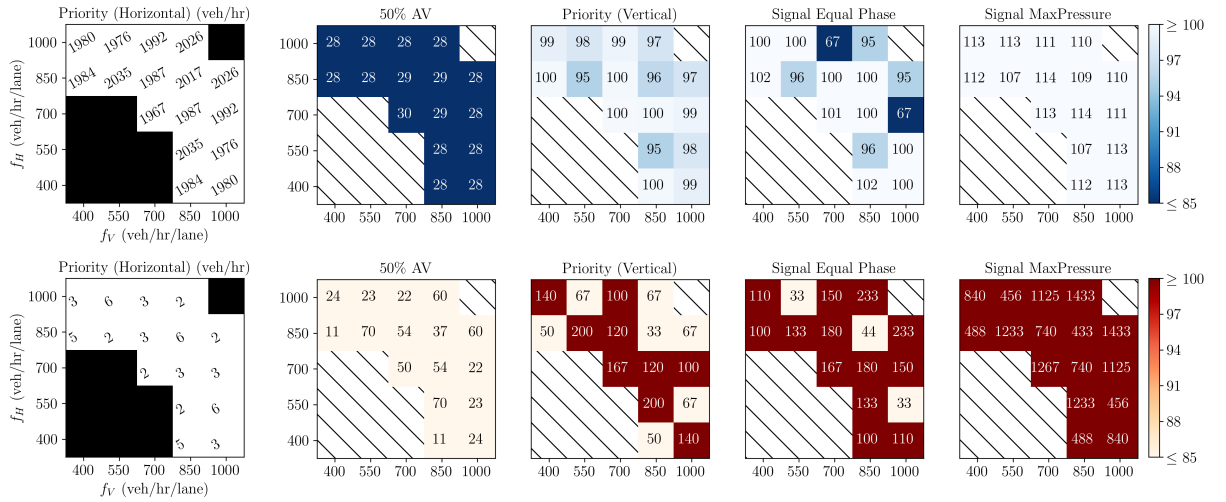


Figure 47: Evaluation of Model 65 (derived from the 02.02 tuning phase).

this model drops to near zero across both training and evaluation phases. This model encapsulates the "parking" phenomenon, proving that mathematical perfection in safety can easily be achieved by paralyzing the transportation network.

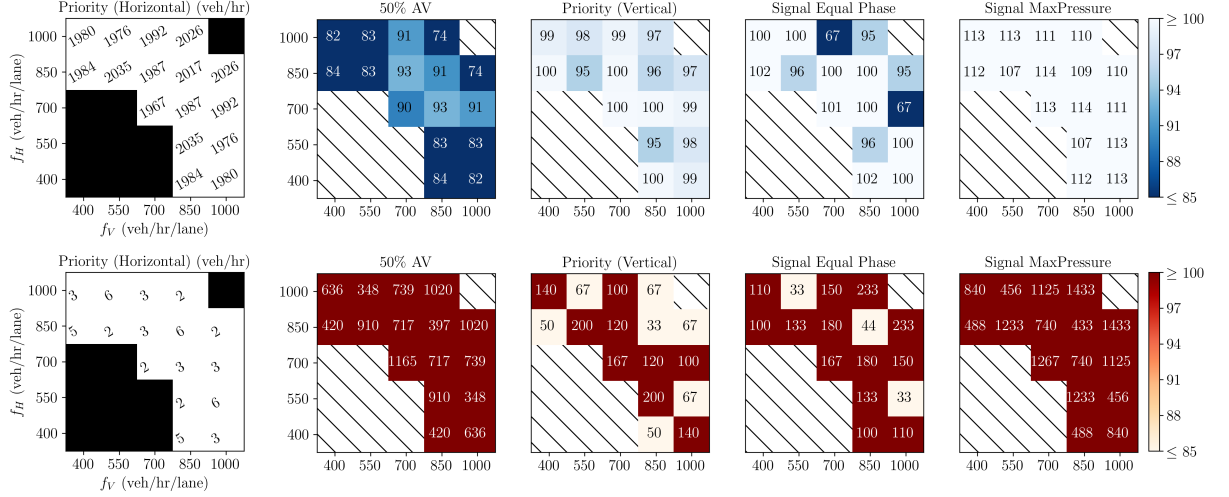


Figure 48: Evaluation of Model 100 (derived from the 13.02 tuning phase).

Transitioning to the 13.02 tuning phase, Model 100 presents a distinctly suboptimal equilibrium, as depicted in Figure 48. Macroscopically, the hourly throughput is reduced by 10% relative to the baseline. Theoretically, this reduction in speed should correlate with an increase in safety, as observed in the 02.02 configuration. However, Model 100 fails to secure this benefit; despite moving slower and yielding less throughput, the collision rates remain unacceptably high. This indicates a severe failure in the network's spatial reasoning. The agents are driving inefficiently, yet still failing to accurately predict and avoid conflicts with the uncontrolled human drivers, representing a worst-case scenario in the Pareto optimization landscape.

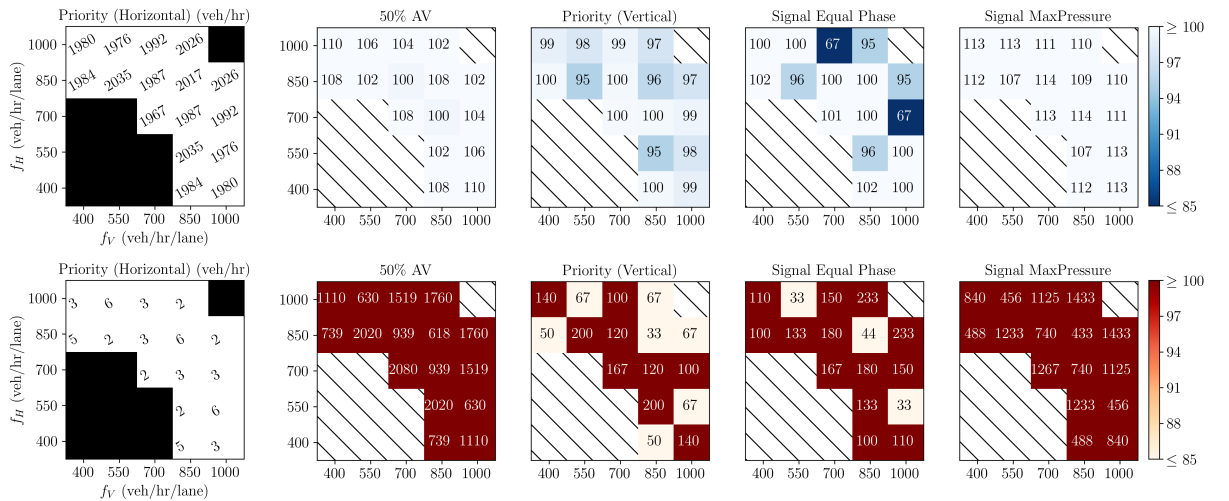


Figure 49: Evaluation of Model 350 (an advanced checkpoint from the 13.02 tuning phase).

Finally, evaluating a later checkpoint from the same advanced architectural series—Model 350 from the 13.02 tuning phase—reveals the inverse behavioral extreme. As shown in Figure 49, this policy successfully overcome the established performance ceiling, operating with an hourly throughput that is 8% *higher* than the traditional rule-based baseline. By leveraging advanced architectural features (such as self-attention mechanisms), the autonomous agents maximize intersection utilization, threading vehicles through infinitesimally small temporal gaps. However, the evaluation clearly exposes the fatal flaw of this hyper-efficient policy: the maximization of throughput is achieved through outright kinematic recklessness. The safety constraints are completely overridden by the drive for volume, resulting in an extreme, unacceptable surge in vehicular collisions. This model confirms that while the architecture is physically capable of outperforming baseline flow, it cannot currently do so without sacrificing systemic safety.

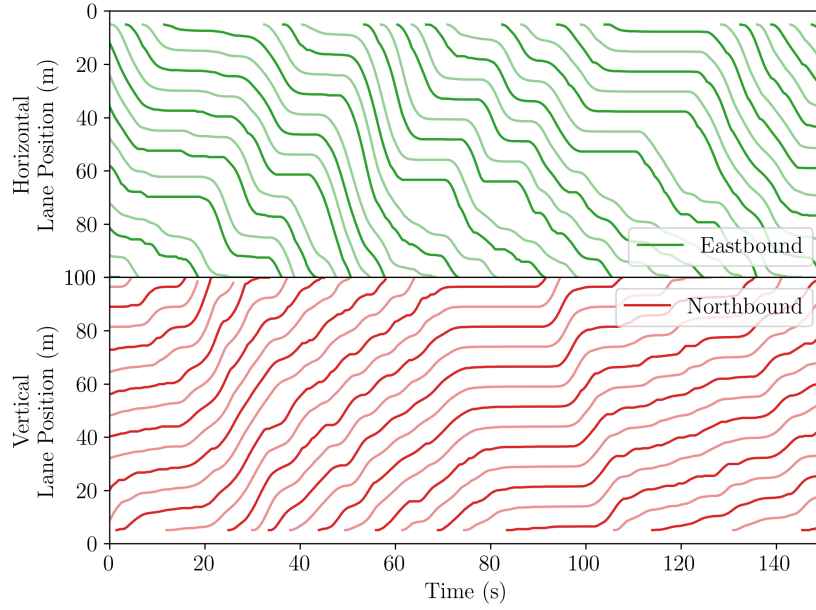


Figure 50: Time-space (trajectory) diagram illustrating the kinematic behavior of the Transformer architecture (Model 350) under a symmetric 700×700 flow configuration.

The time-space diagram in Figure 50 is the visualization is segmented into two distinct panels, each representing the a specific approaching lane. The trajectories of cooperative Autonomous Vehicles (AVs) are denoted by thick lines, while the paths of uncontrolled, human-driven vehicles are represented by normal lines. The diagram show that, the AVs act as a guidance, leading the following human driver in that lane.

4.5 Robustness

As previously established in Section 2.4, stochastic packet loss is an inevitable reality of Vehicle-to-Vehicle (V2V) communication networks. In reality, mixed-autonomy environments, factors such as physical occlusion, signal attenuation, and radio frequency

interference frequently cause observation data to be delayed or dropped entirely. To critically evaluate the operational resilience of the trained policies in a real-world deployment scenario, I designed a stress-testing framework for this model. Specifically, the highest-performing architecture—the Embedded Transformer (Model 350 from the 13.02 tuning phase)—was subjected to synthetic, uniformly distributed packet drop rates of 5%, 10%, 20%, and 50% across its observation space.

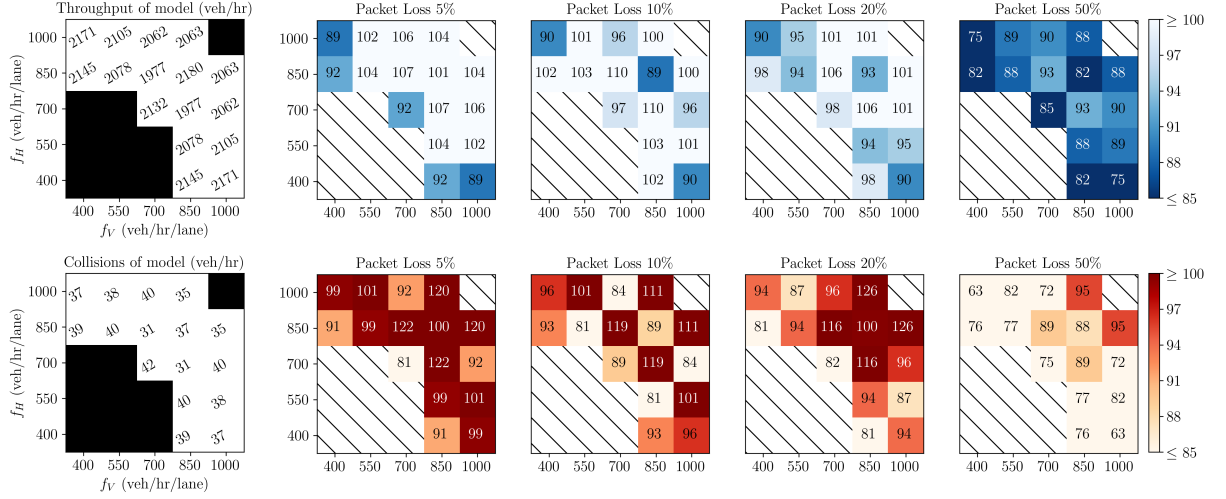


Figure 51: Performance evaluation of the Embedded Transformer architecture (Model 350) subjected to simulated V2V communication degradation.

The empirical results of this robustness evaluation, detailed in Figure 51, reveal a fascinating and highly counterintuitive non-linear degradation profile within the model’s throughput performance. Surprisingly, the model exhibits its most inefficient behavior under the lowest threshold of interference. At a minimal packet drop rate of 5%, the hourly throughput experiences a noticeable decrease of approximately 8% relative to the baseline. However, as the packet loss rate increases to 10% and 20%, the network’s throughput actually recovers, stabilizing with only a marginal 2% to 3% drop in overall flow.

This anomaly suggests a unique dynamic within the Transformer’s self-attention processing. At a 5% loss rate, the dropped packets introduce sporadic, low-frequency noise that disrupts the agent’s fine-grained kinematic predictions, leading to hesitation. Conversely, at sustained 10% and 20% loss rates, the network is forced to rely on more generalized, temporally stable features rather than hyper-optimizing for micro-gaps. This forces a slightly smoother, highly robust driving policy that easily absorbs the missing data without collapsing the traffic flow. Ultimately, this confirms that the proposed architecture is highly robust against moderate V2V signal degradation, maintaining baseline-comparable efficiency even when one-fifth of all communication is lost.

Expectedly, the network’s resilience has a finite limit. When subjected to the extreme 50% packet loss configuration, the spatial perception of the autonomous agents fundamentally fractures, resulting in a severe throughput degradation ranging from 15% to 25% depending on the specific intersection flow baseline. Under these conditions, the

agents lack the critical mass of continuous data required to safely orchestrate cooperative maneuvers, leading to systemic kinematic gridlock. However, it is essential to contextualize this failure mode: a persistent 50% data loss represents a systemic communication failure, such as a localized jamming event or hardware malfunction, rather than standard environmental interference. The fact that the model continues to function and process traffic at all under such severe informational deprivation further underscores the baseline stability of the Transformer architecture.

5 Conclusion and Future Work

5.1 Summary

My research investigated the application of Multi-Agent Reinforcement Learning for the coordination of mixed-autonomy traffic at unsignalized intersections. By evaluating diverse neural architectures, observation space topologies, and reward functions, my thesis shows the basic conditions of autonomous traffic management. The experiences of these experiments give me good insights about how the agents work.

Firstly, the empirical evaluations demonstrate that advanced architectural frameworks, specifically the Embedded Transformer utilizing K-Nearest Neighbor (KNN) state representations, have a substantial potential for maximizing throughput. The self-attention mechanism achieves higher throughput over established baselines, to over 2200 vehicles per hour. Furthermore, this architecture exhibited resilience to environmental degradation, maintaining operational stability even when subjected to Vehicle-to-Vehicle (V2V) communication packet loss rates of up to 20% across various flow configurations. However, this architectural maximization is limited by an inability to satisfy safety constraints. The efficient throughput was linked to unacceptable collision rates exceeding 30 incidents per hour, indicating that raw attention-based gap exploitation is currently not yet suitable for direct deployment without further safety constraints. On the other hand, by shifting the reward parameters toward extreme conservatism, the Transformer demonstrated the capacity to achieve very low collision rates under the tested conditions: the conservative model processing 700 vehicles per hour achieved a zero-collision throughout both extensive training and highly dense evaluation scenarios.

Secondly, the results suggest that traditional Multi-Layer Perceptron architectures, despite their lack of dynamic spatial attention, provide a balance between network throughput and systemic safety. By extracting generalized kinematic rules from the flattened observation space, the MLP successfully stabilized the learning policy, leading to balanced and stable traffic flows. Under optimal tuning, the MLP architecture achieved a throughput of approximately 1700 vehicles per hour with zero or less than 1 collisions, and scaled to 1800 vehicles per hour while suffering only negligible conflict rates (1 to 2 collisions).

Thirdly, from a deployment perspective, this thesis should acknowledge the sim-to-real gap regarding human-machine interaction. Deploying these trained autonomous policies in a real-world mixed-autonomy environment would induce a distribution shift in human driver behavior. Current Intelligent Driver Models (IDM) assume a relatively static heuristic for human reactions. However, in reality, human drivers are not yet accustomed to being actively “guided” or dynamically paced by cooperative Autonomous Vehicles (AVs), particularly in the absence of traditional traffic signals. The extreme behaviors observed in the reinforcement learning models, such as the hyper-conservative driving behavior of the 700 vehicle-per-hour policy, could induce confusion, unpredictable braking, or aggressive behavior by human drivers, thereby challenging the assumptions of the trained policy.

Finally, the results within my thesis represent a step toward the vision of next-generation

transportation infrastructures. By reducing reliance on traffic signals, my work contributes to a future that may include fully unsignalized intersections characterized by continuous free flow, minimized emissions, significantly reduced vehicle power consumption, and the a long-term reduction in collisions.

5.2 Future Work

While my research establishes a foundation for unsignalized intersection coordination, several points remain for future exploration to bridge the gap between simulation and real-world deployment.

A primary limitation of the current methodological framework lies within the constraints of the defined action space. The current policies execute control through a constrained or discretized longitudinal acceleration profile. Future iterations of this research must focus on migrating the control schema to a fully continuous, high-resolution, multi-dimensional action space that integrates both longitudinal acceleration and lateral steering control. Providing the agents with more suitable kinematic control may help to reduce the collisions observed in the high-throughput Transformer models, allowing vehicles to execute evasive maneuvers rather than relying solely on hard deceleration.

Furthermore, future research should address the behavioral fidelity of the uncontrolled vehicles. Replacing my methodologies with deep Imitation Learning (IL) or Inverse Reinforcement Learning (IRL) paradigms would allow the simulation environment to take in massive datasets of real-world human driving trajectories. By training the AVs against highly accurate, stochastic models of actual human hesitation, aggression, and error, the resulting RL policies will be more robust against the distribution shifts anticipated during real-world deployment.

Finally, the rapid advancement of Large Language Models (LLMs) presents a advantage for mixed-autonomy coordination. Future architectures could utilize the localized reinforcement learning policies developed in this thesis as low-level kinematic controllers, while deploying LLMs as a high-level strategic semantic layer. These models could utilize naturalistic reasoning to interpret complex geometric structure, predict human intent from visual, and negotiate right-of-way in a manner that is more natural and predictable for human drivers sharing the intersection.

References

- [1] 3GPP. TS 23.501: System architecture for the 5G System (5GS). Technical Specification TS 23.501, 3GPP, 2020. URL <https://www.3gpp.org/>.
- [2] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety, 2016. URL <https://arxiv.org/abs/1606.06565>. Introduced “reward hacking” as a key AI safety problem.
- [3] Mehdi Bennis, Mérouane Debbah, and H. Vincent Poor. Ultra-reliable and low-latency wireless communication: Tail, risk, and scale. *Proceedings of the IEEE*, 106(10):1834–1853, 2018. doi: 10.1109/JPROC.2018.2867029. URL <https://arxiv.org/abs/1801.01270>.
- [4] Craig Boutilier. Planning, learning and coordination in multiagent decision processes. In *Proceedings of the Conference on Theoretical Aspects of Rationality and Knowledge (TARK)*, volume 96, pages 195–210. Citeseer, 1996.
- [5] Leo Breiman. The poisson tendency in traffic distribution. *The Annals of Mathematical Statistics*, 34(1):308–311, 1963.
- [6] Mario H. Castañeda Garcia, Alejandro Molina-Galan, Mate Boban, Apostolos Kousaridas, Daniel Grimm, Toktam Mahmoodi, and Tommy Svensson. A tutorial on 5g NR V2X communications. *IEEE Communications Surveys & Tutorials*, 23(3):1972–2026, 2021. doi: 10.1109/COMST.2021.3057017. URL <https://arxiv.org/abs/2102.04538>.
- [7] Kurt Dresner and Peter Stone. A multiagent approach to autonomous intersection management. *Journal of Artificial Intelligence Research*, 31:591–656, 2008. doi: 10.1613/jair.2502. URL <https://www.jair.org/index.php/jair/article/view/10529>.
- [8] Shiv Ram Dubey, Satish Kumar Singh, and Bidyut Baran Chaudhuri. Activation functions in deep learning: A comprehensive survey and benchmark. *Neurocomputing*, 2022. doi: 10.1016/j.neucom.2021.08.068. URL <https://arxiv.org/abs/2109.14545>.
- [9] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(61):2121–2159, 2011. URL <https://jmlr.org/papers/v12/duchi11a.html>.
- [10] Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep RL: A case study on PPO and TRPO. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=r1etN1rtPB>.
- [11] Jakob Erdmann. Sumo’s lane-changing model. In *Modeling Mobility with Open Data: 2nd SUMO Conference 2014 Berlin, Germany, May 15-16, 2014*, pages 105–123. Springer, 2015.

-
- [12] Daniel L. Gerlough and André Schuhl. Use of poisson distribution in highway traffic. the probability theory applied to distribution of vehicles on two-lane highways. Research paper, Eno Foundation for Highway Traffic Control, January 1 1955. URL <https://rosap.ntl.bts.gov/view/dot/16299>. Accessed: 2026-02-05.
 - [13] Jayesh K. Gupta, Maxim Egorov, and Mykel J. Kochenderfer. Cooperative multi-agent control using deep reinforcement learning. In *Proceedings of the Adaptive Learning Agents Workshop at the 16th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2017)*, pages 66–83. Springer, 2017.
 - [14] Ye Han, Lijun Zhang, Dejian Meng, Xingyu Hu, and Yixia Lu. SPformer: A transformer based DRL decision making method for connected automated vehicles. arXiv preprint arXiv:2409.15105, 2024. URL <https://arxiv.org/abs/2409.15105>.
 - [15] John T. Hancock and Taghi M. Khoshgoftaar. Survey on categorical data for neural networks. *Journal of Big Data*, 7(1):1–41, 2020. doi: 10.1186/s40537-020-00305-w. URL <https://doi.org/10.1186/s40537-020-00305-w>.
 - [16] Ammar Haydari and Yasin Yilmaz. Deep reinforcement learning for intelligent transportation systems: A survey. *IEEE Transactions on Intelligent Transportation Systems*, 23(1):11–32, 2022. doi: 10.1109/TITS.2020.3008612. URL <https://arxiv.org/abs/2005.00935>.
 - [17] P. B. Hunt, D. I. Robertson, R. D. Bretherton, and R. I. Winton. SCOOT: A traffic responsive method of coordinating signals. Laboratory Report 1014, Transport and Road Research Laboratory (TRRL), 1981. URL <https://books.google.com/books?id=jUH6zAEACAAJ>.
 - [18] IEEE. IEEE standard for information technology–local and metropolitan area networks–specific requirements–part 11: Wireless lan medium access control (MAC) and physical layer (PHY) specifications amendment 6: Wireless access in vehicular environments, 2010.
 - [19] Ian T. Jolliffe. *Principal Component Analysis*. Springer, 2 edition, 2002. doi: 10.1007/b98835. Explained variance / proportion of variance explained by principal components.
 - [20] John B. Kenney. Dedicated short-range communications (dsrc) standards in the united states. *Proceedings of the IEEE*, 99(7):1162–1182, 2011. doi: 10.1109/JPROC.2011.2132790. DSRC operates in the 5.9 GHz band (5.850–5.925 GHz).
 - [21] Taimur S. Khan, Dieter Pfoser, S. Ruan, et al. Simplifying traffic simulation — from euclidean distances to agent-based models. *Computational Urban Science*, 4(1):32, 2024. doi: 10.1007/s43762-024-00145-x. URL <https://doi.org/10.1007/s43762-024-00145-x>.
 - [22] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. URL <http://arxiv.org/abs/1412.6980>.

-
- [23] Solomon Kullback and Richard A. Leibler. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951. doi: 10.1214/aoms/1177729694. URL <https://projecteuclid.org/journals/annals-of-mathematical-statistics/volume-22/issue-1/On-Information-and-Sufficiency/10.1214/aoms/1177729694.full>.
 - [24] Yann LeCun, Léon Bottou, Genevieve B. Orr, and Klaus-Robert Müller. Efficient backprop. In *Neural Networks: Tricks of the Trade*, pages 9–50. Springer, 1998. doi: 10.1007/3-540-49430-8_2. URL <http://yann.lecun.com/exdb/publis/pdf/lecun-98b.pdf>.
 - [25] Wei Li et al. AADS: Augmented autonomous driving simulation using data-driven algorithms. In *Science Robotics*, 2019. doi: 10.1126/scirobotics.aaw0863.
 - [26] M. J. Lighthill and G. B. Whitham. On kinematic waves. II. a theory of traffic flow on long crowded roads. *Proc. Roy. Soc. Lond. A*, 229:317–345, 1955.
 - [27] Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, L Lücken, Johannes Rummelhard, Peter Semmler, and Bajczyk Wiessner. Microscopic traffic simulation using sumo. In *The 21st IEEE International Conference on Intelligent Transportation Systems*, pages 2575–2582. IEEE, 2018.
 - [28] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL <https://arxiv.org/abs/1706.02275>.
 - [29] P. R. Lowrie. SCATS, sydney co-ordinated adaptive traffic system: A traffic responsive method of controlling urban traffic, 1990. URL <https://books.google.com/books?id=V4PTtgAACAAJ>.
 - [30] Lu Lu, Yeonjong Shin, Yanhui Su, and George Em Karniadakis. Dying ReLU and initialization: Theory and numerical examples. *Communications in Computational Physics*, 28(5):1671–1706, 2020. doi: 10.4208/cicp.OA-2020-0165. URL <https://arxiv.org/abs/1903.06733>.
 - [31] Andrew L. Maas, Awni Y. Hannun, and Andrew Y. Ng. Rectifier nonlinearities improve neural network acoustic models. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, 2013. URL https://ai.stanford.edu/~amaas/papers/relu_hybrid_icml2013_final.pdf.
 - [32] A. Parks-Young and Guni Sharon. Intersection management protocol for mixed autonomous and human-operated vehicles. *IEEE Transactions on Intelligent Transportation Systems*, 2022. doi: 10.1109/TITS.2022.315629. URL <https://arxiv.org/abs/2204.07704>.
 - [33] P. I. Richards. Shock waves on the highway. *Operations Research*, 4(1):42–51, 1956.

-
- [34] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986-10. doi: 10.1038/323533a0. URL <https://www.nature.com/articles/323533a0>.
 - [35] Dominik Salles, Stefan Kaufmann, and Hans-Christian Reuss. Extending the intelligent driver model in SUMO and verifying the drive off trajectories with aerial measurements. In *Proceedings of the SUMO User Conference 2020*. DLR, 2020-10. URL https://sumo.dlr.de/2020/SUMO2020_paper_28.pdf. Paper 28.
 - [36] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1889–1897. PMLR, 2015. URL <https://proceedings.mlr.press/v37/schulman15.html>.
 - [37] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
 - [38] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
 - [39] Joar Skalse, Nikolaus Howe, Dmitrii Krashenninnikov, and David Krueger. Advances in neural information processing systems. In *Advances in Neural Information Processing Systems*, 2022.
 - [40] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *Proceedings of the 30th International Conference on Machine Learning (ICML 2013)*, pages 1139–1147, 2013. URL <https://proceedings.mlr.press/v28/sutskever13.html>.
 - [41] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT press, second edition, 2018.
 - [42] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in neural information processing systems*, volume 12, 1999.
 - [43] I. S. Thompson et al. Entity-based reinforcement learning for autonomous cyber defence. *arXiv preprint arXiv:2410.17647*, 2024. URL <https://arxiv.org/abs/2410.17647>.
 - [44] Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural networks for machine learning, 2012.
 - [45] Martin Treiber, Ansgar Hennecke, and Dirk Helbing. Congested traffic states in empirical observations and microscopic simulations. *Physical review E*, 62(2):1805, 2000.

-
- [46] N. Trkulja, R. A. Berry, and D. Starobinski. Denial-of-service attacks on C-V2X networks. In *Proceedings of the 4th Workshop on Security and Privacy for Connected Autonomous Vehicles (AutoSec)*, 2021. URL https://www.ndss-symposium.org/wp-content/uploads/autosec2021_23006_paper.pdf.
 - [47] Pravin Varaiya. Max pressure control of a network of signalized intersections. *Transportation Research Part C: Emerging Technologies*, 36:177–195, 2013-11. doi: 10.1016/j.trc.2013.08.014. URL <https://www.sciencedirect.com/science/article/pii/S0968090X13001782>.
 - [48] Harsha Vasudev, Varad Deshpande, Sajal K. Das, and S. K. Das. A lightweight mutual authentication protocol for V2V communication in internet of vehicles. *IEEE Transactions on Vehicular Technology*, 69(6):6709–6717, 2020. doi: 10.1109/TVT.2020.2986585.
 - [49] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.
 - [50] Hongjun Wang, Jiyuan Chen, Tong Pan, Zheng Dong, Lingyu Zhang, Renhe Jiang, and Xuan Song. STGformer: Efficient spatiotemporal graph transformer for traffic forecasting. *arXiv preprint arXiv:2410.00385*, 2024. URL <https://arxiv.org/abs/2410.00385>.
 - [51] Axel Wegener, Michal Piórkowski, Maxim Raya, Horst Hellbrück, Stefan Fischer, and Jean-Pierre Hubaux. Traci: an interface for coupling road traffic and network simulators. In *Proceedings of the 11th communications and networking simulation symposium*, pages 155–163, 2008.
 - [52] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.
 - [53] Cathy Wu, Aboudy Kreidieh, Eugene Vinitzky, and Alexandre M. Bayen. Emergent behaviors in mixed-autonomy traffic. In *Proceedings of the 1st Annual Conference on Robot Learning*, volume 78 of *Proceedings of Machine Learning Research*, pages 398–407, 2017. URL <https://proceedings.mlr.press/v78/wu17a.html>.
 - [54] Zhongxia Yan and Cathy Wu. Reinforcement learning for mixed autonomy intersections. In *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*, pages 2089–2094. IEEE, 2021. doi: 10.1109/ITSC48978.2021.9565000.
 - [55] Hongxiang Zeng, Yifan Wang, Xumiao Xu, et al. VehCom: Delay-guaranteed message broadcast for large-scale vehicular networks. *IEEE Transactions on Vehicular Technology*, 2021. doi: 10.1109/TVT.2021.someDOI. URL <https://arxiv.org/abs/2103.XXXXX>. Experimental results show packet loss rates up to 6.2% on highways.